



UNIVERSITAS DIPONEGORO

**OPTIMASI RESPONSE TIME DALAM BACKEND SISTEM INFORMASI
SUMBER DAYA MANUSIA BERBASIS LARAVEL
(Studi Kasus di CV Mebel Internasional Semarang)**

TUGAS AKHIR

Diajukan sebagai salah satu syarat untuk memperoleh gelar Sarjana Teknik

**DJIE VALENCIA SANTOSO
21120121130055**

**DEPARTEMEN TEKNIK KOMPUTER
FAKULTAS TEKNIK
UNIVERSITAS DIPONEGORO
SEMARANG**

2025

HALAMAN PENGESAHAN

Tugas Akhir ini diajukan oleh:

Nama : Djie Valencia Santoso
NIM : 21120121130055
Departemen : Teknik Komputer
Judul Tugas Akhir : **Optimasi *Response Time* dalam *Backend* Sistem Informasi Sumber Daya Manusia Berbasis Laravel (Studi Kasus di CV Mebel Internasional Semarang)**

Telah berhasil dipertahankan di hadapan Tim Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Teknik pada Departemen Teknik Komputer, Fakultas Teknik, Universitas Diponegoro.

TIM PENGUJI

Pembimbing I : Rinta Kridalukmana S.Kom., M.T., PhD. ()
Pembimbing II : Bellia Dwi Cahya Putri, S.T., M.T. ()
Ketua Penguji : Yudi Eko Windarto, S.T., M. Kom. ()
Anggota Penguji : Arseto Satriyo Nugroho, S.T., M.Eng. ()

Semarang, 24 Maret 2025

Ketua Departemen Teknik Komputer



Dr. Oky Dwi Nurhayati, S.T., M.T.

NIP. 197910022009122001

HALAMAN PERNYATAAN ORISINALITAS

Tugas Akhir ini adalah hasil karya saya sendiri dan semua sumber baik yang dikutip maupun yang dirujuk telah saya nyatakan dengan benar.

Nama : Djie Valencia Santoso

NIM : 21120121130055

Tanda Tangan : 

Tanggal : Semarang, 10 Maret 2025

**HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS
TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS**

Sebagai sivitas akademik Universitas Diponegoro, saya yang bertanda tangan di bawah ini:

Nama : Djie Valencia Santoso
NIM : 21120121130055
Departemen : TEKNIK KOMPUTER
Fakultas : TEKNIK
Jenis Karya : TUGAS AKHIR

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Diponegoro **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya berjudul:

Optimasi *Response Time* dalam *Backend* Sistem Informasi Sumber Daya Manusia Berbasis Laravel (Studi Kasus di CV Mebel Internasional Semarang)

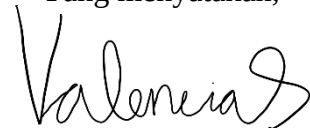
beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti/Noneksklusif ini Universitas Diponegoro berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat dan memublikasikan Tugas Akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Semarang

Pada tanggal : 10 Maret 2025

Yang menyatakan,



(Djie Valencia Santoso)

KATA PENGANTAR

Segala puji dan syukur penulis panjatkan kepada Tuhan Yesus Kristus atas kasih dan anugerah-Nya, sehingga penulis dapat menyelesaikan laporan Tugas Akhir yang berjudul **“Optimasi Response Time dalam Backend Sistem Informasi Sumber Daya Manusia Berbasis Laravel (Studi Kasus di CV Mebel Internasional Semarang)”**.

Laporan ini disusun sebagai salah satu persyaratan untuk menyelesaikan studi serta memenuhi kewajiban akademik di Departemen Teknik Komputer, Fakultas Teknik, Universitas Diponegoro.

Dalam proses penyusunan laporan ini, penulis mendapatkan banyak dukungan, bimbingan, serta bantuan dari berbagai pihak. Oleh karena itu, dengan penuh rasa hormat dan terima kasih, penulis ingin menyampaikan apresiasi yang sebesar-besarnya kepada:

1. Bapak Rinta Kridalukmana S.Kom., M.T., PhD., selaku dosen pembimbing I, yang telah memberikan bimbingan, saran, arahan, serta meluangkan waktu untuk mendampingi penulis dalam penyusunan Tugas Akhir ini.
2. Ibu Bellia Dwi Cahya Putri, S.T., M.T., selaku dosen pembimbing II sekaligus pemangku kepentingan, yang dengan penuh kesabaran telah memberikan bimbingan, saran, arahan, serta dukungan waktu kepada penulis selama proses pengerjaan Tugas Akhir ini.
3. Ibu Dr. Oky Dwi Nurhayati, ST, MT, selaku Ketua Departemen Teknik Komputer Universitas Diponegoro, yang telah memberikan arahan dan dukungan akademik kepada penulis.
4. Bapak Ilmam Fauzi Hashbil Alim, S.T., M.Kom., selaku Koordinator Tugas Akhir, yang telah mengatur dan membimbing mahasiswa dalam pelaksanaan tugas akhir dengan baik.
5. Seluruh Bapak dan Ibu dosen di Jurusan Teknik Komputer, yang telah berbagi ilmu, pengalaman, serta wawasan berharga selama masa perkuliahan penulis.

6. Seluruh *staff* tata usaha dan tenaga kependidikan Departemen Teknik Komputer, yang telah menjalankan tugas dengan baik serta membantu dalam berbagai keperluan administrasi selama masa studi penulis.
7. Kedua orang tua penulis, yang selalu memberikan dukungan moral maupun materi, kasih sayang yang tak terhingga, doa yang tulus, serta kesabaran yang luar biasa dalam setiap langkah kehidupan penulis.
8. Ketiga saudara penulis, yang senantiasa memberikan semangat, dukungan moral, doa, serta kasih sayang dalam perjalanan akademik penulis.
9. Abdul Rozzaq yang telah membantu dan menemani Penulis dalam menyelesaikan Tugas Akhir ini.
10. Teman-teman terdekat, khususnya Syadza Indira Prameswari, Diana Nur Ariva, Vane Karenina Thetan Hartono, dan teman-teman *awardees* IISMA TU Dresden yang telah memberikan dukungan, membantu, dan menemani penulis dalam menjalani masa perkuliahan hingga penyusunan Tugas Akhir ini.
11. Rekan-rekan mahasiswa Jurusan Teknik Komputer angkatan 2021 yang telah berbagi pengalaman, dukungan, serta kebersamaan dalam menghadapi berbagai tantangan selama masa kuliah hingga penyelesaian Tugas Akhir ini.
12. Seluruh pihak yang telah berkontribusi dalam berbagai bentuk, baik secara langsung maupun tidak langsung, dalam membantu kelancaran penyusunan Tugas Akhir ini, yang tidak dapat disebutkan satu per satu.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar dapat menjadi bahan perbaikan ke depan. Semoga laporan ini dapat memberikan manfaat bagi pembaca serta menjadi referensi yang berguna dalam bidang pengembangan sistem informasi.

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN	ii
HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS.....	iv
KATA PENGANTAR.....	v
DAFTAR ISI.....	vii
DAFTAR GAMBAR.....	ix
DAFTAR TABEL	xi
ABSTRAK	xii
ABSTRACT	xiii
BAB I PENDAHULUAN.....	2
1.1 Latar Belakang.....	2
1.2 Rumusan Masalah	5
1.3 Tujuan Penelitian.....	6
1.4 Batasan Masalah	6
1.5 Manfaat Penelitian.....	7
1.6 Metodologi Penelitian	8
1.7 Sistematika Penulisan.....	10
BAB II KAJIAN PUSTAKA	12
2.1 Kajian Penelitian Terdahulu.....	12
2.2 Landasan Teori	19
2.2.1 Sumber Daya Manusia	19
2.2.1 Sistem Informasi	19
2.2.2 Laravel.....	20
2.2.3 Metode RAD	20
2.2.4 Postman.....	21
2.2.5 Whitebox Testing	22
2.2.6 Load Testing.....	22
BAB III PERANCANGAN SISTEM	23
3.1 Analysis and Quick Design.....	24
3.1.1 Kebutuhan Fungsional Sistem	24
3.1.2 Kebutuhan Non-Fungsional Sistem.....	26
3.1.3 Karakteristik Pengguna	26

3.1.4	Arsitektur Sistem.....	27
3.2	Perancangan Sistem.....	30
3.2.1	Gambaran Umum Sistem Saat Ini	30
3.2.2	Target dari Sistem yang Dikembangkan.....	34
3.2.3	Perancangan Diagram Use Case.....	45
3.2.4	Perancangan Skenario Use Case	47
3.2.5	Perancangan Basis Data	54
3.2.6	Perancangan Endpoint.....	56
3.3	Metode Pengujian	58
BAB IV HASIL DAN PEMBAHASAN		60
4.1	Implementasi Sistem	60
4.1.1	Implementasi Basis Data	60
4.1.2	Konfigurasi Routes	62
4.1.3	Konfigurasi Middlewares.....	64
4.1.4	Konfigurasi Controllers	68
4.1.5	Deployment.....	75
4.2	Expert Feedback terhadap Implementasi Awal.....	77
4.3	Hasil Perbaikan Expert Feedback.....	79
4.4	Pengujian Aplikasi	82
4.4.1	Whitebox Testing	82
4.5	Review Kode	100
4.5.1	AuthController	100
4.5.2	UserController	100
4.5.3	PengumumanController	101
4.5.4	CutiPerizinanController	102
4.5.5	KalenderController	102
4.5.6	ShiftController	103
4.5.7	PayrollController	104
4.6	Hasil Implementasi dan Pengujian.....	104
BAB V PENUTUP		106
5.1	Kesimpulan.....	106
5.2	Saran	107
DAFTAR PUSTAKA		109
LAMPIRAN 1 BIODATA MAHASISWA		112
LAMPIRAN 2 SOURCE CODE		113

DAFTAR GAMBAR

Gambar 3.1 Arsitektur sistem berbasis <i>website</i>	27
Gambar 3.2 Arsitektur pengembangan <i>website</i>	28
Gambar 3. 3 Diagram BPMN proses bisnis manajemen SDM saat ini	30
Gambar 3. 4 Proses bisnis pengelolaan Sumber Daya Manusia	35
Gambar 3. 5 Proses bisnis pendataan karyawan	36
Gambar 3. 6 Proses bisnis penjadwalan kerja karyawan	37
Gambar 3. 7 Proses bisnis presensi dan pendataan presensi karyawan	38
Gambar 3. 8 Proses bisnis permohonan dan persetujuan izin dan cuti karyawan	39
Gambar 3. 9 Proses bisnis perhitungan gaji dan upah lembur	40
Gambar 3. 10 Proses bisnis pembuatan pengumuman dan informasi karyawan ..	41
Gambar 3.11 Diagram <i>use case</i> sistem berbasis <i>website</i>	46
Gambar 3. 12 ERD Sistem Informasi Sumber Daya Manusia di CV Mebel International Semarang	55
Gambar 4. 1 Skema tabel <i>users</i> pada <i>database migration</i> Laravel.....	61
Gambar 4. 2 <i>Seeder</i> untuk tabel <i>users</i>	62
Gambar 4. 3 <i>Routes</i> web.php	63
Gambar 4. 4 <i>Routes</i> api.php	63
Gambar 4. 5 <i>Middleware</i> Authenticate	64
Gambar 4. 6 <i>Middleware</i> IsAdmin.....	65
Gambar 4. 7 <i>Middleware</i> RedirectIfAuthenticated.....	65
Gambar 4. 8 <i>Middleware</i> VerifyCsrfToken	66
Gambar 4. 9 <i>Middleware</i> EncryptCookies.....	66
Gambar 4. 10 <i>Middleware</i> ValidateSignature.....	67
Gambar 4. 11 <i>Middleware</i> TrustProxies	67
Gambar 4. 12 <i>UserController</i>	68
Gambar 4. 13 <i>JabatanController</i>	69
Gambar 4. 14 <i>RiwayatJabatanController</i>	69
Gambar 4. 15 <i>GrupController</i>	70
Gambar 4. 16 <i>KantorController</i>	70
Gambar 4. 17 <i>AttendanceController</i>	71
Gambar 4. 18 <i>ShiftController</i>	72
Gambar 4. 19 <i>CutiPerizinanController</i>	73
Gambar 4. 20 <i>PayrollController</i>	74
Gambar 4. 21 <i>PengumumanController</i>	74
Gambar 4. 22 <i>HomeController</i>	75
Gambar 4. 23 <i>Dashboard monitoring</i> SISDM CV. MI pada <i>platform</i> Railway... ..	76
Gambar 4. 24 Tampilan <i>loading</i> data pengguna pada aplikasi <i>web</i> yang lambat. ..	77
Gambar 4. 25 Hasil <i>load testing</i> API SISDM MI sebelum optimasi.....	78
Gambar 4. 26 <i>Profiling query</i> halaman <i>users</i> menggunakan <i>Laravel Debugger</i> ..	78
Gambar 4. 27 Penerapan <i>eager loading</i> pada <i>UserController</i>	79
Gambar 4. 28 <i>Profiling query</i> halaman <i>users</i> setelah penerapan <i>eager loading</i> ..	80
Gambar 4. 29 Penerapan <i>pagination</i> pada <i>UserController</i>	80

Gambar 4. 30 <i>Profiling query</i> halaman <i>users</i> setelah <i>pagination</i>	80
Gambar 4. 31 Penerapan <i>indexing</i> pada skema <i>database migration users</i>	81
Gambar 4. 32 <i>Profiling query</i> halaman <i>users</i> setelah <i>indexing</i>	81
Gambar 4. 33 Hasil <i>Load Testing API Backend SISDM MI</i> setelah optimasi	82
Gambar 4. 34 <i>Output</i> mendaftar dengan <i>email</i> dan <i>password</i> yang valid	84
Gambar 4. 35 <i>Output</i> mendaftar dengan <i>email</i> tidak valid	84
Gambar 4. 36 <i>Output</i> mendaftar dengan <i>email</i> yang sudah terdaftar	85
Gambar 4. 37 <i>Output</i> mendaftar dengan <i>password</i> kurang dari 8 karakter	85
Gambar 4. 38 Diagram <i>cyclomatic complexity</i> halaman <i>login</i>	86
Gambar 4. 39 <i>Output login</i> dengan <i>email</i> dan <i>password</i> yang benar	87
Gambar 4. 40 <i>Output login</i> dengan <i>email</i> dan <i>password</i> yang tidak cocok.....	87
Gambar 4. 41 <i>Output login</i> dengan <i>email</i> yang tidak terdaftar.....	88
Gambar 4. 42 <i>Output</i> mendapatkan daftar semua pengguna	89
Gambar 4. 43 <i>Output</i> mendapatkan data pengguna berdasarkan ID yang ada	90
Gambar 4. 44 <i>Output</i> mendapatkan dan meng- <i>edit</i> data pengguna berdasarkan ID yang tidak ada	90
Gambar 4. 45 <i>Output</i> membuat pengumuman dengan semua <i>field</i> yang wajib diisi	92
Gambar 4. 46 <i>Output</i> membuat permohonan cuti/perizinan dengan semua <i>field</i> yang wajib diisi	94
Gambar 4. 47 <i>Output</i> mendapatkan daftar semua <i>event</i> kalender.....	95
Gambar 4. 48 <i>Output</i> mendapatkan daftar <i>shift</i> berdasarkan ID pengguna yang valid.....	97
Gambar 4. 49 <i>Output</i> mendapatkan daftar <i>shift</i> berdasarkan ID pengguna yang tidak ada	97
Gambar 4. 50 <i>Output</i> mendapatkan daftar <i>payroll</i> berdasarkan ID pengguna yang valid.....	99
Gambar 4. 51 <i>Output</i> mendapatkan daftar <i>payroll</i> berdasarkan ID pengguna yang tidak ada	99
Gambar 4. 52 <i>Output</i> mendapatkan <i>payroll</i> berdasarkan ID <i>payroll</i> yang valid ..	99
Gambar 4. 53 <i>Output</i> mendapatkan <i>payroll</i> berdasarkan ID <i>payroll</i> yang tidak ada	100
Gambar 4. 54 Hasil pengujian PDepend pada AuthController	100
Gambar 4. 55 Hasil pengujian PDepend pada UserController	100
Gambar 4. 56 Hasil pengujian PDepend pada PengumumanController	101
Gambar 4. 57 Hasil pengujian PDepend pada CutiPerizinanController	102
Gambar 4. 58 Hasil pengujian PDepend pada KalenderController	102
Gambar 4. 59 Hasil pengujian PDepend pada ShiftController	103
Gambar 4. 60 Hasil pengujian PDepend pada PayrollController	104

DAFTAR TABEL

Tabel 2.1 Perbandingan penelitian Ervina Rosa Aulia dengan penelitian Penulis	12
Tabel 2.2 Perbandingan penelitian Hakim, M.A., Triesia, D., et al. dengan penelitian Penulis	13
Tabel 2.3 Perbandingan penelitian A. Akbar and Ari dengan penelitian Penulis.	13
Tabel 2.4 Perbandingan Reza Ardianto dan Gunawan Budi Sulistyo dengan penelitian Penulis	14
Tabel 2.5 Perbandingan penelitian Hengki Agung Prayoga dengan penelitian Penulis	14
Tabel 2.6 Perbandingan penelitian Eko Saparnuriyan Putra dengan penelitian Penulis	15
Tabel 2.7 Perbandingan penelitian H. Mahwahulhusna dengan penelitian Penulis	15
Tabel 2.8 Perbandingan penelitian Sastra, R., Muhammad Rizki Akbar, & Dicky Hariyanto dengan penelitian Penulis.....	16
Tabel 2.9 Tabel kajian penelitian terdahulu	16
 Tabel 3.1 Tabel kebutuhan fungsional sistem.....	25
Tabel 3.2 Tabel kebutuhan non-fungsional sistem	26
Tabel 3. 3 Hubungan komponen arsitektur sistem berbasis website	27
Tabel 3. 4 Prosedur manajemen SDM saat ini	31
Tabel 3. 5 <i>Service time</i> dari proses manajemen SDM saat ini	32
Tabel 3. 6 Prosedur proses pengelolaan sumber daya manusia	41
Tabel 3. 7 <i>Target service time</i>	44
Tabel 3.8 Skenario <i>use case</i> HRD membuat dan mengatur akun <i>user</i> karyawan.	48
Tabel 3. 9 Skenario <i>use case</i> HRD membuat dan mengatur akun <i>user</i> karyawan	48
Tabel 3. 10 Skenario <i>use case</i> HRD mengelola informasi pengumuman perusahaan.....	49
Tabel 3. 11 Skenario <i>use case</i> HRD mengelola informasi data <i>history</i> presensi..	50
Tabel 3. 12 Skenario <i>use case</i> HRD mengatur lokasi presensi.....	51
Tabel 3. 13 Skenario <i>use case</i> HRD menyetujui/menolak permohonan cuti dan izin.....	51
Tabel 3. 14 Skenario <i>use case finance</i> menghitung gaji dan upah lembur	52
Tabel 3.15 Skenario <i>use case finance</i> mengakses informasi slip gaji karyawan ..	53
Tabel 3.16 <i>Endpoint</i> RESTful API	56
 Tabel 4. 1 Pengujian <i>whitebox</i> halaman <i>register</i>	83
Tabel 4. 2 Pengujian <i>whitebox</i> halaman <i>login</i>	85
Tabel 4. 3 Pengujian <i>whitebox</i> halaman <i>users</i>	88
Tabel 4. 4 Pengujian <i>whitebox</i> halaman pengumuman.....	90
Tabel 4. 5 Pengujian <i>whitebox</i> halaman pengajuan cuti/perizinan	92
Tabel 4. 6 Pengujian <i>whitebox</i> halaman kalender.....	94
Tabel 4. 7 Pengujian <i>whitebox</i> halaman <i>shifts</i>	95
Tabel 4. 8 Pengujian <i>whitebox</i> halaman <i>payroll</i>	97

ABSTRAK

Sistem Informasi Sumber Daya Manusia (SISDM) di CV Mebel Internasional Semarang dikembangkan untuk mengotomatisasi pengelolaan data karyawan, termasuk presensi, izin cuti, serta perhitungan gaji dan lembur. Penelitian ini berfokus pada pengembangan backend berbasis Laravel, yang menjadi pusat pengelolaan data dan layanan bagi HRD, Finance, serta karyawan melalui platform web dan aplikasi Android. Backend ini dirancang untuk mengelola data pengguna, memvalidasi presensi, serta memproses perhitungan gaji secara otomatis guna mengurangi kesalahan dalam proses manual.

Pengembangan backend menggunakan framework Laravel dengan menerapkan arsitektur Model-View-Controller (MVC), yang memisahkan logika bisnis, tampilan, dan manajemen data guna meningkatkan skalabilitas serta kemudahan pemeliharaan sistem. Komunikasi antara backend dan aplikasi Android dilakukan melalui RESTful API untuk memastikan integrasi data secara real-time. Metode Rapid Application Development (RAD) diterapkan untuk mempercepat iterasi pengembangan dan pengujian fitur sehingga sistem dapat dikembangkan secara adaptif sesuai kebutuhan pengguna.

Pengujian sistem dilakukan menggunakan metode white-box testing untuk menganalisis struktur kode, menguji endpoint API, serta memastikan keandalan dan efisiensi sistem. Selain itu, pengujian response time dilakukan untuk mengevaluasi performa backend dalam menangani beban kerja yang tinggi. Hasil pengujian menunjukkan bahwa backend Laravel yang dikembangkan mampu meningkatkan kecepatan pemrosesan data, mengurangi risiko kesalahan manual, serta meningkatkan transparansi dan efektivitas dalam pengelolaan sumber daya manusia di CV Mebel Internasional Semarang. Dengan optimasi ini, sistem mampu mendukung operasional perusahaan dengan performa yang lebih cepat, stabil, dan efisien.

Kata Kunci: *Sistem Informasi Sumber Daya Manusia, Laravel, RESTful API, Rapid Application Development, White-Box Testing, Response Time.*

ABSTRACT

The Human Resource Information System (HRIS) at CV Mebel Internasional Semarang was developed to automate employee data management, including attendance, leave requests, and payroll calculations. This research focuses on the development of a Laravel-based backend, serving as the central data management system and service provider for HR, Finance, and employees via a web platform and an Android application. The backend is designed to manage user data, validate attendance, and automate payroll processing to reduce errors commonly found in manual processes.

The backend development utilizes the Laravel framework with a Model-View-Controller (MVC) architecture to separate business logic, interface, and data management, thereby enhancing scalability and system maintainability. Communication between the backend and the Android application is facilitated through RESTful API integration, ensuring real-time data synchronization. The Rapid Application Development (RAD) method is implemented to accelerate development iterations and feature testing, allowing the system to be adaptively enhanced according to user needs.

System testing employs the white-box testing method to analyze code structure, verify API endpoints, and ensure system reliability and efficiency. Additionally, response time testing is conducted to evaluate the backend's performance under high workload conditions. The test results indicate that the optimized Laravel backend significantly improves data processing speed, minimizes manual errors, and enhances transparency and efficiency in human resource management at CV Mebel Internasional Semarang. With these optimizations, the system supports business operations with faster, more stable, and efficient performance.

Keywords: *Human Resource Information System, Laravel, RESTful API, Rapid Application Development, White-Box Testing, Response Time.*

BAB I

PENDAHULUAN

1.1 Latar Belakang

CV Mebel Internasional (MI) adalah perusahaan manufaktur furnitur ekspor berpengalaman yang berdiri sejak tahun 2004. MI melayani dua sektor yaitu *hospitality* dan retail, dengan fokus pada pembuatan furnitur pesanan untuk hotel, resor, *timeshare*, hingga retail. MI dikenal dengan komitmennya terhadap kualitas, keindahan, dan ketahanan furnitur, serta pengiriman tepat waktu. Dengan pengalaman memproduksi lebih dari 500.000 produk, MI saat ini menjadi salah satu produsen furnitur ekspor tertua di Indonesia dan terus mengembangkan kemampuan desain untuk memenuhi kebutuhan pasar global.

Di CV Mebel Internasional Semarang, pengelolaan sumber daya manusia (SDM) masih mengandalkan metode manual di berbagai aspek. Divisi *Human Resources Development* (HRD) memiliki tanggung jawab yang luas dalam mengelola SDM, mulai dari penjadwalan kerja hingga pengelolaan perizinan dan pengumuman.

Penjadwalan kerja masih dilakukan menggunakan papan tulis, sebuah pendekatan yang mungkin kurang efisien dan rentan terhadap kesalahan. Meskipun telah menggunakan mesin presensi yang dilengkapi fitur *face recognition*, data kehadiran karyawan masih diolah secara manual oleh HRD. Keadaan ini meningkatkan risiko ketidakakuratan dalam penghitungan upah dan lembur yang dilakukan oleh Divisi *Finance* menggunakan rumus-rumus *Excel* seperti yang tercantum pada Gambar 1.1.

Pegawai						Data scanlog				
PIN	NIP	Nama	Jabatan	Departemen	Kantor	Tanggal	Scan 1	Scan 2	Scan 3	Scan 4
6	6	Muslim		STAFF ALL IN		04-03-2024	06:18:19	15:05:00		
7	7	Ruyati		STAFF ALL IN		04-03-2024	07:21:27	15:36:37		
10	10	Ali Muchtar		STAFF ALL IN		04-03-2024	06:40:15	14:56:19		
21	21	Kasmonah		PR UMUM		04-03-2024	06:48:28	14:47:47		
23	23	Zaenal Arifin		STAFF ALL IN		04-03-2024	07:10:21	17:03:43		
29	29	Zamzuli		SANDING C		04-03-2024	06:52:01	14:46:21		
34	34	Sakroni		WIPING		04-03-2024	06:54:40	14:49:38		
35	35	Agus Hariyanto		SANDING B		04-03-2024	06:51:40	14:48:22		
37	37	Abdul Rokhim		SANDING D		04-03-2024	06:52:51	14:54:59		
39	39	Jayadi		SANDING A		04-03-2024	06:54:30	15:41:52		
41	41	Muhammad Samsudin		SANDING C		04-03-2024	06:47:13	14:48:18		
42	42	Ngationo		SANDING B		04-03-2024	06:54:37	14:47:12		
43	43	Khoirul Anwar		SANDING D		04-03-2024	06:52:54	14:51:15		
44	44	Mudianto		SANDING B		04-03-2024	06:48:53	14:46:51		
49	49	Sutoyo		TUKANG		04-03-2024	07:24:53	15:16:55		
50	50	Nur Fatoni		SANDING C		04-03-2024	06:48:57	14:46:38		
52	52	Sudarmanto		WIPING		04-03-2024	06:52:59	14:46:33		
54	54	Kamdi		SANDING A		04-03-2024	06:39:10	14:45:37		
57	57	Lut Setiawan		FINISHING PUTIH		04-03-2024	06:20:34	14:48:36		

Gambar 1. 1 Contoh data presensi CV Mebel Internasional

Proses perizinan dan cuti kerja bergantung pada aplikasi WhatsApp, yang dapat menyulitkan dalam manajemen data dan risiko kehilangan informasi. Pengumuman dan penyebaran informasi juga mengandalkan WhatsApp, menyebabkan masalah dalam manajemen informasi.

Masalah pengelolaan karyawan secara manual di CV Mebel Internasional Semarang, terutama dengan jumlah karyawan yang signifikan, menyebabkan kesulitan dalam pengambilan keputusan yang cepat, sering kali mengganggu produktivitas, dan meningkatkan risiko adanya *human error*.

Dari masalah tersebut, topik utama yang diangkat dalam proyek *capstone* ini yaitu, Pengembangan dan Implementasi Sistem Informasi Sumber Daya Manusia CV Mebel Internasional Semarang. Sistem informasi merupakan suatu sistem yang terdiri dari orang, proses, dan teknologi informasi yang saling berinteraksi untuk mengumpulkan, memproses, menyimpan, dan mendistribusikan informasi yang dibutuhkan untuk mendukung pengambilan keputusan dan operasi bisnis suatu organisasi atau perusahaan. Sistem informasi ini digunakan untuk mengolah data supaya menghasilkan informasi bagi para penggunanya. ^[1] Sistem ini terdiri dari dua *platform* utama, yaitu aplikasi Android untuk karyawan dan sistem berbasis web untuk HRD, *finance*, dan admin. Namun, dalam laporan ini, pembahasan akan difokuskan pada pengembangan *backend website* berbasis Laravel.

Backend ini akan menjadi inti dari sistem, bertanggung jawab untuk mengelola data karyawan, memproses informasi presensi, jadwal kerja,

perhitungan gaji, serta pengelolaan izin dan cuti. Laravel dipilih sebagai *framework backend* karena memiliki struktur arsitektur *Model-View-Controller* (MVC) yang memisahkan logika bisnis (*Controller*), tampilan antarmuka (*View*), dan pengelolaan data (*Model*). Selain itu, Laravel menawarkan fitur unggulan seperti *Eloquent ORM* untuk optimasi manajemen *database*, sistem autentikasi yang kuat, serta kemampuan integrasi API yang fleksibel. ^[2]

Pengembangan *backend* ini mengikuti metodologi *Rapid Application Development* (RAD), yang memungkinkan iterasi cepat antara desain, implementasi, pengujian, dan perbaikan berdasarkan umpan balik dari pengguna. Metode RAD telah terbukti efektif dalam mempercepat proses pengembangan sistem informasi akademik berbasis web. ^[3]

Dalam pengembangan sistem *backend* yang kompleks seperti yang diterapkan di CV Mebel Internasional Semarang, pengujian *whitebox* menjadi langkah yang krusial untuk memastikan keandalan serta keamanan kode sumber. *Whitebox testing* merupakan metode pengujian yang berfokus pada analisis struktur internal perangkat lunak untuk mengidentifikasi kesalahan logika, redundansi, serta potensi kerentanan keamanan. ^[4] Pengujian ini memungkinkan pengembang untuk memvalidasi setiap alur kontrol dan data dalam kode, memastikan bahwa semua kondisi telah diuji secara menyeluruh sebelum sistem diterapkan dalam lingkungan produksi.

Selain dari aspek keamanan dan validasi logika, performa sistem juga menjadi faktor kritis yang harus diuji dan dioptimalkan agar dapat memenuhi kebutuhan perusahaan skala internasional. Dalam konteks CV Mebel Internasional, sistem *backend* digunakan untuk mengelola berbagai aspek sumber daya manusia (SDM), seperti pencatatan presensi, penjadwalan kerja, serta pengelolaan izin dan cuti karyawan. Apabila waktu respons sistem terlalu lama, hal ini dapat menghambat operasional bisnis, menyebabkan keterlambatan dalam pemrosesan data, dan berdampak pada efisiensi kerja karyawan serta pengambilan keputusan oleh manajemen. ^[5] Oleh karena itu, setelah dilakukan pengujian *whitebox* untuk memastikan struktur kode yang optimal, langkah selanjutnya adalah melakukan optimasi *response time* berdasarkan hasil pengujian *load testing*.

Dengan penerapan pengujian *whitebox* untuk memastikan efisiensi dan keamanan kode, serta optimasi waktu respons berbasis *load testing*, sistem *backend* yang dikembangkan untuk CV Mebel Internasional Semarang dapat berjalan lebih optimal. Hal ini tidak hanya memastikan kelancaran alur kerja perusahaan tetapi juga mendukung operasional bisnis berskala global dengan performa sistem yang stabil, cepat, dan andal.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, permasalahan dalam penulisan TA ini dapat dirumuskan sebagai berikut.

1. Bagaimana mengembangkan sistem *backend* berbasis Laravel yang dapat mengotomatisasi pengelolaan data karyawan, termasuk pencatatan kehadiran, penjadwalan kerja, serta perizinan dan cuti, agar lebih akurat dan efisien dalam mendukung operasional perusahaan?
2. Bagaimana sistem *backend* dapat mengoptimalkan perhitungan upah dan lembur secara otomatis guna menghindari kesalahan perhitungan yang dapat mempengaruhi produktivitas dan kepuasan karyawan?
3. Bagaimana sistem informasi berbasis Laravel dapat diintegrasikan dengan aplikasi Android serta diuji keandalan dan kinerjanya berdasarkan waktu respons, tingkat akurasi data, serta kepuasan pengguna dalam mendukung kelancaran komunikasi dan pengambilan keputusan di perusahaan?
4. Bagaimana memastikan kualitas dan keandalan sistem *backend* yang dikembangkan melalui penerapan metode pengujian *whitebox testing*, sehingga setiap alur logika dalam kode dapat divalidasi untuk mencegah potensi kesalahan dan meningkatkan efisiensi sistem?
5. Bagaimana memastikan bahwa performa sistem *backend* telah optimal berdasarkan pengujian *response time* guna memastikan kecepatan pemrosesan data yang mendukung kebutuhan operasional tanpa menyebabkan keterlambatan?

1.3 Tujuan Penelitian

Penulisan TA ini bertujuan untuk mencapai beberapa sasaran utama, yaitu sebagai berikut.

1. Mengembangkan sistem *backend* berbasis Laravel yang dapat mengotomatisasi pengelolaan data karyawan, termasuk pencatatan presensi, penjadwalan kerja, serta pengelolaan izin dan cuti, guna meningkatkan efisiensi operasional perusahaan.
2. Menyediakan sistem perhitungan gaji dan lembur yang otomatis dan akurat, sehingga dapat mengurangi risiko kesalahan dalam pengolahan data keuangan karyawan dan meningkatkan transparansi serta kepuasan kerja.
3. Mengintegrasikan sistem *backend* dengan aplikasi Android untuk memastikan karyawan dapat mengakses informasi terkait jadwal kerja, riwayat presensi, slip gaji, serta izin dan cuti secara digital guna mendukung proses bisnis perusahaan secara keseluruhan.
4. Melakukan pengujian *whitebox* untuk memastikan kualitas dan keandalan sistem, dengan cara menganalisis alur logika dan struktur kode, sehingga dapat mengurangi potensi kesalahan dan meningkatkan efisiensi sistem secara keseluruhan.
5. Melakukan pengujian *response time* untuk memastikan bahwa sistem *backend* memiliki performa optimal, dengan mengukur kecepatan pemrosesan data guna mendukung kebutuhan operasional perusahaan tanpa mengalami keterlambatan.

1.4 Batasan Masalah

Agar penulisan TA ini tetap fokus dan terarah, terdapat beberapa batasan masalah yang ditetapkan sebagai berikut:.

1. Penelitian ini hanya berfokus pada pengembangan *backend* sistem informasi sumber daya manusia berbasis Laravel, pengembangan *frontend* atau tampilan aplikasi tidak menjadi bagian dari penelitian ini.
2. *Backend* Laravel akan menyediakan RESTful API untuk berkomunikasi dengan aplikasi Android dan *frontend web*. Namun, penelitian ini tidak

membahas detail implementasi API pada sisi klien atau pengelolaan *request-response* di aplikasi Android.

3. Sistem ini menggunakan MySQL sebagai basis data utama, namun penelitian tidak akan membahas aspek optimasi *query*, indeksasi data, atau strategi replikasi basis data secara mendalam.
4. *Backend* Laravel akan menerapkan sistem autentikasi pengguna dan otorisasi peran (*role-based access control*), tetapi penelitian ini tidak mencakup analisis keamanan mendalam, seperti enkripsi data atau mitigasi serangan siber.
5. Pengujian yang dilakukan hanya berfokus pada aspek *backend*, mencakup validasi data, keandalan sistem melalui metode *whitebox* testing, serta pengujian performa berdasarkan *response time* RESTful API. Pengujian keamanan mendalam, seperti *penetration testing* atau analisis *vulnerability assessment*, tidak menjadi cakupan dalam penelitian ini.

1.5 Manfaat Penelitian

Penelitian ini memberikan manfaat untuk Penulis dan untuk pengguna. Dari penelitian ini, Penulis mendapatkan manfaat sebagai berikut.

1. Menerapkan ilmu dan keterampilan pemrograman *backend* berbasis Laravel yang telah diperoleh selama perkuliahan dalam pengembangan sistem informasi berbasis web.
2. Menambah wawasan dan pengalaman dalam membangun *backend* yang mendukung komunikasi dengan aplikasi Android melalui RESTful API.
3. Meningkatkan keterampilan dalam manajemen basis data MySQL, termasuk penerapan *Eloquent* ORM dan migrasi *database* untuk memastikan pengelolaan data yang efisien dan aman.
4. Memperdalam pemahaman tentang metode RAD dalam pengembangan perangkat lunak guna menghasilkan sistem yang cepat beradaptasi terhadap kebutuhan pengguna.

Pengguna sistem ini, khususnya HRD dan *Finance* di CV Mebel International, akan memperoleh manfaat dalam pengelolaan SDM yang lebih

efisien. HRD dapat mengelola data karyawan secara digital, termasuk presensi, jadwal kerja, izin, dan cuti, tanpa bergantung pada metode manual. *Finance* juga dapat menghitung gaji dan lembur secara otomatis, mengurangi risiko kesalahan. Selain itu, sistem ini membantu menyebarkan informasi perusahaan secara lebih terstruktur, meningkatkan komunikasi internal, serta memungkinkan akses data *real-time* untuk pengambilan keputusan yang lebih cepat dan akurat. Dengan *backend* berbasis Laravel, pengelolaan SDM menjadi lebih modern, terstruktur, dan produktif.

1.6 Metodologi Penelitian

Berikut penjelasan masing-masing langkah dalam metodologi penelitian yang digunakan dalam pengerjaan Tugas Akhir.

1. Studi Pustaka

Studi pustaka dilakukan untuk memahami teori-teori yang berkaitan dengan pengembangan *backend* menggunakan *framework* Laravel, konsep arsitektur *Model-View-Controller* (MVC), serta pengelolaan RESTful API yang digunakan untuk komunikasi dengan aplikasi Android. Studi ini juga mencakup pemahaman tentang pengelolaan basis data MySQL, teknik autentikasi dan otorisasi pengguna, serta metode pengujian *white-box testing* untuk memastikan keandalan sistem.

2. Analisis Kebutuhan

Tahap ini bertujuan untuk mengidentifikasi kebutuhan sistem berdasarkan wawancara dan diskusi dengan tim HRD dan *Finance* di CV Mebel Internasional Semarang. Analisis ini mencakup spesifikasi pengguna, fitur-fitur *backend*, serta integrasi data presensi dari aplikasi Android. Data presensi dikumpulkan melalui aplikasi Android yang digunakan oleh karyawan untuk melakukan presensi, baik di lokasi kerja (*on-site*) maupun di luar kantor (*off-site*). *Backend* Laravel bertugas untuk mengelola data presensi ini serta menyediakan fitur-fitur utama seperti pengelolaan izin dan cuti, perhitungan gaji dan lembur, serta sistem penyebaran pengumuman perusahaan.

3. Perancangan Sistem

Pada tahap ini, dilakukan perancangan *backend* berbasis Laravel yang mengatur bagaimana sistem memproses data karyawan dan presensi dari aplikasi Android. Perancangan ini mencakup struktur basis data MySQL untuk menyimpan informasi karyawan, presensi, izin, dan gaji. Selain itu, RESTful API dirancang untuk menghubungkan *backend* dengan aplikasi Android agar data presensi dapat tersimpan secara otomatis di server. Sistem ini juga menerapkan *Model-View-Controller* (MVC) untuk memisahkan logika bisnis, tampilan, dan pengelolaan data, sehingga *backend* lebih modular dan mudah dikembangkan. Dokumen teknis seperti *Business Process Model and Notation* (BPMN), *Entity-Relationship Diagram* (ERD), dan *Use Case Diagram*, serta digunakan sebagai pedoman implementasi sistem.

4. Implementasi

Tahap implementasi meliputi pengembangan *backend* menggunakan Laravel, yang bertanggung jawab untuk mengelola data karyawan, memproses presensi dari aplikasi Android, serta mengotomatisasi perhitungan gaji dan lembur. Pada tahap ini, fitur utama seperti pengelolaan izin dan cuti, *approval* HRD, serta penyebaran pengumuman digital juga dikembangkan. RESTful API dikonfigurasi agar dapat berkomunikasi dengan aplikasi Android, memungkinkan karyawan untuk mengakses data presensi, slip gaji, serta melakukan pengajuan cuti secara langsung.

5. Pengujian Sistem

Tahap ini merupakan langkah terakhir dalam proses pengembangan sistem *backend* berbasis Laravel. Pengujian dilakukan menggunakan metode *white-box* testing dengan fokus pada validasi struktur kode dan logika program. Pengujian dilakukan dalam bentuk tabel pengujian HTTP *response*, yang digunakan untuk memastikan setiap *endpoint* RESTful API memberikan respons yang sesuai dengan permintaan pengguna. Unit testing diterapkan untuk menguji masing-masing fungsi dalam *backend* Laravel, seperti pencatatan presensi, pengelolaan izin dan cuti, serta perhitungan gaji. API testing dilakukan untuk memastikan bahwa setiap request yang dikirim dari aplikasi Android mendapatkan *response* yang benar, baik dalam format data, status kode HTTP, maupun waktu respons. Pengujian ini bertujuan

untuk memastikan bahwa sistem *backend* bekerja secara akurat dan stabil dalam mengelola sumber daya manusia di CV Mebel Internasional Semarang sebelum sistem diimplementasikan sepenuhnya.

1.7 Sistematika Penulisan

Untuk memberikan gambaran mengenai isi laporan Tugas Akhir ini, berikut adalah sistematika penulisan yang disusun secara singkat.

BAB I PENDAHULUAN

Bab pertama menjelaskan latar belakang penelitian, perumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, metode penelitian, serta sistematika penulisan laporan.

BAB II TINJAUAN PUSTAKA

Bab kedua membahas konsep dasar yang mendukung penelitian dan pengembangan sistem. Tinjauan pustaka mencakup kajian dari penelitian terdahulu serta teori-teori yang digunakan dalam perancangan dan implementasi sistem.

BAB III PERANCANGAN SISTEM

Bab ketiga ini menjelaskan tahapan dalam perancangan *backend* SISDM berbasis Laravel. Uraian mencakup perancangan fitur utama, desain sistem, integrasi RESTful API dengan aplikasi Android, serta penerapan arsitektur *Model-View-Controller* (MVC) dalam *framework* Laravel.

BAB IV HASIL DAN PEMBAHASAN

Bab keempat berisi penjelasan mengenai hasil implementasi *backend* serta proses pengujian sistem. Ulasan disertai dengan gambar dan analisis yang menggambarkan bagaimana *backend* bekerja dalam mendukung pengelolaan SDM di perusahaan.

BAB V PENUTUP

Bab terakhir berisi kesimpulan dari hasil pengembangan sistem, termasuk evaluasi apakah sistem yang dibuat telah sesuai dengan rancangan awal. Selain itu, bab ini juga memuat saran untuk perbaikan dan pengembangan lebih lanjut guna meningkatkan sistem di masa depan.

BAB II

KAJIAN PUSTAKA

2.1 Kajian Penelitian Terdahulu

Dalam pelaksanaan penelitian ini, diperlukan beberapa penelitian sebelumnya sebagai acuan dan pembandingan. Penelitian-penelitian terdahulu yang digunakan memiliki tema, metode, atau pendekatan teknologi yang relevan dengan penelitian ini. Perbandingan dilakukan untuk melihat keunggulan dan kelemahan dari berbagai metode serta *framework* yang telah digunakan dalam penelitian sebelumnya guna memperkuat landasan teori dalam pengembangan SISDM CV Mebel Internasional berbasis Laravel dan *database* MySQL menggunakan metode *Rapid Application Development* (RAD).

Penelitian yang dilakukan oleh Ervina Rosa Aulia (2025) dengan judul “Pengembangan Sistem Informasi Manajemen Magang Berbasis *Website* dengan *Framework* Laravel dan VueJS di Kementerian Agama Kota Surabaya” ^[6] bertujuan untuk mengembangkan sistem informasi manajemen magang berbasis *website* yang terintegrasi dengan *backend* Laravel dan *frontend* VueJS. Sistem ini dirancang untuk memudahkan pengelolaan data peserta magang, termasuk pencatatan informasi magang, pengelolaan jadwal, serta pelaporan. Metode pengembangan yang digunakan dalam penelitian ini adalah *waterfall*, di mana setiap tahapan pengembangan dilakukan secara berurutan mulai dari analisis kebutuhan hingga implementasi dan pengujian. Terdapat beberapa perbedaan antara penelitian Ervina Rosa Aulia dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.1.

Tabel 2.1 Perbandingan penelitian Ervina Rosa Aulia dengan penelitian Penulis

Penelitian Ervina Rosa Aulia	Penelitian Penulis
Menggunakan <i>framework</i> Laravel dengan Firebase.	Menggunakan <i>framework</i> Laravel dengan RESTful API.
Metode pengembangan menggunakan <i>Waterfall</i> .	Metode pengembangan menggunakan <i>Rapid Application Development</i> (RAD).

Penelitian lain yang relevan dilakukan oleh Hakim, M.A., Triesia, D., et al. (2024) dengan judul “Sistem Informasi Gaji Karyawan Menggunakan *Framework* Codeigniter pada Yayasan Pendidikan Islam Al Waziriyah.” ^[7] Penelitian ini

membahas pengembangan sistem informasi berbasis web yang berfungsi untuk mengelola gaji karyawan di lingkungan Yayasan Pendidikan Islam Al Waziriyah. Sistem ini dirancang menggunakan *framework* CodeIgniter sebagai *backend* dan basis data MySQL untuk penyimpanan informasi penggajian. Penelitian ini bertujuan untuk meningkatkan efisiensi dalam penghitungan gaji, mencatat tunjangan dan potongan, serta mengotomatisasi proses pencetakan slip gaji bagi karyawan yayasan. Terdapat beberapa perbedaan antara penelitian Hakim, M.A., Triesia, D., et al. dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.2.

Tabel 2.2 Perbandingan penelitian Hakim, M.A., Triesia, D., et al. dengan penelitian Penulis

Penelitian Hakim, M.A., Triesia, D., et al.	Penelitian Penulis
Menggunakan <i>framework</i> CodeIgniter dan MySQL.	Menggunakan <i>framework</i> Laravel dan MySQL.
Fokus pada pengelolaan gaji karyawan.	Fokus pada manajemen SDM (presensi, cuti, gaji, pengumuman).

Penelitian berikutnya yang dilakukan oleh A. Akbar and Ari pada tahun 2025 dengan judul penelitian “Rancang Bangun Sistem Informasi Pengukuran Indeks Profesionalitas ASN dengan Django menggunakan REST API berbasis Django (Python)”^[8], menjelaskan tentang pengembangan sistem informasi untuk mengukur indeks profesionalitas Aparatur Sipil Negara (ASN). Sistem ini dirancang menggunakan *framework* Django dengan arsitektur REST API, memungkinkan integrasi dengan aplikasi lain untuk analisis data profesionalisme ASN berdasarkan indikator kinerja. Basis data yang digunakan adalah PostgreSQL. Terdapat beberapa perbedaan antara penelitian A. Akbar and Ari dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.3.

Tabel 2.3 Perbandingan penelitian A. Akbar and Ari dengan penelitian Penulis

Penelitian A. Akbar and Ari	Penelitian Penulis
Menggunakan <i>framework</i> Django untuk backend.	Menggunakan <i>framework</i> Laravel untuk backend.
Database yang digunakan adalah PostgreSQL.	Database yang digunakan adalah MySQL.
Fokus pada evaluasi kinerja ASN.	Fokus pada pengelolaan SDM secara menyeluruh.

Pada penelitian yang dilakukan Reza Ardianto dan Gunawan Budi Sulistyopada tahun 2020 dengan penelitian yang berjudul, “Perancangan Sistem Informasi Perekrutan Karyawan Pada PT Yogya Indah Sejahtera Yogyakarta”^[9], bertujuan untuk mengembangkan sistem informasi perekrutan karyawan berbasis web.

Sistem ini dibuat dengan menggunakan *framework* Laravel dan bertujuan untuk mempermudah pencatatan pelamar kerja, penjadwalan wawancara, serta proses seleksi karyawan. Sistem ini menggunakan *database* SQLite dan dikembangkan dengan metode *waterfall*. Terdapat beberapa perbedaan antara penelitian Reza Ardianto dan Gunawan Budi Sulisty dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.4.

Tabel 2.4 Perbandingan Reza Ardianto dan Gunawan Budi Sulisty dengan penelitian Penulis

Penelitian Reza Ardianto dan Gunawan Budi Sulisty	Penelitian Penulis
Fokus pada sistem perekrutan karyawan.	Fokus pada pengelolaan SDM secara menyeluruh.
<i>Database</i> yang digunakan adalah SQLite.	<i>Database</i> yang digunakan adalah MySQL.

Pada penelitian yang dilakukan oleh Hengki Agung Prayoga pada tahun 2025 dengan penelitian yang berjudul, “Pemanfaatan *Face Recognition* Facenet Dalam Pembangunan Sistem Informasi Human Resource Pada PT. Comtelindo”^[10], bertujuan mengembangkan sistem informasi *human resource* dengan integrasi teknologi *face recognition* Facenet untuk memvalidasi kehadiran pegawai berbasis pengenalan wajah. Sistem ini berbasis web dan dikembangkan menggunakan *framework* Laravel, dengan metode *waterfall* sebagai pendekatan pengembangannya. Terdapat beberapa perbedaan antara penelitian Hengki Agung Prayoga dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.5.

Tabel 2.5 Perbandingan penelitian Hengki Agung Prayoga dengan penelitian Penulis

Penelitian Hengki Agung Prayoga	Penelitian Penulis
Memanfaatkan teknologi <i>face recognition</i> .	Tidak menggunakan teknologi <i>face recognition</i> .
Fokus pada presensi berbasis pengenalan wajah.	Fokus pada pengelolaan SDM secara menyeluruh.

Pada penelitian yang dilakukan Eko Saparnuriyan Putra pada tahun 2025 dengan penelitian yang berjudul, “Rancang Bangun Sistem Informasi Event dan *Payment Gateway* berbasis Web dengan *Framework* Laravel (Studi Kasus Akiba Matsuri)”^[11] bertujuan untuk mengembangkan sistem informasi berbasis Laravel yang digunakan untuk pengelolaan *event* dan integrasi *payment gateway*. Sistem ini memungkinkan pengguna untuk melakukan registrasi *event* secara digital dan melakukan pembayaran melalui platform berbasis Laravel. Terdapat beberapa

perbedaan antara penelitian Eko Saparnuriyan Putra dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.6.

Tabel 2.6 Perbandingan penelitian Eko Saparnuriyan Putra dengan penelitian Penulis

Penelitian Eko Saparnuriyan Putra	Penelitian Penulis
Menggunakan <i>framework</i> Laravel untuk pengelolaan <i>event</i> dan pembayaran.	Menggunakan <i>framework</i> Laravel untuk manajemen SDM.
Fokus pada sistem pembayaran dan <i>event</i> .	Fokus pada sistem presensi, cuti, gaji, dan pengumuman SDM.

Pada penelitian yang dilakukan H. Mahwahulhusna pada tahun 2025 dengan penelitian yang berjudul, “Implementasi Sistem Informasi SDM untuk Manajemen Cuti Di SD Negeri 2 Cibungur Menggunakan OrangeHRM”^[12], mengembangkan sistem informasi SDM yang berfokus pada manajemen cuti pegawai dengan menggunakan OrangeHRM sebagai platform utama. Sistem ini menggunakan metode pengujian *blackbox* untuk menguji fungsionalitas dasar, tanpa melakukan pengujian lebih lanjut. Terdapat beberapa perbedaan antara penelitian H. Mahwahulhusna dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.7.

Tabel 2.7 Perbandingan penelitian H. Mahwahulhusna dengan penelitian Penulis

Penelitian H. Mahwahulhusna	Penelitian Penulis
Sistem dikembangkan untuk manajemen cuti.	Sistem dikembangkan untuk pengelolaan SDM secara menyeluruh.
Menggunakan OrangeHRM sebagai platform.	Menggunakan Laravel berbasis <i>web</i> dan Android.
Pengujian dilakukan menggunakan <i>blackbox</i> .	Pengujian dilakukan menggunakan <i>whitebox</i> , <i>usability testing</i> , dan UAT.

Penelitian berikutnya yang dilakukan Sastra, R., Muhammad Rizki Akbar, & Dicky Hariyanto pada tahun 2024 dengan judul "Rancang Bangun Perangkat Lunak System Pengelolaan Data Penggajian Berbasis *Framework* CodeIgniter"^[13] membahas pengembangan sistem informasi penggajian karyawan menggunakan *framework* CodeIgniter dan basis data MySQL. Sistem ini bertujuan untuk meningkatkan efisiensi dalam pencatatan data karyawan, penghitungan gaji otomatis, serta pengelolaan slip gaji digital. Metode pengembangan yang digunakan dalam penelitian ini adalah RAD, yang memungkinkan siklus pengembangan lebih cepat melalui iterasi yang berulang dan umpan balik dari pengguna. Penelitian ini menekankan pada kemudahan dalam penggunaan sistem dan percepatan pemrosesan data gaji karyawan. Terdapat beberapa perbedaan antara penelitian

Sastra, R., Muhammad Rizki Akbar, & Dicky Hariyanto dengan penelitian Penulis yang dijelaskan secara lebih rinci dengan perbandingan pada Tabel 2.8.

Tabel 2.8 Perbandingan penelitian Sastra, R., Muhammad Rizki Akbar, & Dicky Hariyanto dengan penelitian Penulis

Penelitian Sastra, R., Muhammad Rizki Akbar, & Dicky Hariyanto	Penelitian Penulis
Sistem dikembangkan untuk pengelolaan data penggajian.	Sistem dikembangkan untuk pengelolaan SDM secara menyeluruh.
Menggunakan <i>framework</i> CodeIgniter untuk <i>backend</i> .	Menggunakan <i>framework</i> Laravel untuk <i>backend</i> .
Metode pengembangan menggunakan <i>Rapid Application Development</i> (RAD).	Metode pengembangan menggunakan <i>Rapid Application Development</i> (RAD).

Penelitian sebelumnya memiliki peran penting sebagai acuan dalam penelitian ini. Dari penelitian-penelitian terdahulu, dapat diperoleh pemahaman yang lebih baik tentang topik yang dibahas, menemukan kekurangan atau celah dalam penelitian sebelumnya, serta merumuskan pertanyaan penelitian yang lebih tepat. Selain itu, referensi dari penelitian terdahulu juga membantu dalam menentukan metode penelitian yang sesuai dan cara terbaik dalam melakukan penelitian ini. Berikut adalah ringkasan dari penelitian-penelitian terdahulu yang mencakup judul, tujuan, metode yang digunakan, serta hasil yang diperoleh, sebagaimana ditampilkan dalam Tabel 2.9.

Tabel 2.9 Tabel kajian penelitian terdahulu

No	Judul dan Peneliti	Tujuan	Metode Penelitian	Hasil
1	Pengembangan Sistem Informasi Manajemen Magang Berbasis Website dengan <i>Framework</i> Laravel dan VueJS di Kementerian Agama Kota Surabaya Oleh: Ervina Rosa Aulia (2025)	Mengembangkan sistem informasi manajemen magang berbasis website yang terintegrasi dengan <i>backend</i> Laravel dan <i>frontend</i> VueJS untuk memudahkan pengelolaan data peserta magang.	Metode <i>waterfall</i> , setiap tahapan dilakukan secara berurutan dari analisis kebutuhan hingga implementasi dan pengujian menggunakan <i>blackbox</i> testing.	Sistem yang dibangun membantu pengelolaan data peserta magang. Laravel memberikan fleksibilitas, tetapi Firebase kurang optimal dibandingkan MySQL.
2	Sistem Informasi Gaji Karyawan Menggunakan <i>Framework</i> CodeIgniter pada Yayasan Pendidikan Islam Al Waziriyah	Mengembangkan sistem informasi berbasis web yang berfungsi untuk mengelola gaji karyawan di Yayasan Pendidikan Islam Al Waziriyah.	<i>Framework</i> CodeIgniter, <i>database</i> MySQL, metode pengembangan <i>Waterfall</i> , pengujian menggunakan <i>blackbox</i> testing.	Sistem mempermudah pencatatan gaji. Laravel lebih unggul dalam manajemen <i>query</i> dibandingkan CodeIgniter.

No	Judul dan Peneliti	Tujuan	Metode Penelitian	Hasil
	Oleh: Hakim, M.A., Triesia, D., et al. (2024)			
3	Rancang Bangun Sistem Informasi Pengukuran Indeks Profesionalitas ASN dengan Django menggunakan REST API Oleh: M. A. A. Tanjung dan A. P. W. Wibowo (2025)	Mengembangkan sistem informasi untuk mengukur indeks profesionalitas Aparatur Sipil Negara (ASN).	<i>Framework</i> Django, <i>database</i> PostgreSQL, arsitektur REST API, metode pengembangan <i>waterfall</i> .	Sistem dapat mengukur profesionalitas ASN. MySQL lebih efisien dalam transaksi dibandingkan PostgreSQL.
4	Perancangan Sistem Informasi Perekrutan Karyawan Pada PT Yogya Indah Sejahtera Yogyakarta Oleh: Reza Ardianto dan Gunawan Budi Sulistyo (2020)	Mengembangkan sistem informasi perekrutan karyawan berbasis web untuk mempermudah pencatatan pelamar kerja, penjadwalan wawancara, serta proses seleksi karyawan.	<i>Framework</i> Laravel, <i>database</i> SQLite, metode pengembangan <i>waterfall</i> .	Sistem mempermudah proses perekrutan. MySQL lebih unggul dibandingkan SQLite dalam menangani data besar.
5	Pemanfaatan <i>Face Recognition</i> Dalam Pembangunan Sistem Informasi Human Resource Pada PT. Comtelindo Oleh: Hengki Agung Prayoga (2025)	Mengembangkan sistem informasi HR dengan integrasi teknologi <i>face recognition</i> untuk validasi kehadiran pegawai berbasis pengenalan wajah.	<i>Framework</i> Laravel, menggunakan <i>face recognition</i> , metode pengembangan <i>waterfall</i> .	Sistem membantu validasi kehadiran pegawai. Fokus pada biometrik, sementara penelitian ini mengoptimalkan <i>database</i> .
6	Rancang Bangun Sistem Informasi Event dan <i>Payment Gateway</i> berbasis Web dengan <i>Framework</i> Laravel (Studi Kasus Akiba Matsuri) Oleh: Eko Saparnuriyan Putra (2025)	Mengembangkan sistem informasi berbasis Laravel yang digunakan untuk pengelolaan <i>event</i> dan integrasi <i>payment gateway</i> .	<i>Framework</i> Laravel, metode pengembangan <i>waterfall</i> .	Sistem mempermudah registrasi <i>event</i> . Laravel digunakan, tetapi penelitian ini berfokus pada SDM bukan <i>event</i> .
7	Implementasi Sistem Informasi SDM untuk	Mengembangkan sistem informasi SDM yang	OrangeHRM, metode pengembangan <i>waterfall</i> , pengujian	Sistem mengelola cuti pegawai. Laravel

No	Judul dan Peneliti	Tujuan	Metode Penelitian	Hasil
	Manajemen Cuti di SD Negeri 2 Cibungur Menggunakan OrangeHRM Oleh: Aris Saparudin dan Edi Wibowo (2025)	berfokus pada manajemen cuti pegawai dengan menggunakan OrangeHRM sebagai platform utama.	menggunakan <i>blackbox</i> testing.	lebih fleksibel dibandingkan OrangeHRM dalam pengelolaan SDM secara menyeluruh.
8	Rancang Bangun Perangkat Lunak Sistem Pengelolaan Data Penggajian Berbasis <i>Framework</i> CodeIgniter Oleh: Sastra, R., Muhammad Rizki Akbar, & Dicky Hariyanto (2024)	Mengembangkan sistem informasi penggajian berbasis CodeIgniter untuk meningkatkan efisiensi pencatatan data karyawan dan penghitungan gaji otomatis.	<i>Framework</i> CodeIgniter, <i>database</i> MySQL, metode pengembangan <i>Rapid Application Development</i> (RAD).	Sistem mempercepat pencatatan gaji. RAD memungkinkan iterasi cepat dalam pengembangan, tetapi Laravel lebih unggul dibandingkan CodeIgniter dalam optimasi <i>query database</i> .

Berdasarkan penelitian-penelitian sebelumnya, penggunaan *framework* Laravel dan *database* MySQL terbukti memberikan fleksibilitas dan efisiensi dalam pengembangan sistem informasi, terutama dalam manajemen sumber daya manusia (SDM). Laravel menyediakan fitur bawaan seperti *Eloquent* ORM, yang mampu mengoptimalkan *query database* dan mempermudah pengelolaan data dibandingkan *framework* lain seperti CodeIgniter dan Django. MySQL juga lebih unggul dalam menangani transaksi data dalam jumlah besar dibandingkan SQLite dan PostgreSQL, sehingga cocok untuk sistem informasi SDM yang kompleks.

Selain itu, penerapan *Rapid Application Development* (RAD) dalam penelitian ini lebih fleksibel dibandingkan *waterfall*, yang digunakan oleh sebagian besar penelitian sebelumnya. RAD memungkinkan pengembangan lebih cepat dengan iterasi berulang dan *feedback* pengguna yang lebih dinamis, sehingga lebih sesuai untuk sistem yang membutuhkan pengelolaan data secara *real-time*, seperti presensi, cuti, gaji, dan pengumuman SDM.

Dengan demikian, kombinasi Laravel, MySQL, dan RAD dalam penelitian ini dipilih karena memberikan kecepatan pengembangan, efisiensi pengelolaan

data, serta fleksibilitas dalam perancangan sistem dibandingkan metode dan teknologi yang digunakan dalam penelitian sebelumnya.

2.2 Landasan Teori

2.2.1 Sumber Daya Manusia

Sumber daya manusia (SDM) merupakan salah satu aset terpenting dalam sebuah organisasi yang bertanggung jawab atas berbagai aspek operasional dan pengambilan keputusan. Pengelolaan SDM yang baik dapat meningkatkan efisiensi kerja serta produktivitas perusahaan. ^[14] Manajemen SDM mencakup perekrutan, pelatihan, pengembangan karyawan, evaluasi kinerja, serta pemberian kompensasi. Dengan adanya sistem informasi yang mendukung pengelolaan SDM, perusahaan dapat mengoptimalkan pencatatan data karyawan, pengaturan jadwal kerja, serta penghitungan gaji dan tunjangan secara lebih efisien.

Seiring dengan perkembangan teknologi, digitalisasi dalam pengelolaan SDM menjadi semakin penting. Sistem informasi SDM memungkinkan otomatisasi proses administratif yang sebelumnya dilakukan secara manual, sehingga mengurangi kesalahan dan mempercepat pengambilan keputusan. Implementasi sistem informasi SDM berbasis teknologi dapat memberikan transparansi dan kemudahan akses terhadap informasi karyawan, termasuk presensi, izin cuti, serta riwayat gaji.

2.2.1 Sistem Informasi

Sistem informasi adalah kombinasi dari teknologi informasi, prosedur bisnis, dan manusia yang berfungsi untuk mengelola, mengolah, serta mendistribusikan informasi guna mendukung operasional organisasi. Sistem informasi membantu organisasi dalam menyajikan data yang lebih akurat, terorganisir, dan dapat diakses dengan cepat. Dalam konteks perusahaan, sistem informasi dapat digunakan untuk berbagai keperluan, seperti manajemen keuangan, SDM, produksi, dan pemasaran.

Pengembangan sistem informasi memerlukan pendekatan yang terstruktur agar dapat memenuhi kebutuhan organisasi secara efektif. Sistem informasi modern

biasanya berbasis web dan *mobile* untuk memastikan aksesibilitas yang lebih luas bagi pengguna. Sistem informasi yang terintegrasi dapat meningkatkan efisiensi kerja, mempermudah pelaporan, serta memungkinkan perusahaan untuk mengelola data secara lebih aman dan terpusat.

2.2.2 Laravel

Laravel merupakan salah satu *framework* PHP terpopuler yang banyak digunakan dalam pengembangan aplikasi web. Awalnya dirilis oleh Taylor Otwell pada tahun 2011, Laravel menawarkan sintaks yang elegan dan bersih, serta berbagai fitur yang memudahkan pengembangan aplikasi web yang tangguh dan *scalable*. *Framework* ini mengadopsi arsitektur MVC (*Model-View-Controller*), yang memisahkan logika aplikasi, antarmuka pengguna, dan data, sehingga memungkinkan proses pengembangan menjadi lebih terstruktur dan mudah dikelola ^{[15][15]}.

Laravel menyediakan beragam fitur bawaan seperti sistem routing yang fleksibel, *Eloquent ORM (Object-Relational Mapping)* untuk interaksi dengan basis data, sistem migrasi *database* yang kuat, serta berbagai alat bantu pengembangan seperti Artisan CLI. Selain itu, Laravel juga mendukung fitur keamanan seperti perlindungan terhadap serangan CSRF (*Cross-Site Request Forgery*), validasi *input*, serta enkripsi data, yang sangat penting dalam menjaga integritas dan keamanan aplikasi web ^[15].

2.2.3 Metode RAD

Rapid Application Development (RAD) adalah metode pengembangan perangkat lunak yang menekankan iterasi cepat, pengembangan prototipe, dan umpan balik langsung dari pengguna untuk mempercepat siklus pengembangan. Penerapan metode RAD dalam pengembangan perangkat lunak dapat meningkatkan fleksibilitas dalam perancangan sistem serta memungkinkan perubahan fitur berdasarkan kebutuhan pengguna dalam waktu yang lebih singkat dibandingkan metode konvensional seperti *Waterfall*. Metode ini sangat cocok

digunakan dalam proyek yang membutuhkan implementasi cepat, seperti pengembangan sistem informasi berbasis *web* dan *mobile*.

Selain itu, RAD memanfaatkan teknik pemodelan yang berorientasi pada komponen, sehingga memungkinkan pengembang untuk membangun sistem dengan pendekatan modular. RAD memungkinkan integrasi dengan standar pengujian perangkat lunak untuk memastikan kualitas aplikasi yang dikembangkan tetap tinggi. Dengan pendekatan ini, SISDM dapat dikembangkan dalam waktu yang lebih singkat dengan tetap mempertahankan standar keamanan dan performa yang optimal. ^[17]

2.2.4 Postman

Postman merupakan alat yang banyak digunakan dalam pengujian API untuk memastikan kualitas, keandalan, dan performa sistem *backend*. Dalam pengujian *whitebox*, Postman memungkinkan pengembang untuk memverifikasi logika bisnis dalam API dengan menguji berbagai skenario *input*, validasi data, dan jalur eksekusi kode. Dengan fitur *Test Scripts*, pengembang dapat menulis skrip otomatis berbasis JavaScript untuk mengevaluasi respons API serta memastikan jalur eksekusi bekerja sesuai harapan. ^[18] Pengujian ini penting dalam memastikan bahwa setiap permintaan API diproses dengan benar sesuai dengan struktur internal sistem *backend*, termasuk validasi data dan penanganan *error*.

Selain itu, Postman juga berperan dalam pengujian *response time* untuk menilai performa API dalam kondisi nyata. Dengan fitur pemantauan waktu respons, Postman dapat mengukur kecepatan pemrosesan data, sedangkan integrasinya dengan Apache JMeter memungkinkan simulasi beban tinggi (*load testing*) guna mengidentifikasi *bottleneck* dalam sistem. ^[19] Studi yang dilakukan oleh menunjukkan bahwa Postman dapat meningkatkan efisiensi dalam validasi API melalui pengujian otomatis, sehingga meminimalkan kesalahan serta mempercepat siklus pengembangan perangkat lunak. Dengan demikian, Postman tidak hanya digunakan sebagai alat pengujian fungsional API, tetapi juga sebagai pendukung dalam pengujian *whitebox* dan optimasi performa sistem sebelum diterapkan ke lingkungan produksi.

2.2.5 *Whitebox Testing*

Whitebox Testing, atau pengujian kotak putih, adalah metode pengujian perangkat lunak yang berfokus pada analisis struktur internal dan kode sumber program untuk memastikan keandalan serta keamanan sistem. Metode ini memungkinkan pengembang untuk memeriksa aliran kontrol dan data dalam aplikasi, memastikan bahwa setiap jalur dan kondisi dalam kode telah diuji secara menyeluruh sebelum implementasi di lingkungan produksi. Teknik ini efektif untuk mengidentifikasi kelemahan dalam kode program, termasuk redundansi, kesalahan logika, serta potensi kerentanan keamanan seperti injeksi SQL dan *Cross-Site Scripting* (XSS).^[20]

2.2.6 *Load Testing*

Load Testing, atau pengujian beban, adalah proses pengujian yang dilakukan untuk memahami perilaku sistem di bawah beban tertentu yang diharapkan. Pengujian ini menyimulasikan sejumlah pengguna yang mengakses aplikasi secara bersamaan untuk memastikan bahwa sistem dapat menangani beban tersebut tanpa mengalami penurunan kinerja atau kegagalan. Tujuan utama dari *load testing* adalah mengidentifikasi batas kapasitas operasional maksimum dari suatu aplikasi serta mengidentifikasi *bottleneck* yang dapat menyebabkan degradasi kinerja.^[21] Dalam konteks sistem informasi SDM berbasis Laravel, *load testing* dapat membantu memastikan bahwa aplikasi dapat menangani jumlah pengguna yang diharapkan tanpa mengalami masalah kinerja. Pengujian ini juga dapat membantu mengidentifikasi area dalam aplikasi yang mungkin memerlukan optimasi untuk meningkatkan skalabilitas dan responsivitas sistem.

BAB III

PERANCANGAN SISTEM

SISDM di CV Mebel Internasional Semarang dikembangkan menggunakan *framework* Laravel berbasis *Model-View-Controller* (MVC) dan *database* MySQL. Proses pengembangan dilakukan dengan metode *Rapid Application Development* (RAD) yang terdiri dari beberapa tahap utama, yaitu *analysis and quick design*, iterasi (*development*, *testing*, dan *demonstrate*), serta *deployment*.

Tahap pertama, *analysis and quick design*, dilakukan dengan mengumpulkan kebutuhan sistem dari pemangku kepentingan perusahaan. Analisis ini bertujuan untuk mengidentifikasi kendala dalam pengelolaan data kepegawaian, seperti pengajuan cuti, presensi, penggajian, serta distribusi pengumuman, dan merancang solusi yang tepat. Rancangan awal sistem dibuat untuk menentukan struktur *database*, relasi antar tabel, serta alur data yang efisien dalam MySQL.

Tahap iterasi terdiri dari tiga proses utama: *development*, *testing*, dan *demonstrate*. Pada tahap *development*, sistem mulai diimplementasikan menggunakan Laravel sebagai *backend*, dengan RESTful API untuk memudahkan komunikasi antar modul dan integrasi dengan berbagai layanan. Eloquent ORM digunakan untuk mengelola *database* MySQL secara lebih efisien dan mengoptimalkan performa *query*. Laravel juga menyediakan fitur seperti *middleware* untuk autentikasi, manajemen *session* dan *role-based access control* (RBAC), serta validasi data untuk meningkatkan keamanan dan integritas sistem.

Setelah pengembangan *backend* selesai, dilakukan pengujian *whitebox* untuk memastikan bahwa setiap *endpoint* API dan fungsi *backend* bekerja sesuai spesifikasi. *Whitebox testing* digunakan untuk menguji efisiensi dan optimasi *query* MySQL, sehingga sistem dapat menangani jumlah data yang besar dengan performa yang optimal. *Usability testing* juga dilakukan untuk memastikan bahwa API yang dikembangkan dapat dengan mudah diintegrasikan dengan *frontend* sistem.

Pada tahap *demonstrate*, sistem diuji coba oleh pemangku kepentingan untuk mendapatkan umpan balik terkait performa dan fungsionalitas *backend*.

Masukan dari pengguna digunakan sebagai dasar untuk melakukan perbaikan dan optimasi lebih lanjut, terutama dalam struktur *database*, performa *query*, dan pengelolaan data pengguna.

Tahap akhir adalah *deployment*, di mana SISDM berbasis Laravel dan MySQL diterapkan dalam lingkungan produksi. Dengan adanya sistem ini, pengelolaan SDM menjadi lebih efisien, aman, dan terstruktur, memungkinkan perusahaan untuk mengoptimalkan pengolahan data pegawai, mempermudah proses administrasi kepegawaian, serta meningkatkan produktivitas dalam pengelolaan sumber daya manusia.

3.1 Analysis and Quick Design

Tahap *analysis and quick design* dilakukan melalui wawancara dengan pemangku kepentingan di CV Mebel Internasional Semarang, khususnya pihak yang bertanggung jawab dalam pengelolaan SDM. Wawancara ini bertujuan untuk mengidentifikasi permasalahan dalam pengelolaan data kepegawaian, seperti pengajuan cuti, presensi, penggajian, serta distribusi pengumuman internal. Dari hasil wawancara, diperoleh kebutuhan fungsional dan non-fungsional yang harus dipenuhi oleh SISDM berbasis web.

Pada tahap ini, dilakukan perancangan awal arsitektur *backend* menggunakan *framework* Laravel dan *database* MySQL. Struktur *database* dirancang dengan relasi yang efisien antar tabel untuk memastikan integritas data dan performa optimal dalam menangani transaksi data dalam jumlah besar. Selain itu, ditetapkan juga pola arsitektur *Model-View-Controller* (MVC) untuk memisahkan logika bisnis (*Model*), pengelolaan *request* (*Controller*), dan antarmuka pengguna (*View*). Pendekatan ini bertujuan untuk memastikan bahwa sistem dapat dikembangkan dan dipelihara dengan lebih mudah di masa mendatang, serta mendukung fleksibilitas dalam pengembangan fitur tambahan.

3.1.1 Kebutuhan Fungsional Sistem

Kebutuhan fungsional merupakan aspek penting dalam pengembangan sistem, yang mencakup fasilitas dan aktivitas yang harus dapat dilakukan oleh

sistem sesuai dengan kebutuhan pengguna. ^[22] SISDM CV Mebel Internasional Semarang dikembangkan untuk mempermudah pengelolaan data kepegawaian seperti manajemen karyawan, presensi, cuti, penggajian, dan pengumuman melalui platform berbasis *web* yang dibangun menggunakan *framework* Laravel dan *database* MySQL. Khusus untuk kebutuhan fungsional Karyawan, sebagian besar fitur yang dibutuhkan akan lebih terfokus pada aplikasi Android, karena aplikasi *mobile* inilah yang menjadi platform utama yang digunakan oleh karyawan untuk mengakses berbagai layanan dan informasi terkait kepegawaian.

Penentuan kebutuhan fungsional dilakukan melalui analisis mendalam guna memastikan bahwa setiap fitur sistem telah terdefinisi secara jelas sejak tahap awal pengembangan. Proses ini bertujuan untuk mengoptimalkan struktur *backend*, memastikan efisiensi dalam pengolahan data melalui MySQL, serta mengimplementasikan arsitektur yang memungkinkan sistem berkembang secara fleksibel. Kebutuhan fungsional yang telah dirancang untuk sistem web SISDM dapat dilihat pada Tabel 3.1.

Tabel 3.1 Tabel kebutuhan fungsional sistem

No.	Kategori Pengguna	Deskripsi Kebutuhan	Prioritas
1.	Karyawan	Melakukan <i>login</i> .	Tinggi
2.	Karyawan	Melihat jadwal kerja.	Tinggi
3.	Karyawan	Melihat pengumuman dan informasi.	Tinggi
4.	Karyawan	Melihat surat peringatan.	Tinggi
5.	Karyawan	Melihat slip gaji.	Tinggi
6.	Karyawan	Mengajukan izin dan cuti.	Tinggi
7.	Karyawan	Melihat profil.	Tinggi
8.	Karyawan	Mengedit profil.	Tinggi
9.	Karyawan	Melakukan presensi <i>off-site</i> .	Tinggi
10.	Karyawan	Melihat <i>history</i> presensi.	Tinggi
11.	Karyawan	Melakukan <i>logout</i> .	Tinggi
12.	HRD	Melakukan <i>login</i> .	Tinggi
13.	HRD	Mengelola data karyawan.	Tinggi
14.	HRD	Mengelola jadwal kerja.	Tinggi
15.	HRD	Mengelola pengumuman dan informasi.	Tinggi
16.	HRD	Mengelola <i>history</i> presensi.	Tinggi
17.	HRD	Menetapkan lokasi presensi.	Tinggi
18.	HRD	Menyetujui/menolak izin dan cuti.	Tinggi
19.	HRD	Melakukan <i>logout</i> .	Tinggi

No.	Kategori Pengguna	Deskripsi Kebutuhan	Prioritas
20.	Finance	Melakukan <i>login</i> .	Tinggi
21.	Finance	Menghitung gaji dan upah lembur.	Tinggi
22.	Finance	Melakukan <i>logout</i> .	Tinggi

3.1.2 Kebutuhan Non-Fungsional Sistem

Selain kebutuhan fungsional, sistem juga memiliki kebutuhan non-fungsional, yang dapat dilihat pada Tabel 3.2. Kebutuhan ini mencakup aspek teknis dan operasional yang memastikan sistem berjalan dengan optimal, stabil, dan aman sesuai dengan standar yang telah ditetapkan.

Tabel 3.2 Tabel kebutuhan non-fungsional sistem

Parameter	Requirement
<i>Availability</i>	Aplikasi ini dapat beroperasi 7 hari dalam seminggu dan 24 jam dalam satu hari.
<i>Reliability</i>	Sistem akan menjamin minimalisasi tingkat kegagalan dalam pengoperasian.
<i>Response time</i>	Memberikan waktu respons maksimal kira-kira 5 detik.

3.1.3 Karakteristik Pengguna

SISDM CV Mebel Internasional Semarang dikembangkan sebagai platform berbasis web yang dirancang untuk digunakan oleh HRD dan *Finance* dalam mengelola data kepegawaian secara efisien. *Role* pengguna dalam sistem ini difokuskan pada administrator yang memiliki akses untuk mengelola informasi terkait karyawan, jadwal kerja, presensi, cuti, penggajian, serta pengumuman perusahaan.

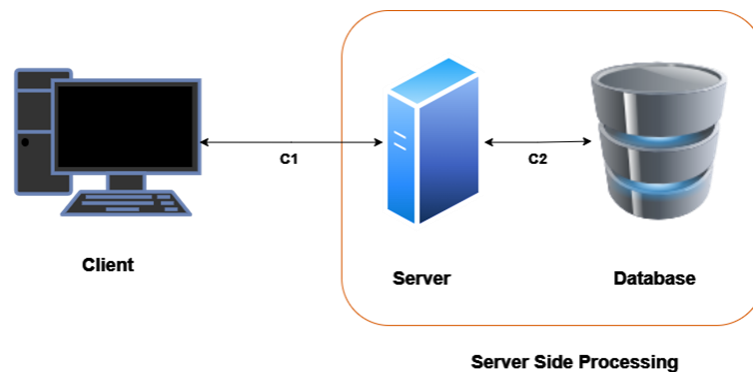
Sistem ini memungkinkan HRD untuk mengelola data karyawan, menetapkan jadwal kerja, mengelola presensi, menyetujui atau menolak pengajuan cuti, serta mengelola pengumuman internal. Sementara itu, *Finance* dapat mengakses fitur untuk menghitung gaji, memproses upah lembur, serta memastikan pembayaran gaji berjalan dengan akurat.

Setiap pengguna dalam sistem diberikan hak akses yang telah ditentukan, memastikan bahwa fitur yang tersedia sesuai dengan peran dan tanggung jawab masing-masing. Dengan implementasi *framework* Laravel dan *database* MySQL, sistem ini dirancang untuk memberikan manajemen data yang lebih terstruktur,

efisien, dan aman, mendukung kelancaran operasional CV Mebel Internasional Semarang dalam mengelola sumber daya manusia.

3.1.4 Arsitektur Sistem

Website SISDM CV Mebel Internasional Semarang dikembangkan menggunakan *framework* Laravel dan *database* MySQL, dengan arsitektur berbasis *web* yang dapat dilihat pada Gambar 3.1.



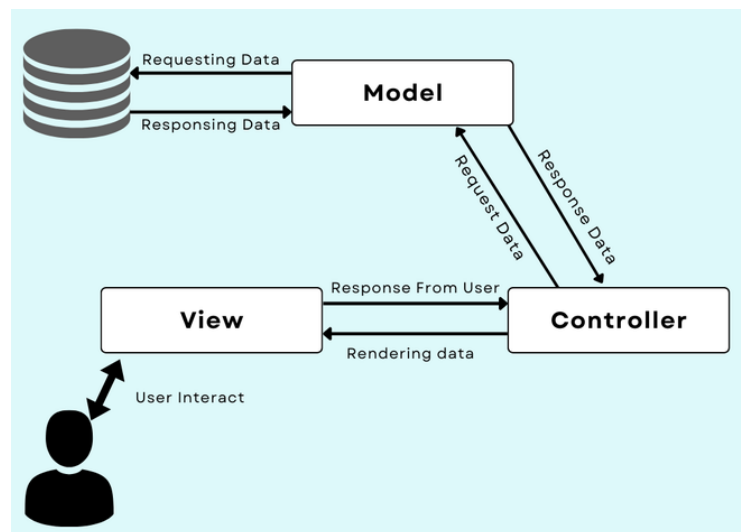
Gambar 3.1 Arsitektur sistem berbasis *website*

Arsitektur komponen sistem berbasis *website* memiliki tiga komponen utama, yaitu *client*, *server* (*web server/backend*), dan *database* (*database server/MySQL*). API akan menjadi jembatan transportasi antara perangkat *client* dengan *database* aplikasi. Masing-masing komponen memiliki hubungan yang dapat dilihat pada Tabel 3. 3.

Tabel 3. 3 Hubungan komponen arsitektur sistem berbasis *website*

Komponen Pengirim	Komponen Penerima	Media Transmisi	Metode Transmisi Data
<i>Client</i>	<i>Server</i>	Internet	HTTP
<i>Server</i>	<i>Database</i>	PDO	PDO

Backend website SISDM CV Mebel Internasional Semarang memiliki arsitektur pengembangan yang dapat dilihat pada Gambar 3.2.



Gambar 3.2 Arsitektur pengembangan *website*

Gambar 3.2 menampilkan arsitektur pengembangan *backend* untuk SISDM CV Mebel Internasional Semarang, yang menerapkan pola arsitektur *Model-View-Controller* (MVC). Arsitektur ini dirancang untuk memisahkan tanggung jawab dalam pengelolaan data, logika bisnis, dan antarmuka pengguna, sehingga meningkatkan modularitas, efisiensi pengembangan, serta kemudahan pemeliharaan sistem dalam jangka panjang.

Pada implementasi MVC dalam Laravel, sistem ini terdiri dari tiga komponen utama. Model bertanggung jawab atas interaksi langsung dengan *database* MySQL, mencakup operasi membaca, menyimpan, serta memperbarui data terkait karyawan, presensi, pengajuan cuti, dan penggajian. Model diimplementasikan dalam direktori `app/Models`, menggunakan *Eloquent ORM* untuk mengoptimalkan pengelolaan *query database*, sehingga transaksi data dapat dilakukan dengan lebih efisien. View berfungsi sebagai antarmuka pengguna dalam sistem web yang menampilkan informasi yang telah diproses oleh sistem. View diimplementasikan melalui *Blade Template Engine* yang memungkinkan pembuatan tampilan dinamis dengan integrasi data dari *backend*. Antarmuka ini bertindak sebagai penghubung antara pengguna dan sistem, di mana *input* yang diberikan akan diproses lebih lanjut oleh *Controller*. *Controller* bertindak sebagai perantara antara Model dan View, menerima *request* dari pengguna, memprosesnya, lalu mengambil atau memperbarui data dari Model sebelum mengembalikannya ke View. *Controller* diimplementasikan dalam direktori `app/Http/Controllers`, yang

menangani berbagai proses dalam sistem seperti pengelolaan data karyawan, persetujuan cuti, dan pengelolaan penggajian.

Alur kerja sistem dalam arsitektur ini dimulai ketika pengguna berinteraksi dengan *View*, misalnya saat mengajukan cuti atau melihat slip gaji. *Input* yang diberikan pengguna dikirimkan dalam bentuk *HTTP request* ke *Controller*. *Controller* kemudian melakukan validasi terhadap *request* yang diterima dan meneruskannya ke *Model* untuk diproses. *Model* akan mengambil atau memperbarui data dalam *database* MySQL, lalu mengembalikan hasil pemrosesan tersebut ke *Controller*. Setelah itu, *Controller* meneruskan data ke *View*, sehingga dapat ditampilkan dalam bentuk informasi yang mudah dipahami oleh pengguna.

Arsitektur *backend* ini didukung oleh struktur direktori Laravel yang terorganisir, di mana *app/Models* digunakan untuk mengelola entitas data, *app/Http/Controllers* mengatur logika pemrosesan *request*, dan *resources/views* digunakan untuk tampilan antarmuka pengguna. Pengelolaan rute dilakukan melalui *routes/web.php*, sementara *database/migrations* berperan dalam memastikan skema *database* tetap konsisten. Penerapan arsitektur MVC dalam Laravel memberikan beberapa keunggulan utama dalam pengembangan SISDM. Pemisahan antara logika bisnis dan tampilan memungkinkan pengembangan sistem yang lebih modular dan fleksibel, sehingga setiap perubahan dalam satu komponen tidak memengaruhi komponen lainnya. *Eloquent ORM* dalam Laravel mengoptimalkan *query database*, sehingga meningkatkan efisiensi dalam pengelolaan data SDM. Selain itu, pendekatan ini juga memungkinkan integrasi dengan RESTful API, yang memberikan fleksibilitas lebih besar dalam menghubungkan sistem dengan aplikasi eksternal atau *frontend* berbasis JavaScript.

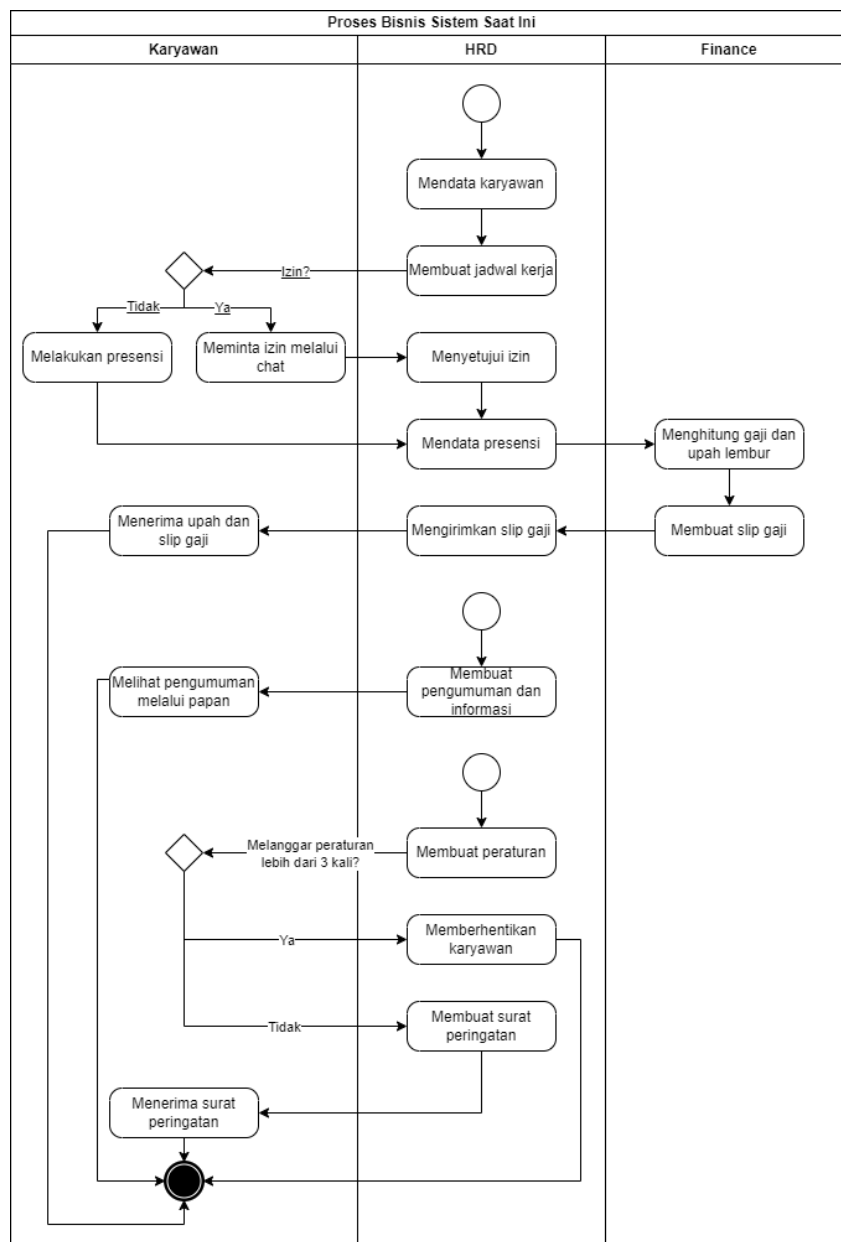
Dengan penerapan arsitektur MVC berbasis Laravel dan MySQL, SISDM CV Mebel Internasional Semarang diharapkan dapat memberikan kinerja yang optimal, keamanan yang lebih baik, serta fleksibilitas dalam pengembangan dan pemeliharaan sistem SDM.

3.2 Perancangan Sistem

3.2.1 Gambaran Umum Sistem Saat Ini

3.2.1.1. Proses Rekayasa/Proses Bisnis

Saat ini, proses manajemen SDM di CV. MI dikelola oleh Divisi HRD menggunakan alat konvensional seperti papan tulis, rumus Excel, dan aplikasi WhatsApp serta email. Proses bisnis manajemen SDM saat ini dapat dilihat pada Gambar 3.3.



Gambar 3. 3 Diagram BPMN proses bisnis manajemen SDM saat ini

Proses manajemen tersebut meliputi pendataan karyawan, pembuatan jadwal kerja, persetujuan izin, pendataan presensi, penghitungan gaji dan upah lembur, serta penyebarluasan pengumuman dan informasi,.

3.2.1.2. Prosedur

Terdapat beberapa prosedur dalam proses manajemen SDM di CV Mebel International. Prosedur rinci dalam proses manajemen SDM dapat dilihat pada Tabel 3.4.

Tabel 3. 4 Prosedur manajemen SDM saat ini

Aktor	Aksi	Prosedur
HRD	Mendata karyawan.	HRD mendata karyawan meliputi nama, foto diri, foto KTP, BPJS kesehatan, BPJS ketenagakerjaan, NIK, tempat dan tanggal lahir, usia, agama, jenis kelamin, status perkawinan, NPWP, alamat, nomor <i>handphone</i> , pendidikan, departemen, grup, jabatan, serta email menggunakan Microsoft Excel.
HRD	Membuat jadwal kerja.	HRD membuat jadwal kerja yang dibedakan per grup dan disiarkan menggunakan pesan <i>broadcast</i> WhatsApp.
Karyawan	Melakukan presensi.	Karyawan di pabrik Semarang presensi menggunakan mesin presensi <i>face recognition</i> , sedangkan karyawan di pabrik Jepara presensi dengan menggunakan mesin presensi kertas kartu.
Karyawan	Meminta izin.	Karyawan meminta izin kepada divisi HRD melalui WhatsApp dan telepon.
HRD	Menyetujui izin.	HRD yang menyetujui izin karyawan melanjutkan prosesnya. Jika karyawan sudah bekerja lebih dari 1 tahun, mereka berhak mendapatkan cuti selama 12 hari. Izin dan alpa akan mengakibatkan pemotongan gaji, sedangkan cuti tidak akan mempengaruhi gaji (tetap mendapatkan gaji). Sebaliknya, apabila izin tidak diberikan, karyawan tetap harus hadir dan melakukan

Aktor	Aksi	Prosedur
		presensi. Jika karyawan tetap membolos maka dianggap alpa.
HRD	Mendata presensi.	HRD mendata presensi menggunakan data CSV yang dihasilkan dari mesin presensi <i>face recognition</i> . Data tersebut disimpan dalam bentuk <i>file</i> Microsoft Excel dan mencakup data presensi untuk 2 tahun terakhir.
Finance	Menghitung gaji dan upah lembur.	Finance menghitung gaji dan upah lembur menggunakan rumus pada Microsoft Excel. Perhitungan gaji berbeda antara operator, staf, dan pegawai magang. Komponen yang dihitung terdiri dari gaji pokok per hari, upah lembur, tunjangan, kasbon, tali asih, potongan sosial, dan BPJS.
Finance	Membuat slip gaji.	Finance membuat slip gaji menggunakan <i>mailings</i> dengan <i>export</i> data dari Excel.
HRD	Mengirimkan slip gaji.	HRD mengirimkan slip gaji satu persatu via email.
HRD	Membuat pengumuman dan informasi.	HRD membuat pengumuman dan informasi seputar hari libur, pelatihan, dll yang diedarkan melalui WhatsApp supervisor dan ditempelkan pada papan pengumuman.

3.2.1.3. Service Time

Proses manajemen sumber daya manusia di CV. MI meliputi pendataan karyawan, penjadwalan kerja, persetujuan izin dan cuti, pendataan presensi, pengiriman slip gaji, serta pembuatan pengumuman dan informasi oleh divisi HRD. Selanjutnya, presensi, permohonan izin dan cuti oleh karyawan. Kemudian penghitungan gaji dan upah lembur, serta pembuatan slip gaji oleh divisi *Finance*. Penjelasan *service time* dari proses manajemen dapat dilihat pada Tabel 3.5.

Tabel 3. 5 *Service time* dari proses manajemen SDM saat ini

Layanan	Keterangan
Mendata karyawan.	Saat ini, HRD mendata karyawan memakan waktu 5-10 menit per karyawan, total 30 jam untuk 180 karyawan. Hal ini disebabkan banyaknya komponen data yang didata.
Membuat jadwal kerja.	Saat ini, HRD menyusun jadwal kerja memakan waktu 1-2 jam per grup dan

Layanan	Keterangan
	penyebarannya membutuhkan 30 menit. Total waktu yang dihabiskan mencapai 1,5-2,5 jam per grup.
Melakukan presensi.	Saat ini, proses presensi di berbeda-beda tergantung lokasi pabrik. Di pabrik Semarang, karyawan menggunakan mesin presensi <i>face recognition</i> yang memakan waktu sekitar 5 detik per karyawan. Sedangkan di pabrik Jepara, karyawan masih menggunakan mesin presensi kertas kartu yang memakan waktu 1 menit per presensi. Proses presensi ini membuat karyawan harus menunggu dalam antrean.
Meminta izin.	Saat ini, karyawan mengajukan izin membutuhkan waktu 1-2 menit per karyawan.
Menyetujui izin.	Saat ini, HRD menyetujui izin karyawan memakan waktu 5-10 menit per izin, tergantung pada kebijakan atasan.
Mendata presensi.	Saat ini, HRD mendata presensi dengan mengunduh data dari mesin presensi dan mengonversinya dari format CSV ke Excel. Diperkirakan waktu yang dibutuhkan untuk proses ini adalah 10-20 menit per mesin presensi <i>face recognition</i> dan 15-25 menit per mesin presensi kertas kartu.
Menghitung gaji dan upah lembur.	Saat ini, <i>finance</i> menghitung gaji dan upah lembur memerlukan waktu antara 30 hingga 60 menit untuk setiap pabrik. Ini disebabkan oleh pengolahan data presensi dan lembur yang diterima dalam format Excel, diikuti dengan perhitungan menggunakan rumus yang rumit. Setelah perhitungan selesai, atasan memerlukan waktu 1-3 hari untuk mengecek setiap perhitungan tersebut.
Membuat slip gaji.	Saat ini, <i>finance</i> membuat slip gaji secara otomatis menggunakan <i>mailings</i> yang membutuhkan proses selama sekitar 15-30 menit untuk setiap <i>batch</i> penggajian.
Mengirimkan slip gaji.	Saat ini, HRD mengirimkan slip gaji satu per satu kepada 180 karyawan via email yang memakan waktu kurang lebih 3-4 jam.
Membuat pengumuman dan informasi.	Saat ini, HRD menyebarluaskan pengumuman dan informasi melalui WhatsApp yang dikirimkan ke supervisor kemudian di lanjutkan ke bawahannya dan ditempelkan pada papan pengumuman. Perkiraan waktu yang dibutuhkan hingga pengumuman diterima oleh karyawan adalah sekitar 30 menit-2 jam.

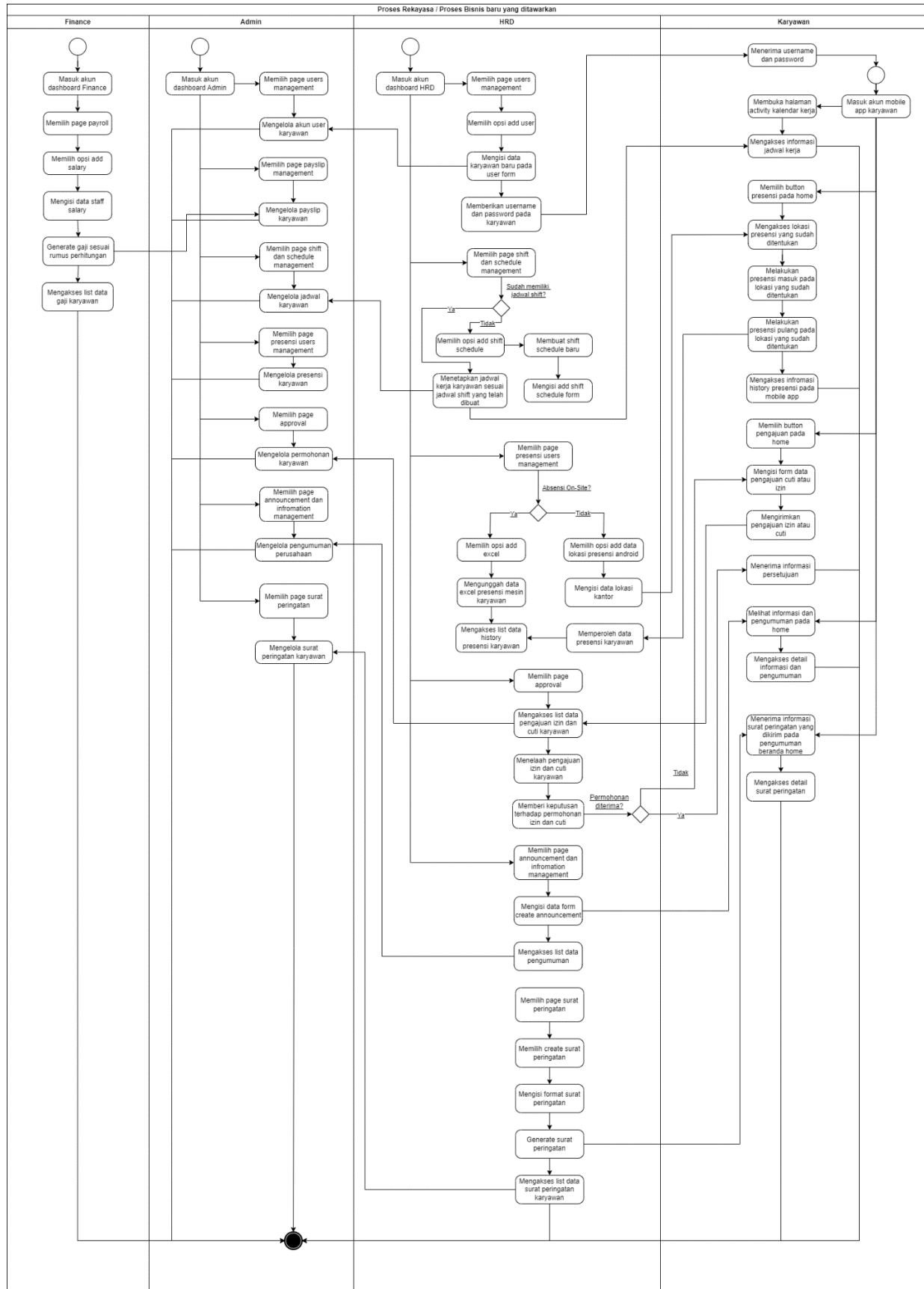
3.2.2 Target dari Sistem yang Dikembangkan

3.2.2.1. Ruang Lingkup Sistem

SISDM akan bisa diakses oleh *role* HRD dan *finance* melalui situs *web*. Sistem informasi yang dikembangkan akan meng-*handle* proses bisnis baru SDM di CV Mebel Internasional Semarang meliputi pendataan karyawan, penjadwalan kerja, presensi dan pendataan presensi, penghitungan gaji dan upah lembur serta pembuatan dan pengiriman slip gaji, permohonan dan persetujuan izin dan cuti, serta pembuatan pengumuman dan informasi.

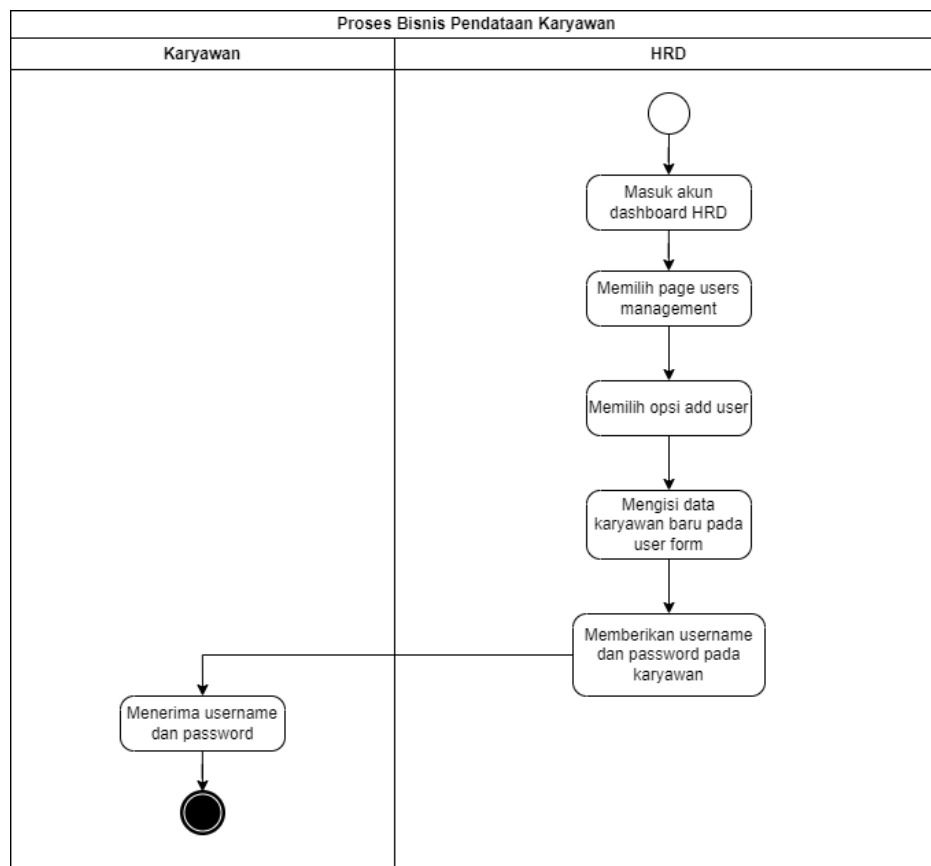
3.2.2.2. Proses Rekayasa / Proses Bisnis Baru yang Ditawarkan

Dengan permasalahan yang ada, maka direncanakan perbaikan mekanisme dengan pengembangan sistem informasi berbasis *website* yang ditujukan pada *role* HRD dan *finance*. Proses bisnis manajemen SDM dapat dilihat pada Gambar 3.4.



Gambar 3. 4 Proses bisnis pengelolaan Sumber Daya Manusia

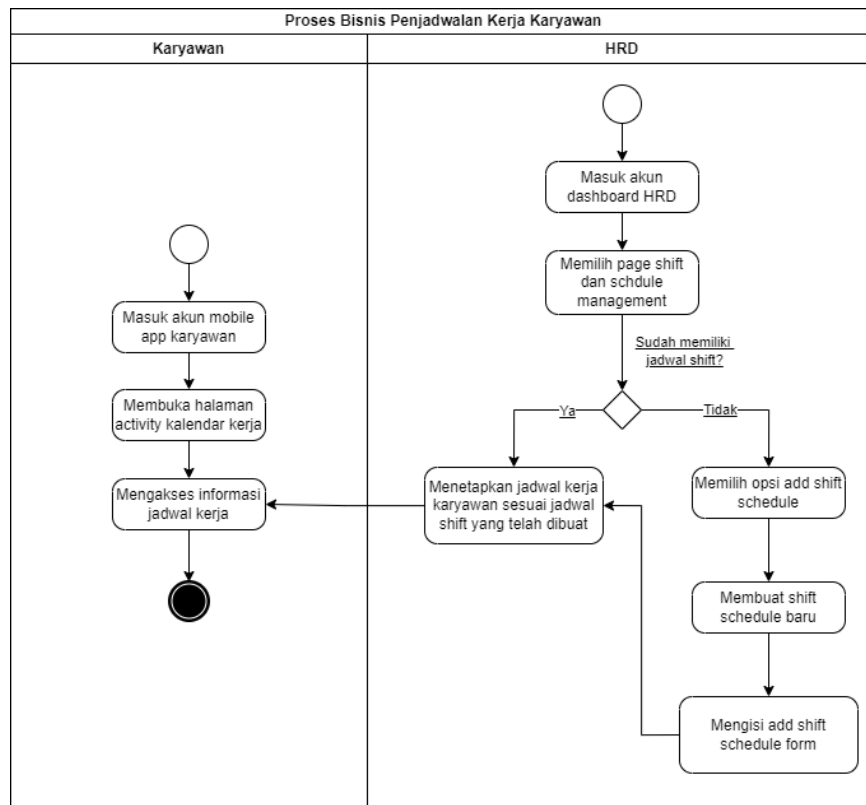
Proses bisnis pengelolaan SDM di CV Mebel International Semarang dimulai dengan pendataan informasi karyawan atau *user* pada *role* HRD di *dashboard website*. Proses bisnis pendataan karyawan dapat dilihat pada Gambar 3.5.



Gambar 3. 5 Proses bisnis pendataan karyawan

Fitur pendataan karyawan dalam sistem manajemen SDM dimulai ketika HRD masuk ke akun *dashboard* mereka. Setelah masuk, HRD memilih halaman *Users Management* di mana mereka dapat mengelola pengguna sistem. Di halaman ini, HRD memilih opsi *Add User* untuk menambahkan karyawan baru. HRD kemudian mengisi formulir dengan data karyawan baru, termasuk informasi pribadi dan detail *role* pekerjaan. Setelah formulir lengkap, HRD menetapkan *username* dan *password* untuk karyawan tersebut. *Username* dan *password* ini kemudian disampaikan kepada karyawan yang bersangkutan, memungkinkan mereka untuk mengakses sistem. Proses ini memastikan bahwa data karyawan baru dicatat dengan akurat dan mereka dapat segera mulai menggunakan aplikasi Android.

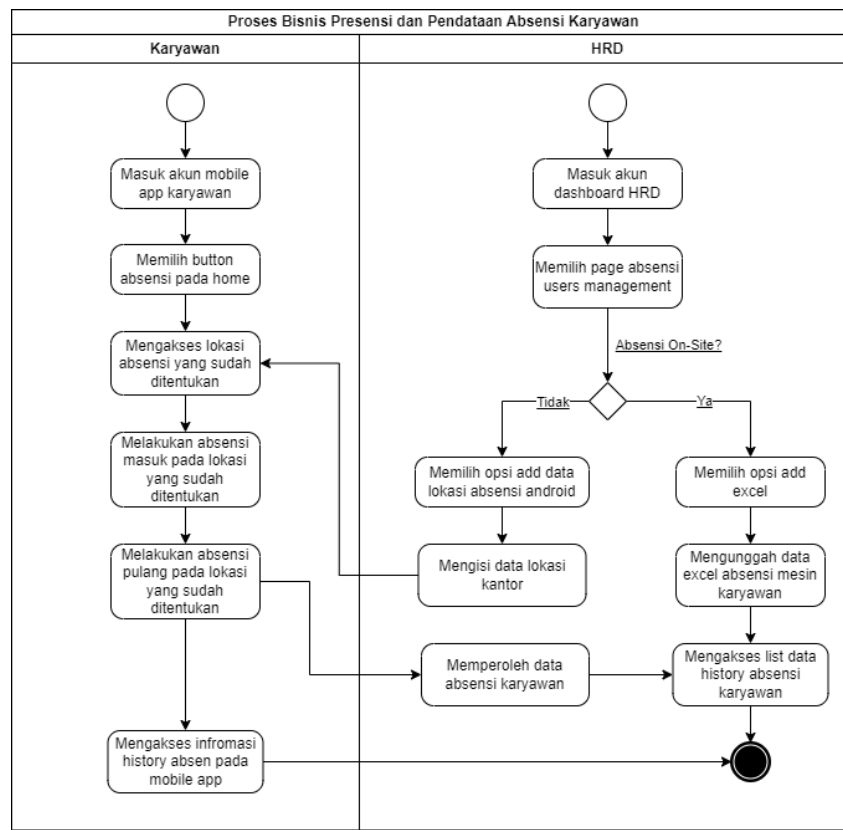
Setelah para karyawan mendapatkan *username* dan *password*, maka proses bisnis selanjutnya adalah penjadwalan kerja. Proses bisnis penjadwalan kerja karyawan dapat dilihat pada Gambar 3.6.



Gambar 3. 6 Proses bisnis penjadwalan kerja karyawan

Proses penjadwalan kerja karyawan dilakukan melalui fitur penjadwalan yang tersedia di sistem informasi berbasis *web*. HRD memulai dengan memilih halaman *Shift* dan *Schedule Management*. Jika belum memiliki jadwal *shift*, HRD akan membuat jadwal *shift* baru dengan mengisi *form* pada *Add Shift Schedule*. Setelah jadwal *shift* tersedia, HRD dapat menetapkan jadwal kerja karyawan sesuai dengan jadwal *shift* yang telah dibuat. Setelah penetapan jadwal selesai, karyawan dapat langsung mengakses informasi jadwal kerja mereka melalui halaman kalender kerja pada sistem Android.

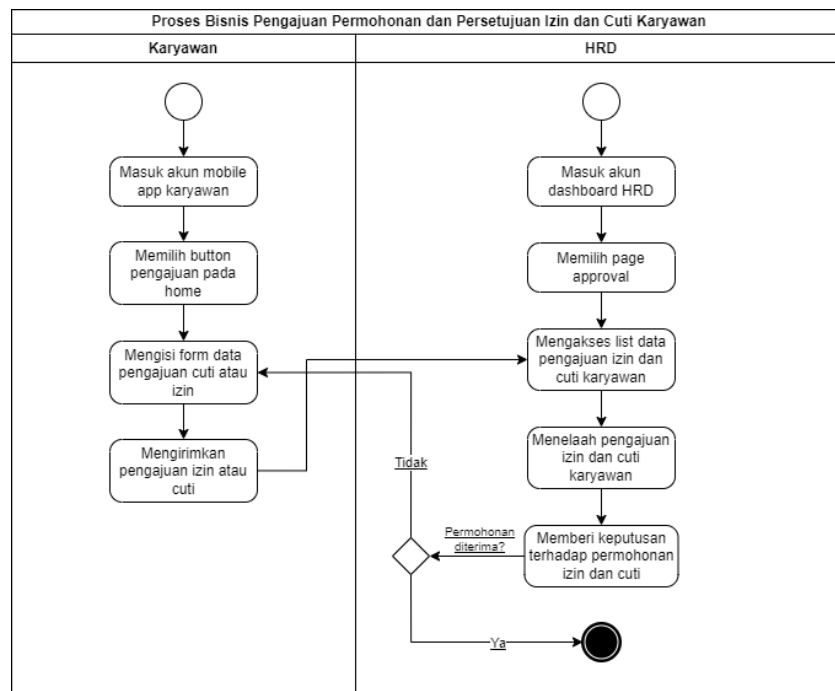
Setelah para karyawan memperoleh jadwal kerja, maka proses bisnis selanjutnya adalah presensi karyawan sesuai dengan hari dan jam kerja yang telah ditentukan oleh HRD dan pendataan presensi. Proses bisnis presensi dan pendataan presensi karyawan dapat dilihat pada Gambar 3.7.



Gambar 3. 7 Proses bisnis presensi dan pendataan presensi karyawan

Proses presensi dan pendataan presensi karyawan dilakukan melalui fitur yang tersedia di sistem informasi berbasis *web*. HRD memulai dengan memilih halaman Presensi *Users Management* kemudian memilih opsi *Add Excel* untuk mengunggah data *excel* presensi dari mesin presensi jika karyawan menggunakan presensi *onsite*. Jika karyawan menggunakan presensi melalui sistem Android, HRD memilih opsi *Add Data Lokasi Presensi Android* dan mengisi data lokasi kantor. Karyawan kemudian dapat melakukan presensi dengan menekan tombol presensi pada beranda sistem Android, dan presensi masuk atau pulang hanya dapat dilakukan di lokasi yang telah ditentukan oleh HRD. Data historis presensi dapat diakses oleh karyawan pada akun Android, sementara HRD juga dapat memperoleh dan mengelola data presensi karyawan melalui sistem *website*.

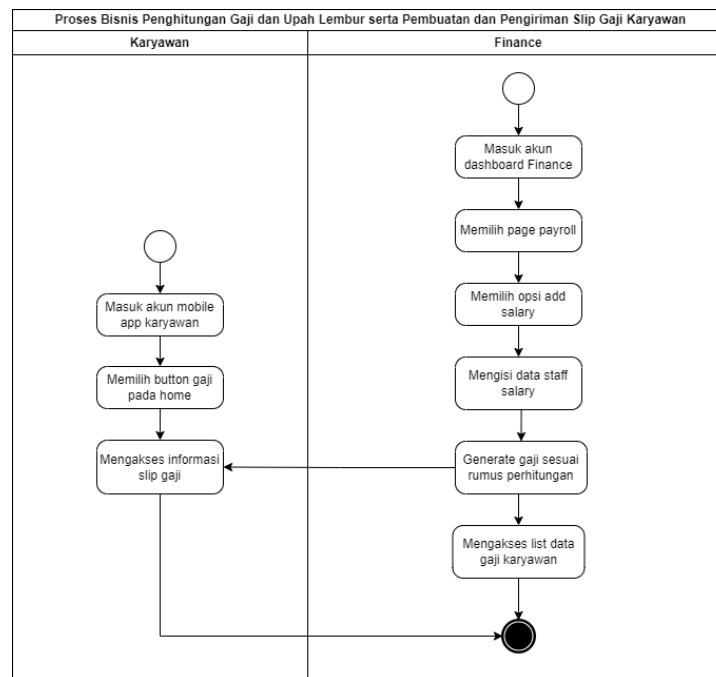
Selain melakukan presensi, para karyawan dapat mengajukan permohonan izin dan cuti yang kemudian HRD akan membuat keputusan persetujuan dan penolakan terhadap permohonan tersebut. Proses bisnis permohonan dan persetujuan izin dan cuti karyawan dapat dilihat pada Gambar 3.8.



Gambar 3. 8 Proses bisnis permohonan dan persetujuan izin dan cuti karyawan

Proses pengajuan permohonan izin dan cuti karyawan dilakukan melalui fitur yang telah dikembangkan pada sistem informasi berbasis Android dan *web*. Karyawan memulai dengan mengisi *form* data pengajuan izin dan cuti pada aplikasi Android, yang kemudian data permohonannya diterima oleh HRD melalui sistem *website*. HRD kemudian masuk ke akun *dashboard* dan memilih halaman *Approval*. Selanjutnya, HRD mengakses daftar pengajuan izin dan cuti karyawan, menelaah setiap pengajuan, dan memberikan keputusan terhadap permohonan tersebut. Dengan prosedur baru ini, karyawan dapat dengan mudah mengajukan izin dan cuti, serta memantau status permohonan mereka secara *real-time*, sementara HRD dapat mengelola dan menyetujui permohonan dengan lebih terstruktur dan cepat.

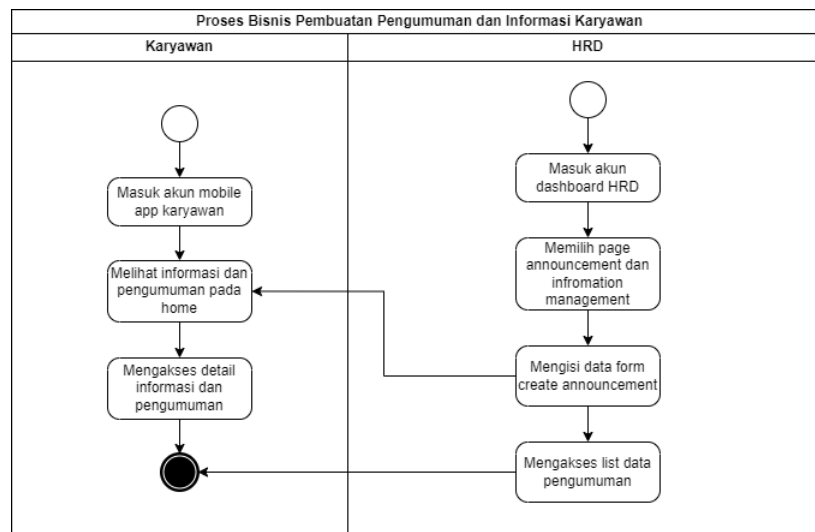
Setelah para karyawan melakukan presensi atau mengajukan permohonan izin ketika berhalangan untuk bekerja, maka proses bisnis selanjutnya adalah penghitungan gaji dan upah lembur oleh *finance* berdasarkan *history* presensi sesuai dengan jumlah masuk kerja, izin, dan cuti dari karyawan serta pembuatan dan pengiriman slip gaji kepada karyawan. Proses bisnis penghitungan gaji dan upah lembur serta pembuatan dan pengiriman slip gaji karyawan dapat dilihat pada Gambar 3.9.



Gambar 3. 9 Proses bisnis perhitungan gaji dan upah lembur

Proses penghitungan gaji dan upah lembur serta pembuatan dan pengiriman slip gaji karyawan dilakukan melalui fitur yang telah dikembangkan pada sistem informasi berbasis *web* dan *Android*. *Finance* memulai dengan masuk ke akun *dashboard* dan memilih halaman *Payroll*. Selanjutnya, *finance* memilih opsi *Add Salary* dan mengisi data gaji karyawan. Sistem kemudian akan secara otomatis menghitung gaji berdasarkan rumus perhitungan yang telah ditentukan, dan slip gaji yang telah dibuat akan langsung masuk ke *database*. Karyawan dapat mengakses slip gaji mereka melalui aplikasi *Android* dengan menekan tombol "Gaji" pada aplikasi tersebut. Selain itu, *finance* juga dapat mengakses daftar slip gaji karyawan melalui sistem *website*.

Setelah HRD menghitung gaji dan memberikan slip gaji kepada para karyawan, maka proses bisnis selanjutnya adalah pembuatan pengumuman dan informasi oleh HRD yang akan diberikan kepada para karyawan untuk memberi kabar atau informasi terkini terkait pekerjaan dan perusahaan. Proses bisnis pembuatan pengumuman dan informasi karyawan dapat dilihat pada Gambar 3.10.



Gambar 3. 10 Proses bisnis pembuatan pengumuman dan informasi karyawan

Proses pembuatan pengumuman dan informasi karyawan dilakukan melalui fitur yang telah dikembangkan pada sistem informasi berbasis *web* dan Android. HRD memulai dengan masuk ke akun *dashboard* dan memilih halaman *Announcement and Information Management*. Selanjutnya, HRD mengisi *form Create Announcement* dengan data yang diperlukan, yang kemudian akan secara otomatis disimpan ke dalam *database*. Pengumuman tersebut dapat langsung diakses oleh karyawan melalui aplikasi Android dengan melihat beranda aplikasi dan memilih halaman "Detail Pengumuman." Selain itu, HRD juga dapat untuk mengakses daftar pengumuman yang telah dibuat, yang dapat diedit atau dihapus sesuai kebutuhan melalui sistem *website*.

3.2.2.3. Prosedur

Terdapat aturan, prosedur, atau *business rules* yang baru akibat dengan pengembangan sistem yang ditawarkan. Prosedur proses pengelolaan SDM di CV MI Semarang setelah sistem yang diusulkan dibuat dapat dilihat pada Tabel 3.6.

Tabel 3. 6 Prosedur proses pengelolaan sumber daya manusia

Aktor	Aksi	Prosedur
HRD	Mendata dan membuat akun karyawan.	HRD mengisi <i>form</i> data karyawan termasuk <i>username</i> dan <i>password</i> , yang tersedia melalui sistem informasi berbasis web untuk membuat akun baru karyawan.

Aktor	Aksi	Prosedur
HRD	Membuat dan menetapkan jadwal kerja karyawan.	HRD membuat <i>shift</i> baru dan menetapkan jadwal kerja karyawan sesuai <i>shift</i> yang telah dibuat pada sistem informasi berbasis web. Hal tersebut mengurangi kemungkinan karyawan harus melihat informasi terkait jadwal dan <i>shift</i> kerja di papan tulis karena informasi jadwal kerja dapat langsung diakses melalui kalender kerja di sistem berbasis Android.
HRD	Mengatur lokasi presensi dan mengelola data presensi karyawan.	HRD dapat mengunggah data <i>excel</i> presensi karyawan dari mesin presensi <i>onsite</i> ke dalam sistem informasi berbasis <i>website</i> . Prosedur ini bertujuan untuk meningkatkan efisiensi waktu dalam pengolahan data presensi karyawan, mengurangi kebutuhan <i>input</i> data manual, dan meminimalkan kesalahan. Selain itu, HRD dapat menambahkan lokasi presensi karyawan untuk presensi berbasis Android, sehingga karyawan hanya dapat melakukan presensi di lokasi yang telah ditentukan. Setelah data diunggah dan karyawan melakukan presensi, HRD dapat mengakses daftar riwayat presensi karyawan yang mencakup data dari mesin presensi <i>onsite</i> dan presensi berbasis Android. Proses ini memungkinkan HRD untuk memantau dan mengelola kehadiran karyawan dengan lebih efektif dan akurat.
HRD	Melakukan persetujuan dan penolakan terhadap pengajuan permohonan izin dan cuti karyawan.	HRD melakukan persetujuan dan penolakan terhadap pengajuan izin dan cuti karyawan melalui sistem informasi berbasis web. Prosedur ini dimulai dengan HRD mengakses daftar pengajuan di <i>website</i> , menelaah setiap permohonan, dan memberikan keputusan permohonan tersebut. Metode

Aktor	Aksi	Prosedur
		baru ini menggantikan penggunaan WhatsApp, yang meningkatkan keamanan dan privasi data, efisiensi waktu, serta memberikan rekam jejak yang terstruktur. Selain itu, transparansi dan akuntabilitas proses meningkat, karena karyawan dapat melacak status pengajuan mereka.
HRD	Membuat pengumuman dan informasi karyawan.	HRD membuat pengumuman dan informasi karyawan dengan mengisi data <i>form</i> pada sistem <i>website</i> , yang kemudian dapat langsung dikirimkan ke karyawan sehingga mereka dapat melihatnya melalui akun mereka pada sistem Android. Prosedur baru ini menggantikan metode lama yang menggunakan papan tulis untuk membagikan informasi dan pengumuman perusahaan. Manfaat dari prosedur baru ini adalah informasi dapat disebarkan dengan efisien dan semua karyawan dapat mengakses pengumuman kapan saja dan di mana saja.
Finance	Menghitung gaji dan upah lembur, serta mengirimkan slip gaji kepada karyawan.	Pihak <i>finance</i> menghitung gaji dan upah lembur secara otomatis dengan mengisi data gaji karyawan pada <i>form</i> di sistem informasi berbasis web. Data tersebut akan dihitung berdasarkan rumus perhitungan gaji yang telah ditentukan, sehingga slip gaji akan langsung terbuat dan terkirim ke karyawan. Prosedur ini menggantikan metode lama di mana gaji karyawan dihitung secara manual oleh <i>finance</i> , yang berisiko menimbulkan kesalahan. Manfaat dari prosedur baru ini adalah peningkatan efisiensi dan kecepatan proses penggajian, pengurangan risiko kesalahan perhitungan, serta kemudahan

Aktor	Aksi	Prosedur
		dalam distribusi slip gaji kepada karyawan.

3.2.2.4. Target Service Time

Target *service time* setelah sistem ini dikembangkan dapat dilihat secara lebih rinci pada Tabel 3.7.

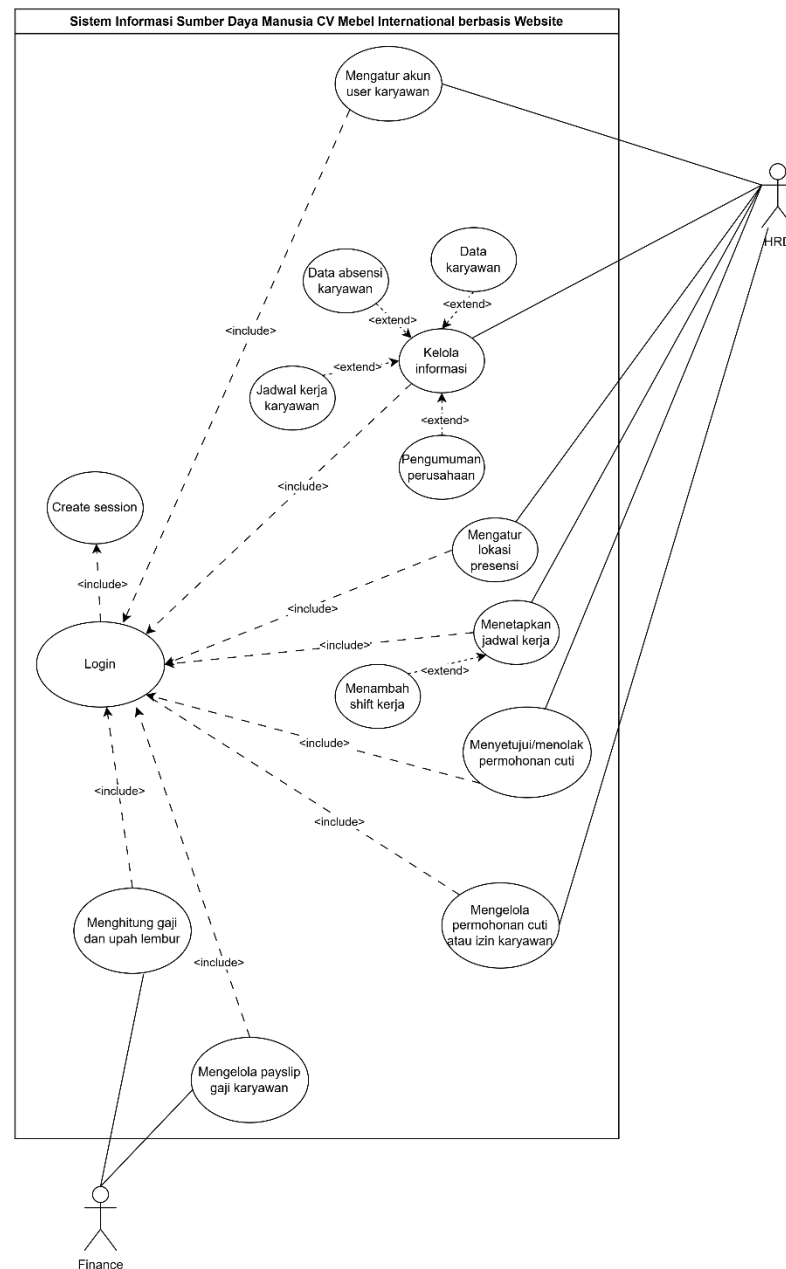
Tabel 3. 7 Target *service time*

Layanan	Keterangan
Pendataan karyawan, penjadwalan kerja, presensi, dan pendataan presensi.	Dengan adanya sistem baru yang dikembangkan, Target <i>Service Time</i> untuk proses Pendataan Karyawan, Penjadwalan Kerja, Presensi, dan Pendataan Presensi mengalami peningkatan efisiensi yang signifikan. HRD kini dapat mengisi <i>form</i> data karyawan untuk membuat akun baru hanya dalam waktu sekitar 1-2 menit. Penjadwalan kerja karyawan dengan sistem kalender kerja di Android membutuhkan waktu sekitar 0-1 menit per <i>shift</i> , mengurangi waktu sebelumnya yang memakan 30 menit hingga 1 jam per minggu. Proses pendataan presensi yang sebelumnya memakan waktu 1-2 jam per minggu, kini dapat diselesaikan dalam waktu sekitar 5-10 menit per karyawan untuk pengunggahan data dan presensi berbasis Android, mengurangi waktu secara signifikan. Dengan demikian, <i>service time</i> untuk layanan tersebut berkurang drastis dari sebelumnya.
Penghitungan gaji dan upah lembur serta pembuatan dan pengiriman slip gaji.	Dengan sistem baru, penghitungan gaji dan upah lembur serta pembuatan dan pengiriman slip gaji menjadi lebih efisien. Pihak <i>finance</i> kini dapat menghitung gaji dan upah lembur secara otomatis melalui sistem informasi berbasis web, mengurangi waktu yang sebelumnya memakan waktu 3-5 menit per karyawan menjadi hanya sekitar 1-2 menit per karyawan. Selain itu, karyawan dapat mengakses detail informasi slip gaji langsung pada sistem berbasis Android, menghilangkan proses permintaan dan pengiriman slip gaji melalui aplikasi WhatsApp yang sebelumnya memakan waktu sekitar 2-3 menit per karyawan. Dengan demikian, total <i>service time</i> untuk penghitungan gaji dan upah lembur serta distribusi slip gaji menjadi lebih cepat dan efisien.
Permohonan dan persetujuan izin dan cuti.	Dengan sistem baru, HRD dapat melakukan persetujuan dan penolakan terhadap pengajuan izin dan cuti karyawan secara langsung melalui sistem informasi berbasis web. Proses ini

Layanan	Keterangan
	mempersingkat waktu akses dan persetujuan, hanya membutuhkan sekitar 1-2 menit untuk menelaah dan memberikan keputusan terhadap setiap pengajuan. Dibandingkan dengan prosedur sebelumnya yang menggunakan WhatsApp, yang memakan waktu lebih lama karena karyawan harus mengirim pesan dan menunggu balasan, sistem baru ini mengurangi <i>service time</i> secara signifikan. Selain itu, penggunaan sistem berbasis Android untuk mengajukan permohonan juga menghemat waktu karena karyawan tidak perlu lagi mengajukan izin dan cuti secara terpisah melalui aplikasi WhatsApp.
Pembuatan pengumuman dan informasi.	Dengan sistem baru, HRD dapat membuat pengumuman dan informasi karyawan secara langsung melalui sistem <i>website</i> , yang kemudian dapat langsung dikirimkan ke karyawan. Proses ini mempersingkat waktu penyebaran informasi menjadi hanya sekitar 1-2 menit per pengumuman, jauh lebih efisien dibandingkan dengan metode lama yang memakan waktu 30-45 menit per pengumuman. Selain itu, karyawan dapat dengan cepat mengakses detail informasi dan pengumuman perusahaan melalui sistem berbasis Android, menggantikan prosedur sebelumnya yang melibatkan papan tulis perusahaan. Dengan demikian, <i>service time</i> untuk layanan pembuatan pengumuman dan informasi mengalami peningkatan signifikan dalam efisiensi dan kecepatan akses informasi.

3.2.3 Perancangan Diagram Use Case

Diagram *use case* merupakan representasi grafis yang menggambarkan interaksi antara sistem dan aktor yang berperan dalam sistem. Dalam rekayasa perangkat lunak, diagram *use case* digunakan untuk memvisualisasikan fungsionalitas sistem serta bagaimana aktor berinteraksi dengan sistem dalam berbagai skenario atau situasi. Gambar 3.10 menampilkan diagram *use case* sistem, yang memperlihatkan hubungan antara pengguna dan fitur-fitur utama yang tersedia dalam sistem.



Gambar 3.11 Diagram *use case* sistem berbasis *website*

SISDM CV Mebel Internasional *Semarang* berbasis *website* memiliki dua peran utama pengguna, yaitu HRD dan *Finance*. Untuk dapat menggunakan sistem ini, pengguna harus masuk ke akun mereka melalui *browser* Google Chrome dengan menggunakan *email* dan kata sandi yang telah terdaftar. Setelah berhasil *login*, sistem akan membuat *session* untuk memastikan keamanan dan validitas akses pengguna selama sesi penggunaan berlangsung.

HRD memiliki akses ke berbagai fitur yang mendukung pengelolaan data kepegawaian, termasuk pengelolaan akun karyawan, manajemen presensi, serta penjadwalan kerja. Dalam fitur pengelolaan jadwal kerja, HRD dapat menetapkan jadwal serta menambahkan *shift* kerja bagi karyawan. Selain itu, HRD juga bertanggung jawab dalam mengelola informasi internal perusahaan, termasuk pengumuman perusahaan, yang dapat diperbarui secara berkala melalui sistem.

Fitur presensi memungkinkan HRD untuk mengatur lokasi guna memastikan bahwa pencatatan kehadiran dilakukan secara akurat. HRD juga memiliki kewenangan untuk menyetujui atau menolak permohonan cuti dan izin yang diajukan oleh karyawan, serta mengelola data presensi yang digunakan dalam perhitungan gaji.

Finance memiliki akses terhadap fitur pengelolaan penggajian, yang mencakup penghitungan gaji dan upah lembur, serta pengelolaan slip gaji karyawan. Setiap proses penghitungan gaji dilakukan berdasarkan data presensi yang telah diverifikasi oleh HRD, sehingga memastikan akurasi dalam sistem pembayaran gaji.

Sistem ini juga dirancang untuk memastikan keamanan data dengan menerapkan mekanisme *session management*, yang memastikan bahwa setiap pengguna hanya dapat mengakses fitur sesuai dengan hak aksesnya. Dengan adanya sistem berbasis *web* ini, pengelolaan sumber daya manusia menjadi lebih efisien, terstruktur, dan dapat diakses secara *real-time* oleh HRD dan *Finance* dari perangkat *desktop*.

3.2.4 Perancangan Skenario Use Case

Diagram *use case* sistem dijelaskan lebih lanjut dalam bentuk tabel yang memuat skenario *use case*. Dalam SISDM CV Mebel Internasional Semarang berbasis *website*, HRD memiliki tanggung jawab dalam mengelola akun pengguna karyawan. HRD dapat membuat dan mengatur akun karyawan untuk memberikan akses ke sistem. Proses ini mencakup pembuatan akun baru, pengaturan akses pengguna, serta validasi data sebelum akun dapat digunakan. Setelah akun berhasil dibuat, karyawan dapat melakukan *login* menggunakan kredensial yang telah

diberikan oleh HRD. Skenario *use case* untuk HRD dalam membuat dan mengatur akun karyawan dapat dilihat pada Tabel 3.8.

Tabel 3.8 Skenario *use case* HRD membuat dan mengatur akun *user* karyawan

Use Case ID Number	1	
Use Case Name	Membuat dan mengatur akun <i>user</i> karyawan	
Use Case Description	<i>Use case</i> ini menggambarkan proses ketika HRD ingin membuat akun <i>user</i> baru untuk para karyawan baru	
Primary Actor	HRD	
Secondary Actor	Karyawan	
Pre-Condition	HRD telah <i>login</i> ke sistem	
Primary Flow of Events	User Action	System Response
	1. HRD memilih menu <i>users management</i> .	
		2. Sistem menampilkan halaman daftar pengguna.
	3. HRD memilih tombol <i>add user</i> .	
		4. Sistem menampilkan halaman formulir untuk membuat akun baru.
	5. HRD mengisi formulir dan menekan tombol simpan untuk menyimpan akun.	
Error Flow of Events		6. Sistem menyimpan data akun baru ke <i>database</i> .
	5a. HRD mengisi formulir tidak sesuai dengan ketentuan.	
Post-Condition		5b. Sistem menampilkan pesan kesalahan format.
	Karyawan dapat <i>login</i> ke sistem dengan akun baru yang telah dibuat oleh HRD.	

Selain pengelolaan akun pengguna, HRD juga bertanggung jawab dalam mengelola informasi jadwal kerja yang akan ditampilkan kepada karyawan. HRD dapat menambahkan *shift* kerja baru, menetapkan jadwal kerja, serta mengubah atau menghapus jadwal yang telah ada. Informasi jadwal kerja ini akan langsung tersedia dan dapat diakses oleh karyawan melalui sistem. Skenario *use case* untuk pengelolaan informasi jadwal kerja dijelaskan lebih lanjut pada Tabel 3.9.

Tabel 3.9 Skenario *use case* HRD membuat dan mengatur akun *user* karyawan

Use Case ID Number	2
Use Case Name	Kelola informasi jadwal kerja
Use Case Description	<i>Use case</i> ini menggambarkan proses ketika HRD mengelola informasi seperti jadwal kerja untuk ditampilkan ke karyawan.
Primary Actor	HRD
Secondary Actor	Karyawan

Pre-Condition	HRD telah <i>login</i> ke sistem	
Primary Flow of Events	User Action	System Response
	1. HRD memilih menu <i>schedule</i> dan <i>shift management</i> .	
		2. Sistem menampilkan halaman kalender jadwal.
	3. HRD memilih tombol <i>add shift/assign schedule</i>	
		4. Sistem menampilkan halaman formulir untuk membuat <i>shift</i> baru atau langsung menetapkan jadwal.
	5. HRD mengisi formulir dan menekan tombol simpan untuk menyimpan <i>shift</i> dan jadwal.	
Error Flow of Events		6. Sistem menyimpan data <i>shift</i> dan jadwal baru ke <i>database</i> .
	5a. HRD mengisi formulir tidak sesuai dengan ketentuan.	
		5b. Sistem menampilkan pesan kesalahan format.
Post-Condition	Karyawan dapat melihat jadwal kerja pada kalender sistem Android.	

Sistem ini juga menyediakan fitur pengelolaan pengumuman perusahaan, yang memungkinkan HRD untuk membuat, memperbarui, dan menghapus pengumuman yang ditujukan bagi karyawan. Pengumuman terbaru akan ditampilkan di halaman utama sistem dan dapat diakses kapan saja oleh karyawan. Dengan fitur ini, informasi perusahaan dapat disampaikan dengan lebih cepat dan efisien. Skenario *use case* untuk HRD dalam mengelola informasi pengumuman perusahaan dapat dilihat pada Tabel 3.10.

Tabel 3. 10 Skenario *use case* HRD mengelola informasi pengumuman perusahaan

Use Case ID Number	3	
Use Case Name	Kelola informasi pengumuman perusahaan	
Use Case Description	<i>Use case</i> ini menggambarkan proses ketika HRD mengelola informasi seperti pengumuman untuk ditampilkan ke karyawan.	
Primary Actor	HRD	
Secondary Actor	Karyawan	
Pre-Condition	HRD telah <i>login</i> ke sistem	
Primary Flow of Events	User Action	System Response
	1. HRD memilih menu <i>announcement management</i> .	
		2. Sistem menampilkan halaman daftar pengumuman.
	3. HRD memilih tombol <i>create announcement</i> .	

		4. Sistem menampilkan halaman formulir untuk membuat pengumuman baru.
	5. HRD mengisi formulir dan menekan tombol simpan untuk menyimpan pengumuman.	
	6. HRD dapat mengubah dan menghapus pengumuman.	
		7. Sistem menyimpan data <i>shift</i> dan jadwal baru ke <i>database</i> .
Error Flow of Events	5a. HRD mengisi formulir tidak sesuai dengan ketentuan.	
		5b. Sistem menampilkan pesan kesalahan format.
Post-Condition	Karyawan dapat melihat pengumuman perusahaan pada sistem Android.	

HRD juga memiliki akses untuk melihat dan mengelola riwayat presensi karyawan melalui sistem. Fitur ini memungkinkan HRD untuk melakukan *monitoring* terhadap kehadiran karyawan, mengevaluasi data absensi, serta memastikan bahwa data yang tersimpan telah sesuai dengan ketentuan perusahaan. Skenario *use case* untuk HRD dalam mengelola informasi data *history* presensi dijelaskan pada Tabel 3.11.

Tabel 3. 11 Skenario *use case* HRD mengelola informasi data *history* presensi

Use Case ID Number	4	
Use Case Name	Kelola informasi data <i>history</i> presensi.	
Use Case Description	<i>Use case</i> ini menggambarkan proses ketika HRD mengelola informasi seperti data karyawan dan data presensi	
Primary Actor	HRD	
Secondary Actor	-	
Pre-Condition	HRD telah <i>login</i> ke sistem	
Primary Flow of Events	User Action	System Response
	1. HRD memilih menu <i>attendance</i> .	
		2. Sistem menampilkan halaman daftar <i>history</i> presensi.
Error Flow of Events	-	
		-
Post-Condition	HRD dapat melihat data <i>history</i> presensi karyawan.	

Selain itu, HRD dapat mengatur lokasi presensi karyawan, yang menentukan titik lokasi validasi saat karyawan melakukan presensi melalui sistem. Dengan adanya fitur ini, perusahaan dapat memastikan bahwa kehadiran karyawan

dicatat secara akurat berdasarkan lokasi yang telah ditentukan. Skenario *use case* untuk HRD dalam mengatur lokasi presensi dijelaskan dalam Tabel 3.12.

Tabel 3. 12 Skenario *use case* HRD mengatur lokasi presensi

Use Case ID Number	5	
Use Case Name	Mengatur lokasi presensi	
Use Case Description	Use case ini menggambarkan proses ketika HRD mengatur lokasi presensi yang akan digunakan pada sistem Android.	
Primary Actor	HRD	
Secondary Actor	-	
Pre-Condition	HRD telah <i>login</i> ke sistem	
Primary Flow of Events	User Action	System Response
	1. HRD memilih menu <i>attendance management</i> .	
		2. Sistem menampilkan halaman daftar lokasi kantor.
	3. HRD dapat menambahkan, mengubah, dan menghapus lokasi kantor.	
		4. Sistem menampilkan formulir lokasi kantor.
	5. HRD mengisi formulir dan menekan tombol simpan.	
Error Flow of Events		6. Sistem akan merekam data lokasi presensi ke dalam <i>database</i> .
	5a. HRD mengisi data formulir tidak sesuai dengan format yang ditentukan.	
		5b. Sistem akan menampilkan pesan <i>error</i> .
Post-Condition	Karyawan dapat melakukan presensi sesuai lokasi yang telah ditentukan oleh HRD.	

Dalam hal manajemen cuti dan izin, HRD memiliki peran penting dalam menyetujui atau menolak permohonan cuti dan izin karyawan. Permohonan yang diajukan oleh karyawan akan ditinjau oleh HRD berdasarkan kebijakan perusahaan. Setelah diproses, sistem akan memberikan pemberitahuan kepada karyawan mengenai status persetujuan atau penolakan permohonan tersebut. Skenario *use case* untuk HRD dalam menyetujui atau menolak permohonan cuti dan izin dapat dilihat pada Tabel 3.13.

Tabel 3. 13 Skenario *use case* HRD menyetujui/menolak permohonan cuti dan izin

Use Case ID Number	6
Use Case Name	Menyetujui/menolak permohonan cuti dan izin

Use Case Description	Use case ini menggambarkan proses ketika HRD menyetujui/menolak pengajuan permohonan cuti atau izin dari karyawan.	
Primary Actor	HRD	
Secondary Actor	-	
Pre-Condition	HRD telah <i>login</i> ke sistem	
Primary Flow of Events	User Action	System Response
	1. HRD memilih menu <i>approval management</i> .	
		2. Sistem menampilkan halaman daftar permohonan izin dan cuti karyawan.
	3. HRD memilih tombol detail dari salah satu karyawan.	
		4. Sistem menampilkan halaman detail dari salah satu karyawan.
	5. HRD meninjau permohonan dan bisa memilih tombol setuju/tolak.	
		6. Sistem menyimpan informasi persetujuan/penolakan dari HRD.
Error Flow of Events		7. Sistem menampilkan pemberitahuan persetujuan/penolakan ke halaman karyawan pemohon.
	-	-
Post-Condition	Karyawan dapat melihat pemberitahuan persetujuan/penolakan izin dan cuti dari HRD.	

Finance memiliki peran utama dalam menghitung gaji dan upah lembur karyawan berdasarkan data yang telah di-*input* dalam sistem. Sistem secara otomatis melakukan perhitungan berdasarkan rumus yang telah ditentukan oleh perusahaan, sehingga memastikan transparansi dan akurasi dalam pembayaran gaji. Setelah slip gaji dibuat, sistem akan menampilkan notifikasi kepada karyawan bahwa slip gaji mereka telah tersedia. Skenario *use case* untuk *Finance* dalam menghitung gaji dan upah lembur dapat dilihat pada Tabel 3.14.

Tabel 3. 14 Skenario *use case finance* menghitung gaji dan upah lembur

Use Case ID Number	7
Use Case Name	Menghitung gaji dan upah lembur
Use Case Description	Use case ini menggambarkan proses ketika <i>finance</i> menghitung gaji dan upah lembur karyawan berdasarkan rumus perhitungan yang telah ditentukan.
Primary Actor	<i>Finance</i>
Secondary Actor	-

Pre-Condition	Finance telah login ke sistem	
Primary Flow of Events	User Action	System Response
	1. Finance memilih menu payroll.	
		2. Sistem menampilkan halaman slip gaji/payroll.
	3. Finance memilih tombol add staff salary	
		4. Sistem menampilkan halaman formulir gaji karyawan.
	5. Finance mengisi formulir gaji karyawan dengan data yang sesuai dengan rumus perhitungan.	
		6. Sistem membuat slip gaji sesuai isi data yang telah di-input oleh finance.
Error Flow of Events		7. Sistem menampilkan pemberitahuan slip gaji ke halaman karyawan.
	5a. Finance mengisi data formulir tidak sesuai dengan format yang ditentukan.	
		5b. Sistem akan menampilkan pesan error.
Post-Condition	Karyawan dapat melihat pemberitahuan dan detail slip gaji dari finance.	

Selain melakukan perhitungan gaji, Finance juga memiliki akses untuk mengelola slip gaji karyawan yang telah dihasilkan oleh sistem. Finance dapat meninjau dan mengakses slip gaji yang telah dibuat, memastikan bahwa data gaji karyawan tersimpan dengan benar, serta memberikan akses kepada karyawan untuk melihat slip gaji mereka secara langsung melalui sistem. Skenario *use case* untuk Finance dalam mengakses informasi slip gaji karyawan dijelaskan pada Tabel 3.15.

Tabel 3.15 Skenario *use case* finance mengakses informasi slip gaji karyawan

Use Case ID Number	8	
Use Case Name	Mengakses informasi slip gaji karyawan	
Use Case Description	Use Case ini menggambarkan proses mengakses slip gaji karyawan yang di-generate oleh finance.	
Primary Actor	Finance	
Secondary Actor	HRD	
Pre-Condition	Finance telah login ke sistem	
Primary Flow of Events	User Action	System Response
	1. Finance memilih menu gaji pada beranda sistem.	
		2. Sistem menampilkan daftar slip gaji.

	3. <i>Finance</i> memilih salah satu slip gaji pada halaman daftar slip gaji.	
		4. Sistem menampilkan detail isi slip gaji karyawan.
Error Flow of Events	-	
		-
Post-Condition	Karyawan dan <i>finance</i> dapat melihat detail isi slip gaji	

3.2.5 Perancangan Basis Data

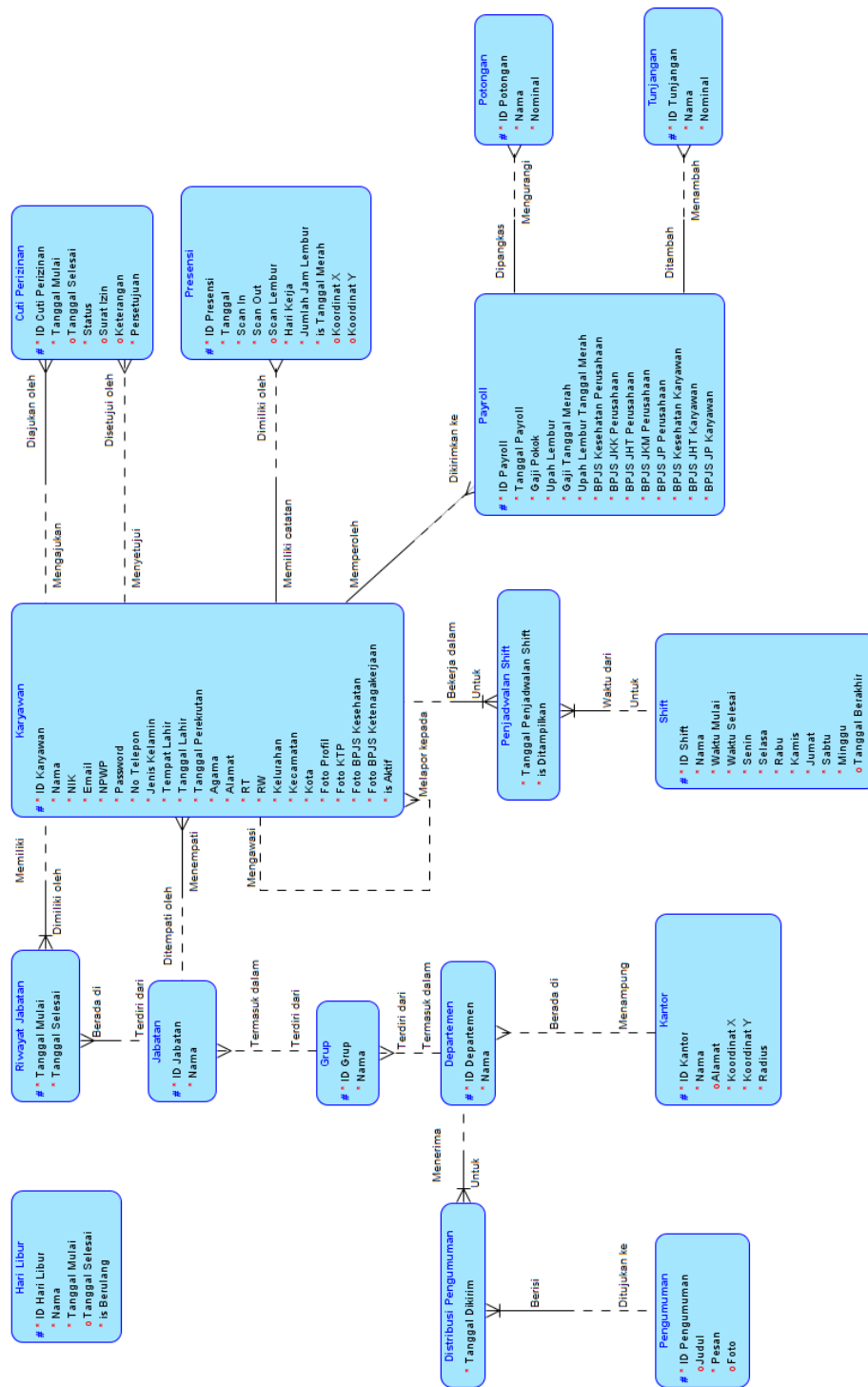
Sebelum mendesain sistem basis data untuk SISDM CV Mebel Internasional Semarang, dilakukan analisis kebutuhan data berdasarkan proses bisnis yang berjalan di perusahaan. Proses ini bertujuan untuk memastikan bahwa struktur basis data dirancang secara efisien, baik dari segi jumlah tabel, tipe data, maupun relasi antar tabel, sehingga dapat mengoptimalkan kinerja sistem dalam pengelolaan informasi kepegawaian.

Setelah mengumpulkan seluruh informasi dari HRD dan Finance mengenai kebutuhan sistem, dilakukan perancangan basis data menggunakan *Entity Relationship Diagram* (ERD). ERD adalah diagram berbentuk notasi grafis yang berada dalam pembuatan *database* yang menghubungkan antara data satu dengan yang lain. Fungsi ERD adalah sebagai alat bantu dalam pembuatan *database* dan memberikan gambaran bagaimana kerja *database* yang akan dibuat. ^[23]

Dalam SISDM, terdapat tiga jenis aktor utama, yaitu HRD, *Finance*, dan Karyawan. Oleh karena itu, desain basis data mencakup entitas user, roles, dan *role_user*, yang digunakan untuk mengatur hak akses setiap pengguna dalam sistem. Selain itu, desain basis data juga menyesuaikan dengan alur kerja operasional, seperti pengelolaan absensi, jadwal kerja, pengumuman, pengajuan cuti, serta penggajian. Dengan demikian, setiap data yang memiliki keterkaitan dengan tabel lain dapat dikelola dengan relasi yang efisien, sehingga mengurangi redundansi dan memastikan konsistensi data dalam sistem. ERD dari sistem yang dikembangkan dapat dilihat pada Gambar 3.11.

Setelah perancangan selesai, basis data diimplementasikan dalam *Relational Database Management System* (RDBMS) MySQL, yang dioptimalkan dengan

indeks dan relasi tabel untuk menjamin performa dan skalabilitas sistem dalam menangani transaksi data secara *real-time*.



Gambar 3. 12 ERD Sistem Informasi Sumber Daya Manusia di CV Mebel International Semarang

3.2.6 Perancangan *Endpoint*

Dalam pengembangan SISDM CV Mebel Internasional Semarang berbasis *website*, terdapat sekumpulan *endpoint* RESTful API yang digunakan untuk menghubungkan *frontend web* dengan *server backend*. *Endpoint* ini memungkinkan berbagai proses utama dalam sistem, termasuk autentikasi pengguna, pengelolaan karyawan, pengajuan izin, pencatatan presensi, manajemen jadwal kerja, pengelolaan penggajian, serta pengumuman perusahaan. Daftar lengkap *endpoint* REST API yang digunakan dalam sistem ini dijelaskan lebih lanjut pada Tabel 3.16.

Tabel 3.16 *Endpoint* RESTful API

Method	Route Path	Keterangan
POST	/api/auth/register	<i>Endpoint</i> ini digunakan untuk registrasi pengguna baru, yang hanya dapat dilakukan oleh HRD.
POST	/api/auth/login	<i>Endpoint</i> ini digunakan untuk <i>login</i> HRD dan Finance dengan mengirimkan <i>email</i> dan <i>password</i> .
POST	/api/auth/logout	<i>Endpoint</i> ini digunakan untuk <i>logout</i> pengguna, yang akan menghapus token autentikasi aktif.
POST	/api/auth/password/reset	<i>Endpoint</i> ini digunakan untuk reset <i>password</i> , hanya dapat diakses oleh pengguna yang telah <i>login</i> .
POST	/api/auth/password/forgot	<i>Endpoint</i> ini digunakan untuk mengajukan permintaan reset <i>password</i> . Jika <i>email</i> valid, sistem akan mengirimkan instruksi reset melalui <i>email</i> .
GET	/api/users	<i>Endpoint</i> ini digunakan untuk mengambil daftar semua pengguna dalam sistem.
GET	/api/users/{id}	<i>Endpoint</i> ini digunakan untuk mengambil detail informasi pengguna berdasarkan ID.
PUT	/api/users/{id}	<i>Endpoint</i> ini digunakan untuk memperbarui informasi pengguna berdasarkan ID.
DELETE	/api/users/{id}	<i>Endpoint</i> ini digunakan untuk menghapus pengguna dari sistem.
GET	/api/pengumuman	<i>Endpoint</i> ini digunakan untuk mengambil daftar pengumuman perusahaan yang dapat diakses oleh karyawan.
POST	/api/pengumuman	<i>Endpoint</i> ini digunakan untuk membuat pengumuman baru yang dapat ditampilkan kepada karyawan.

<i>Method</i>	<i>Route Path</i>	<i>Keterangan</i>
GET	/api/pengumuman/{id}	<i>Endpoint</i> ini digunakan untuk mengambil detail pengumuman tertentu berdasarkan ID.
PUT	/api/pengumuman/{id}	<i>Endpoint</i> ini digunakan untuk memperbarui pengumuman yang sudah ada dalam sistem.
DELETE	/api/pengumuman/{id}	<i>Endpoint</i> ini digunakan untuk menghapus pengumuman dari sistem.
GET	/api/attendance/history	<i>Endpoint</i> ini digunakan untuk mengambil riwayat presensi karyawan berdasarkan rentang tanggal tertentu.
POST	/api/attendance	<i>Endpoint</i> ini digunakan untuk mencatat presensi karyawan, termasuk data lokasi presensi.
GET	/api/cuti-perizinan	<i>Endpoint</i> ini digunakan untuk mengambil daftar cuti dan izin yang telah diajukan oleh karyawan.
POST	/api/cuti-perizinan	<i>Endpoint</i> ini digunakan untuk mengajukan permohonan cuti atau izin baru oleh karyawan.
PUT	/api/cuti-perizinan/{id}	<i>Endpoint</i> ini digunakan untuk memperbarui status permohonan cuti atau izin.
GET	/api/kalender	<i>Endpoint</i> ini digunakan untuk mengambil daftar hari kerja, jadwal <i>meeting</i> , dan hari libur perusahaan dalam kalender kerja.
GET	/api/users/{userId}/payroll	<i>Endpoint</i> ini digunakan untuk mengambil informasi penggajian karyawan berdasarkan <i>user ID</i> .
GET	/api/payroll/{payrollId}	<i>Endpoint</i> ini digunakan untuk mengambil detail penggajian berdasarkan ID slip gaji.
POST	/api/payroll/calculate	<i>Endpoint</i> ini digunakan untuk menghitung gaji karyawan, termasuk upah lembur dan tunjangan.
PUT	/api/payroll/{id}/mark-as-paid	<i>Endpoint</i> ini digunakan untuk menandai slip gaji sebagai telah dibayarkan.
POST	/api/tunjangan/store/{id_payroll}	<i>Endpoint</i> ini digunakan untuk menambahkan tunjangan ke dalam slip gaji karyawan.
PUT	/api/tunjangan/{id}	<i>Endpoint</i> ini digunakan untuk memperbarui informasi tunjangan dalam slip gaji.
DELETE	/api/tunjangan/{id}	<i>Endpoint</i> ini digunakan untuk menghapus tunjangan yang telah terdaftar dalam slip gaji.
POST	/api/potongan/store/{id_payroll}	<i>Endpoint</i> ini digunakan untuk menambahkan potongan ke dalam slip gaji karyawan.

<i>Method</i>	<i>Route Path</i>	<i>Keterangan</i>
PUT	/api/potongan/{id}	<i>Endpoint</i> ini digunakan untuk memperbarui informasi potongan gaji dalam sistem.
DELETE	/api/potongan/{id}	<i>Endpoint</i> ini digunakan untuk menghapus potongan yang telah diterapkan dalam slip gaji karyawan.

3.3 Metode Pengujian

Perangkat lunak yang dikembangkan dalam proyek ini dievaluasi menggunakan dua jenis pengujian utama, yaitu pengujian *load testing* dan *whitebox testing*. Selain itu, tingkat kompleksitas kode juga dianalisis menggunakan *cyclomatic complexity*. Setiap metode pengujian memiliki peran penting dalam memastikan bahwa SISDM CV Mebel Internasional Semarang berbasis *website* dapat berjalan dengan optimal, sesuai spesifikasi, mudah digunakan, serta mampu menangani beban kerja yang diharapkan.

- Load testing* dilakukan untuk menilai kinerja sistem saat menangani banyak pengguna secara bersamaan. Pengujian ini bertujuan untuk memastikan bahwa SISDM dapat menangani beban kerja yang tinggi, terutama pada saat proses *payroll*, absensi massal, atau pengelolaan pengumuman dalam jumlah besar. Parameter seperti *response time*, penggunaan CPU, serta kecepatan *query database* dianalisis untuk memastikan bahwa sistem tetap stabil dan responsif di bawah tekanan beban kerja tinggi.
- Pengujian *whitebox* dilakukan untuk menganalisis struktur kode *backend* yang dikembangkan menggunakan Laravel dan MySQL. Tujuan utama dari pengujian ini adalah untuk memastikan efisiensi algoritma, optimasi *query database*, serta pengelolaan logika bisnis dalam sistem. Pengujian dilakukan pada *controller*, *model*, dan *middleware* untuk memastikan bahwa setiap fungsi dapat berjalan dengan benar serta memiliki performa yang optimal. Selain itu, uji coba *unit testing* juga diterapkan untuk menguji modul tertentu secara individual.
- Analisis *cyclomatic complexity* dilakukan untuk menilai kompleksitas logika program pada kode sumber yang dikembangkan dalam sistem.

Tujuan dari analisis ini adalah untuk mengukur jumlah jalur independen yang dapat diambil dalam kode, yang mencerminkan seberapa rumit logika yang diterapkan dalam sistem. Dalam konteks SISDM, analisis *cyclomatic complexity* berguna untuk menilai kualitas dan keterbacaan kode pada bagian-bagian yang terlibat dalam proses *payroll*, absensi massal, dan pengelolaan pengumuman. Semakin tinggi nilai *cyclomatic complexity*, semakin kompleks dan berpotensi rentan terhadap *bug* atau kesalahan dalam pemrograman. Oleh karena itu, pengurangan nilai *cyclomatic complexity* dapat meningkatkan pemeliharaan kode dan memudahkan pengujian unit. Analisis ini dilakukan dengan memetakan titik-titik percabangan dalam kode seperti kondisi *if*, *loop*, dan *switch*, serta memberikan wawasan tentang apakah suatu bagian kode perlu disederhanakan untuk meningkatkan efisiensi dan pemeliharaan jangka panjang.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Sistem

Pada tahap implementasi, pengembangan sistem dilakukan dengan membagi proses ke dalam beberapa modul utama, yaitu penerapan basis data, pengaturan *routes*, pengelolaan *controllers*, serta proses *deployment*.

4.1.1 Implementasi Basis Data

Implementasi basis data yang digunakan oleh SISDM CV Mebel Internasional Semarang dilakukan menggunakan bahasa SQL dengan RDBMS MySQL. SQL digunakan untuk menyusun struktur basis data dan mengelola data di dalamnya, sehingga sistem dapat melakukan penyimpanan, pembaruan, dan pengambilan data secara efisien.

Pembuatan struktur basis data dilakukan menggunakan *Data Definition Language* (DDL), yang mencakup pembuatan tabel, pengaturan indeks, serta definisi relasi antar tabel. Sementara itu, proses manipulasi data dalam sistem, seperti penyisipan, pembaruan, dan penghapusan data karyawan, absensi, dan penggajian, dilakukan menggunakan *Data Manipulation Language* (DML).

Dalam pengembangannya, Laravel *Eloquent* ORM digunakan sebagai pustaka utama untuk mempermudah pengelolaan *database* melalui model yang mendukung DDL dan DML. Dengan bantuan *Eloquent* ORM, pendefinisian tabel dalam basis data tidak lagi dilakukan secara langsung dengan SQL, tetapi melalui pendefinisian model di Laravel. Setiap model dalam sistem mewakili tabel tertentu dan memiliki atribut dengan tipe data yang telah ditentukan. Selain itu, relasi antar model, seperti relasi antara tabel *user* dengan *role*, absensi, dan penggajian, didefinisikan menggunakan fitur *one-to-many*, *many-to-many*, dan *hasOne* dalam *Eloquent* ORM. Dengan pendekatan ini, pengembangan sistem menjadi lebih fleksibel, terstruktur, dan mudah dikelola.

Berikut adalah contoh pendefinisian model untuk tabel *users* dalam Laravel:

Setelah pendefinisian model dan relasi dalam SISDM CV Mebel Internasional Semarang, Laravel *Eloquent* ORM secara otomatis menerjemahkan model menjadi perintah SQL yang mencakup DDL dan DML. Proses ini memungkinkan sistem untuk membuat tabel, mengelola indeks, serta memelihara hubungan antar entitas dalam basis data MySQL.

Laravel mendukung migrasi *database* untuk mempermudah pengelolaan struktur basis data secara otomatis. Migrasi memungkinkan perubahan struktur *database* tanpa perlu mengedit langsung dalam MySQL. Berikut adalah contoh skema migrasi untuk tabel *users* yang dibuat menggunakan Laravel:

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->foreignId('id_atasan')->nullable()->constrained('users')
    ->onDelete('set null');           You, 2 weeks ago • First commit
    $table->string('nama');
    $table->char('nik', 16)->unique()->default('');
    $table->char('npwp', 16)->unique()->default('');
    $table->string('email')->unique()->index();
    $table->string('password')->default('');
    $table->string('no_telepon', 20)->unique()->default('');
    $table->enum('jenis_kelamin', ['P', 'L'])->default('L');
    $table->string('tempat_lahir')->default('');
    $table->date('tanggal_lahir')->nullable();
    $table->date('tanggal_perekrutan')->nullable();
    $table->date('tanggal_pemutusan_kontrak')->nullable();
    $table->string('agama')->default('');
    $table->string('pendidikan')->default('');
    $table->enum('status_perkawinan', ['Menikah', 'Belum menikah'])->default
    ('Belum menikah');
    $table->string('alamat')->default('');
    $table->string('rt')->nullable();
    $table->string('rw')->nullable();
    $table->string('kelurahan')->default('');
```

Gambar 4. 1 Skema tabel *users* pada *database migration* Laravel

Migrasi dapat dijalankan dengan perintah `php artisan migrate`. Perintah ini akan membuat tabel *users* dengan semua atribut yang telah didefinisikan, termasuk relasi dengan tabel lain.

Laravel juga mendukung *seeding*, yang digunakan untuk memasukkan data awal ke dalam basis data saat sistem pertama kali di-*install*. Berikut contoh *seeder* untuk tabel *users*:

```
DB::table('users')->insert([
    [
        'nama' => 'John Doe',
        'nik' => '1234567890123456',
        'email' => 'johndoe@example.com',
        'npwp' => '9876543210123456',
        'password' => Hash::make('password123'),
        'no_telepon' => '081234567890',
        'jenis_kelamin' => 'L',
        'tempat_lahir' => 'Jakarta',
        'tanggal_lahir' => '1990-05-15',
        'tanggal_perekrutan' => '2020-01-10',
        'agama' => 'Islam',
        'pendidikan' => 'S1 Teknik Informatika',
        'status_perkawinan' => 'Menikah',
        'alamat' => 'Jl. Merdeka No. 10',
        'kelurahan' => 'Menteng',
        'kecamatan' => 'Jakarta Pusat',
        'kabupaten_kota' => 'DKI Jakarta',
        'is_aktif' => true,
        'is_admin' => true,
        'is_archived' => false,
        'is_remote' => false,
    ]
]);
```

Gambar 4. 2 Seeder untuk tabel users

Seeder ini dapat dijalankan dengan perintah `php artisan db:seed --class=UserSeeder`. Dengan menggunakan *seeding*, sistem dapat langsung memiliki data awal seperti data karyawan, peran pengguna, dan status kepegawaian.

4.1.2 Konfigurasi Routes

Dalam pengembangan Sistem Informasi Sumber Daya Manusia (SISDM) CV Mebel Internasional Semarang, *routes* dibagi menjadi dua kategori utama, yaitu *routes* untuk *website* dan *routes* untuk API yang digunakan untuk komunikasi dengan aplikasi Android.

4.1.2.1. Routes Website

Routes yang digunakan dalam *website* SISDM menangani proses autentikasi, pengelolaan data pengguna, pengelolaan absensi, cuti, *payroll*, dan pengumuman. *Routes* ini menghubungkan *frontend* berbasis Vue.js dengan

backend Laravel yang bertanggung jawab untuk menangani logika bisnis dan operasi *database*.

```

routes > web.php
22 // Authentication Routes
23 Auth::routes();
24
25 Route::get('/', function () {
26     return view('welcome');
27 });
28
29 // Home Route
30 Route::get('/home', [HomeController::class, 'index'])->name('home');
31
32 // Protect all web routes for admin access only
33 Route::middleware(['auth', 'is_admin'])->group(function () {
34     // User Management
35     Route::patch('user/{id}/archive', [UserController::class, 'archive'])->name
36         ('user.archive');
37     Route::get('user/archived', [UserController::class, 'archivedUsers'])->name
38         ('user.archived');
39     Route::patch('user/{id}/restore', [UserController::class, 'restore'])->name
40         ('user.restore');
41     Route::resource('user', UserController::class);
42
43     // Attendance
44     Route::resource('attendance', AttendanceController::class);
45
46     // Shift Management
47     Route::get('shift/{shift}/assign', [ShiftController::class, 'assignForm'])->name
48         ('shift.assignForm');
49

```

Gambar 4. 3 Routes web.php

4.1.2.2. Routes API

Routes API digunakan untuk berkomunikasi dengan aplikasi Android. API ini digunakan untuk autentikasi pengguna, pencatatan presensi, pengelolaan cuti, pengumuman, dan *payroll*. Semua API ini dilindungi oleh *middleware* autentikasi Laravel Sanctum untuk memastikan keamanan data.

```

routes > api.php
26 // Auth Routes (Public)
27 Route::prefix('auth')->group(function () {
28     Route::post('/register', [AuthController::class, 'register']);
29     Route::post('/login', [AuthController::class, 'login']);
30     Route::post('/password/forgot', [PasswordController::class,
31         'sendResetLinkEmail']);
32 });
33
34 // Protected Routes (Require Authentication)
35 Route::middleware('auth:sanctum')->group(function () {
36     // Authenticated User Info
37     Route::get('/user', function (Request $request) {
38         return $request->user();
39     });
40
41     // Authenticated Actions
42     Route::post('/auth/logout', [AuthController::class, 'logout']);
43     Route::post('/auth/password/reset', [PasswordController::class, 'reset']);
44
45     // User Routes
46     Route::prefix('users')->group(function () {
47         Route::get('/', [UserController::class, 'index']);
48         Route::get('/{id}', [UserController::class, 'show']);
49         Route::put('/{id}', [UserController::class, 'update']);
50     });
51

```

Gambar 4. 4 Routes api.php

4.1.3 Konfigurasi *Middlewares*

Dalam SISDM CV Mebel Internasional Semarang, *middleware* digunakan untuk mengamankan, memproses, serta memfilter *request* yang masuk ke sistem. *Middleware* berfungsi sebagai lapisan pengaman dan pengelola *request* HTTP, memastikan bahwa hanya pengguna yang berwenang dapat mengakses fitur tertentu, serta menjaga integritas dan keamanan data dalam sistem.

4.1.3.1. *Middleware* Autentikasi dan Otorisasi

Middleware ini bertanggung jawab untuk memastikan bahwa hanya pengguna yang telah terautentikasi dan memiliki hak akses tertentu yang dapat menggunakan fitur sistem.

Middleware *Authenticate* digunakan untuk memverifikasi apakah pengguna telah *login* sebelum mengakses halaman yang dilindungi. Jika pengguna belum *login*, sistem akan mengarahkan mereka ke halaman *login*.

```
<?php
namespace App\Http\Middleware;

use Illuminate\Auth\Middleware\Authenticate as Middleware;
use Illuminate\Http\Request;

You, 2 weeks ago | 1 author (You)
class Authenticate extends Middleware
{
    /**
     * Get the path the user should be redirected to when they are not authenticated.
     */
    protected function redirectTo(Request $request): ?string
    {
        return $request->expectsJson() ? null : route('login');
    }
}
```

Gambar 4. 5 *Middleware* *Authenticate*

Middleware *IsAdmin* memastikan bahwa hanya admin yang dapat mengakses fitur tertentu dalam sistem, seperti manajemen pengguna, *payroll*, dan data kepegawaian.

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

You, 2 weeks ago | 1 author (You)
class IsAdmin
{
    public function handle(Request $request, Closure $next)
    {
        if (Auth::check() && Auth::user()->is_admin) {
            return $next($request);
        }

        return redirect('/')->with('error', 'Akses ditolak. Anda bukan admin.');
```

Gambar 4. 6 *Middleware IsAdmin*

Middleware RedirectIfAuthenticated digunakan untuk mencegah pengguna yang telah *login* untuk mengakses halaman *login* atau *register*.

```
class RedirectIfAuthenticated
{
    /**
     * Handle an incoming request.
     *
     * @param \Closure(\Illuminate\Http\Request):
     * (\Symfony\Component\HttpFoundation\Response) $next
     */
    public function handle(Request $request, Closure $next, string ...$guards):
    Response
    {
        $guards = empty($guards) ? [null] : $guards;

        foreach ($guards as $guard) {
            if (Auth::guard($guard)->check()) {
                return redirect(RouteServiceProvider::HOME);
            }
        }

        return $next($request);
    }
}
```

Gambar 4. 7 *Middleware RedirectIfAuthenticated*

4.1.3.2. *Middleware Keamanan dan Validasi*

Middleware kelompok keamanan dan validasi bertanggung jawab untuk mengamankan *request* dari ancaman eksternal, mencegah manipulasi data, serta memastikan validitas setiap *request* yang dikirimkan ke server.

Middleware VerifyCsrfToken digunakan untuk mencegah serangan CSRF (*Cross-Site Request Forgery*) dengan memastikan bahwa setiap *request* POST, PUT, dan DELETE memiliki *token* CSRF yang valid.

```
<?php

namespace App\Http\Middleware;

use Illuminate\Foundation\Http\Middleware\VerifyCsrfToken as Middleware;

You, 2 weeks ago | 1 author (You)
class VerifyCsrfToken extends Middleware
{
    /**
     * The URIs that should be excluded from CSRF verification.
     *
     * @var array<int, string>
     */
    protected $except = [
        //
    ];
}
```

Gambar 4. 8 *Middleware* VerifyCsrfToken

Middleware EncryptCookies mengenkripsi *cookie* yang dikirim dan diterima oleh browser untuk meningkatkan keamanan data pengguna.

```
<?php

namespace App\Http\Middleware;

use Illuminate\Cookie\Middleware\EncryptCookies as Middleware;

You, 2 weeks ago | 1 author (You)
class EncryptCookies extends Middleware
{
    /**
     * The names of the cookies that should not be encrypted.
     *
     * @var array<int, string>
     */
    protected $except = [
        //
    ];
}
```

Gambar 4. 9 *Middleware* EncryptCookies

Middleware ValidateSignature digunakan untuk memvalidasi tanda tangan URL yang telah dienkrpsi, mencegah manipulasi parameter URL yang dapat membahayakan sistem.

```

<?php

namespace App\Http\Middleware;

use Illuminate\Routing\Middleware\ValidateSignature as Middleware;

You, 2 weeks ago | 1 author (You)
class ValidateSignature extends Middleware
{
    /**
     * The names of the query string parameters that should be ignored.
     *
     * @var array<int, string> You, 2 weeks ago * first commit
     */
    protected $except = [
        // 'fbclid',
        // 'utm_campaign',
        // 'utm_content',
        // 'utm_medium',
        // 'utm_source',
        // 'utm_term',
    ];
}

```

Gambar 4. 10 *Middleware ValidateSignature*

4.1.3.3. *Middleware* Manajemen Sistem dan Optimasi

Middleware kategori manajemen sistem dan optimasi digunakan untuk mengoptimalkan performa sistem serta menangani pengaturan *server*.

Middleware TrustProxies digunakan untuk menentukan *proxy* yang dapat dipercaya dalam sistem.

```

class TrustProxies extends Middleware
{
    /**
     * The trusted proxies for this application.
     *
     * @var array<int, string>|string|null
     */
    protected $proxies;

    /**
     * The headers that should be used to detect proxies.
     *
     * @var int
     */
    protected $headers =
        Request::HEADER_X_FORWARDED_FOR |
        Request::HEADER_X_FORWARDED_HOST |
        Request::HEADER_X_FORWARDED_PORT |
        Request::HEADER_X_FORWARDED_PROTO |
        Request::HEADER_X_FORWARDED_AWS_ELB;
}

```

Gambar 4. 11 *Middleware TrustProxies*

4.1.4 Konfigurasi *Controllers*

4.1.4.1. *Controllers* Manajemen Akun dan Pengguna

Controllers dalam kategori ini digunakan untuk mengelola akun pengguna, termasuk pembuatan akun baru, pengarsipan akun, pemulihan akun, serta validasi data pengguna.

UserController digunakan untuk mengelola akun pengguna dalam sistem, termasuk pendaftaran pengguna baru, pembaruan profil, serta pengarsipan akun yang sudah tidak aktif. *Controller* ini juga menangani pengelolaan dokumen yang diunggah oleh karyawan, seperti foto profil, KTP, dan BPJS, serta memastikan bahwa data pengguna tetap valid dan dapat diakses oleh admin.

```
class UserController extends Controller
{
    use ImageStorage;

    public function __construct()
    {
        $this->middleware(['auth', 'is_admin']);
    }

    /**
     * Display a listing of the resource.
     */
    public function index(Request $request)
    {
        if ($request->ajax()) {
            try {
                $data = User::where('is_archived', false);

                return DataTables::of($data)
                    ->addColumn('action', function ($data) {
                        return view('layouts._action', [
                            'model' => $data,
                            'edit_url' => route('user.edit', $data->id),
                            'show_url' => route('user.show', $data->id),
                            'archive_url' => route('user.archive', $data->id),
                            'delete_url' => route('user.destroy', $data->id),
                        ]);
                    });
            }
        }
    }
}
```

Gambar 4. 12 *UserController*

4.1.4.2. *Controllers* Manajemen Jabatan, Departemen, dan Struktur Organisasi

Kategori ini mencakup pengelolaan jabatan, grup, departemen, serta kantor dalam perusahaan.

JabatanController mengatur pengelolaan jabatan di perusahaan, memungkinkan HRD untuk menambahkan, memperbarui, atau menghapus jabatan yang tersedia. Jabatan dalam sistem dikaitkan dengan grup dan departemen tertentu,

yang membantu dalam mengorganisir struktur organisasi perusahaan.

```
class JabatanController extends Controller
{
    public function index()
    {
        $jabatan = Jabatan::with('grup')->get();
        return view('pages.jabatan.index', compact('jabatan'));
    }

    public function create()
    {
        $grup = Grup::all();
        return view('pages.jabatan.create', compact('grup'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'id_grup' => 'nullable|exists:grup,id',
            'nama' => 'required|string|max:255|unique:jabatan,nama',
            'description' => 'nullable|string',
        ]);

        Jabatan::create($request->all());
        return redirect()->route('jabatan.index')->with('status', 'Jabatan created');
    }
}
```

Gambar 4. 13 JabatanController

RiwayatJabatanController menangani pencatatan riwayat jabatan karyawan, memastikan bahwa setiap karyawan memiliki satu jabatan aktif dalam satu waktu. Jika seorang karyawan mendapatkan jabatan baru, maka sistem akan mengharuskan pengguna untuk menutup jabatan sebelumnya sebelum menambahkan yang baru.

```
class RiwayatJabatanController extends Controller
{
    public function create($user_id)
    {
        $user = User::findOrFail($user_id);
        $jabatanList = Jabatan::with(['grup.departemen.kantor'])->get();

        return view('pages.riwayat_jabatan.create', compact('user', 'jabatanList'));
    }

    public function store(Request $request, $user_id)
    {
        $validatedData = $this->validateData($request);
        $validatedData['id_user'] = $user_id;

        $existingActiveJabatan = RiwayatJabatan::where('id_user', $user_id)
            ->whereNull('tanggal_selesai')
            ->exists();

        if ($existingActiveJabatan && $request->tanggal_selesai == null) {
            return redirect()->back()->withErrors(['error' => 'Karyawan sudah memiliki jabatan yang aktif. Harap berikan tanggal berakhirnya jabatan saat ini sebelum menambahkan jabatan baru.']);
        }
    }
}
```

Gambar 4. 14 RiwayatJabatanController

GrupController berperan dalam mengelola grup kerja dalam departemen, yang memungkinkan HRD untuk menentukan struktur kerja dalam perusahaan. Setiap grup dikaitkan dengan departemen tertentu, dan *controller* ini

bertanggung jawab dalam membuat, mengedit, serta menghapus grup kerja.

```
class GrupController extends Controller
{
    public function index()
    {
        $grup = Grup::with('departemen')->get();
        return view('pages.grup.index', compact('grup'));
    }

    public function create()
    {
        $departemen = Departemen::all();
        return view('pages.grup.create', compact('departemen'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'id_departemen' => 'nullable|exists:departemen,id',
            'nama' => 'required|string|max:255',
        ]);

        Grup::create($request->all());
        return redirect()->route('grup.index')->with('status', 'Grup created successfully!');
    }
}
```

Gambar 4. 15 GrupController

KantorController digunakan untuk mengelola data kantor perusahaan, termasuk alamat, koordinat lokasi, dan radius presensi karyawan. *Controller* ini juga menangani hubungan antara kantor dan manajer yang bertanggung jawab, memastikan bahwa setiap kantor memiliki manajer yang mengawasi operasional di lokasi tersebut.

```
class KantorController extends Controller
{
    public function index()
    {
        $kantor = Kantor::all();
        return view('pages.kantor.index', compact('kantor'));
    }

    public function create()
    {
        // Fetch only users with a jabatan containing "Manager"
        $managers = User::whereHas('riwayatJabatan', function ($query) {
            $query->whereNull('tanggal_selesai') // Only current positions
                ->whereHas('jabatan', function ($subQuery) {
                    $subQuery->where('nama', 'like', '%Manager%'); // Jabatan contains "Manager"
                });
        }->get();

        return view('pages.kantor.create', compact('managers'));
    }

    public function store(Request $request)
    {
        $validatedData = $this->validateKantor($request);
    }
}
```

Gambar 4. 16 KantorController

4.1.4.3. Controllers Manajemen Absensi dan Presensi Karyawan

Controllers dalam kategori ini digunakan untuk mengelola sistem kehadiran karyawan, termasuk pencatatan presensi harian, pengecekan keterlambatan, serta validasi lokasi absensi.

AttendanceController menangani pencatatan presensi karyawan, memastikan bahwa setiap karyawan hanya dapat melakukan presensi berdasarkan jadwal kerja yang telah ditentukan. *Controller* ini juga bertanggung jawab dalam memverifikasi lokasi presensi, memastikan bahwa karyawan hadir di lokasi kerja yang telah ditentukan sebelum melakukan *check-in* atau *check-out*. Selain itu, sistem juga menghitung keterlambatan serta jam lembur karyawan berdasarkan data presensi yang dikirimkan.

```
class AttendanceController extends Controller
{
    /**
     * Tampilkan daftar Attendance.
     */

    /**
     * Form untuk membuat Attendance baru.
     */
    use ImageStorage; // Jika Anda punya trait ini. Jika tidak, silakan buat fungsi
    upload sendiri.
    public function index()
    {
        // Contoh minimal:
        $attendances = Attendance::all();
        return view('pages.attendance.index', compact('attendances'));
    }

    /**
     * Tampilkan form create attendance.
     */
    public function create()
    {

```

Gambar 4. 17 AttendanceController

4.1.4.4. *Controllers* Manajemen Shift dan Jadwal Kerja

Kategori ini mencakup pengelolaan *shift* kerja dan penjadwalan karyawan berdasarkan divisi dan departemen.

ShiftController digunakan untuk mengelola jadwal kerja karyawan, memungkinkan HRD untuk membuat *shift*, menentukan jam kerja, serta mengatur hari kerja dalam seminggu. *Controller* ini juga menangani penugasan karyawan ke *shift* tertentu, serta memastikan bahwa setiap karyawan memiliki jadwal kerja yang sesuai dengan kebijakan perusahaan.

```

class ShiftController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $shifts = Shift::all();
        return view('pages.shift.index', compact('shifts'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        $departments = Departemen::all();
        return view('pages.shift.create', compact('departments'));
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {

```

Gambar 4. 18 ShiftController

4.1.4.5. *Controllers* Manajemen Cuti dan Izin Karyawan

Controllers dalam kategori ini digunakan untuk mengelola permohonan cuti dan izin karyawan, termasuk persetujuan atau penolakan permohonan oleh HRD.

CutiPerizinanController menangani proses pengajuan cuti dan izin karyawan, memungkinkan karyawan untuk mengajukan permohonan secara digital, serta memungkinkan HRD untuk menyetujui atau menolak pengajuan berdasarkan kebijakan perusahaan. *Controller* ini juga mencatat riwayat cuti setiap karyawan, sehingga HRD dapat dengan mudah melacak jumlah cuti yang telah digunakan.

```

class CutiPerizinanController extends Controller
{
    public function index()
    {
        $cutiPerizinans = CutiPerizinan::with('user')->get(['id', 'id_user',
        'tanggal_mulai', 'tanggal_selesai', 'keterangan', 'status_pengajuan']);
        return view('pages.cuti_perizinan.index', compact('cutiPerizinans'));
    }

    public function edit(CutiPerizinan $cutiPerizinan)
    {
        $users = User::all();
        return view('pages.cuti_perizinan.edit', compact('cutiPerizinan', 'users'));
    }

    public function update(Request $request, CutiPerizinan $cutiPerizinan)
    {
        $request->validate([
            'id_user' => 'required|exists:users,id',
            'tanggal_mulai' => 'required|date',
            'tanggal_selesai' => 'required|date|after_or_equal:tanggal_mulai',
            'keterangan' => 'required|string',
            'jenis' => 'required|in:izin,alpa,sakit',
        ]);
    }
}

```

Gambar 4. 19 CutiPerizinanController

4.1.4.6. *Controllers* Manajemen Penggajian dan *Payroll* Karyawan

Kategori ini mencakup penghitungan gaji, pengelolaan tunjangan, serta potongan gaji karyawan.

PayrollController bertanggung jawab dalam penghitungan gaji karyawan berdasarkan data kehadiran dan tunjangan yang diberikan. *Controller* ini memastikan bahwa setiap karyawan menerima gaji yang sesuai dengan jam kerja, jam lembur, serta gaji hari libur. Selain itu, *PayrollController* juga menangani *review* gaji sebelum pembayaran serta validasi oleh Finance. Sistem ini juga memungkinkan karyawan untuk melihat slip gaji mereka secara *online* setelah *payroll* diproses.

```

class PayrollController extends Controller
{
    public function index()
    {
        $payrolls = Payroll::with('user')->get();
        return view('pages.payroll.index', compact('payrolls'));
    }

    public function review($id)
    {
        $payroll = Payroll::with(['user', 'tunjangan', 'potongan'])->findOrFail($id);

        return view('pages.payroll.review', [
            'payroll' => $payroll,
            'tunjangan' => $payroll->tunjangan,
            'potongan' => $payroll->potongan,
        ]);
    }

    public function create()
    {
        $users = User::all();
        return view('pages.payroll.create', compact('users'));
    }
}

```

Gambar 4. 20 PayrollController

4.1.4.7. *Controllers* Manajemen Pengumuman dan Informasi Perusahaan

Controllers dalam kategori ini digunakan untuk mengelola pengumuman yang ditampilkan dalam sistem, baik untuk HRD maupun karyawan.

PengumumanController digunakan untuk mengelola informasi dan pengumuman perusahaan, memastikan bahwa HRD dapat menyampaikan informasi penting kepada karyawan melalui sistem. *Controller* ini memungkinkan HRD untuk menambahkan, memperbarui, dan menghapus pengumuman yang dapat diakses oleh semua karyawan dalam sistem.

```

class PengumumanController extends Controller
{
    public function index()
    {
        // Ambil data pengumuman dengan distribusi dan creator
        $pengumuman = Pengumuman::with(['distribusi', 'creator', 'updater'])->get();
        return view('pages.pengumuman.index', compact('pengumuman'));
    }

    public function create()
    {
        // Load semua departemen untuk pilihan distribusi
        $departemen = \App\Models\Departemen::all();
        return view('pages.pengumuman.create', compact('departemen'));
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'judul' => 'required|string|max:255',
            'pesan' => 'required',
            'foto' => 'nullable|image|max:2048',
            'departemen' => 'required|array', // Departemen harus berupa array
            'departemen.*' => 'exists:departemen,id', // Setiap ID harus ada di
            // tabel departemen
        ]);
    }
}

```

Gambar 4. 21 PengumumanController

4.1.4.8. Controllers Manajemen Halaman Utama dan Navigasi Sistem

Controllers dalam kategori ini bertanggung jawab atas tampilan *dashboard* serta pengelolaan akses sistem.

HomeController menangani halaman utama dan navigasi sistem, memastikan bahwa hanya pengguna yang telah *login* yang dapat mengakses *dashboard*. *Controller* ini bertanggung jawab dalam mengarahkan pengguna setelah *login* dan memastikan autentikasi yang sesuai dengan sistem.

```
class HomeController extends Controller
{
    /**
     * Create a new controller instance.
     *
     * @return void
     */
    public function __construct()
    {
        $this->middleware('auth');
    }

    /**
     * Show the application dashboard.
     *
     * @return \Illuminate\Contracts\Support\Renderable
     */
    public function index()
    {
        return view('home');
    }
}
```

Gambar 4. 22 HomeController

4.1.5 Deployment

Proses *deployment backend* SISDM dilakukan menggunakan layanan *Railway*, sebuah *Platform-as-a-Service* (PaaS) yang memungkinkan *deploy* aplikasi Laravel secara cepat tanpa perlu konfigurasi server secara manual. *Railway* dipilih karena kemudahan integrasi dengan GitHub, kemudahan dalam manajemen *database*, serta fleksibilitas dalam skala sumber daya yang digunakan.

Sebelum aplikasi *backend* Laravel di-*deploy* di *Railway*, beberapa konfigurasi perlu dilakukan untuk memastikan bahwa sistem berjalan dengan baik. *Railway* menyediakan server berbasis *container*, sehingga tidak memerlukan

konfigurasi sistem operasi secara manual seperti pada VPS. Railway juga memiliki dukungan otomatis untuk *database* MySQL, sehingga sistem dapat langsung terhubung dengan layanan *database* yang tersedia.

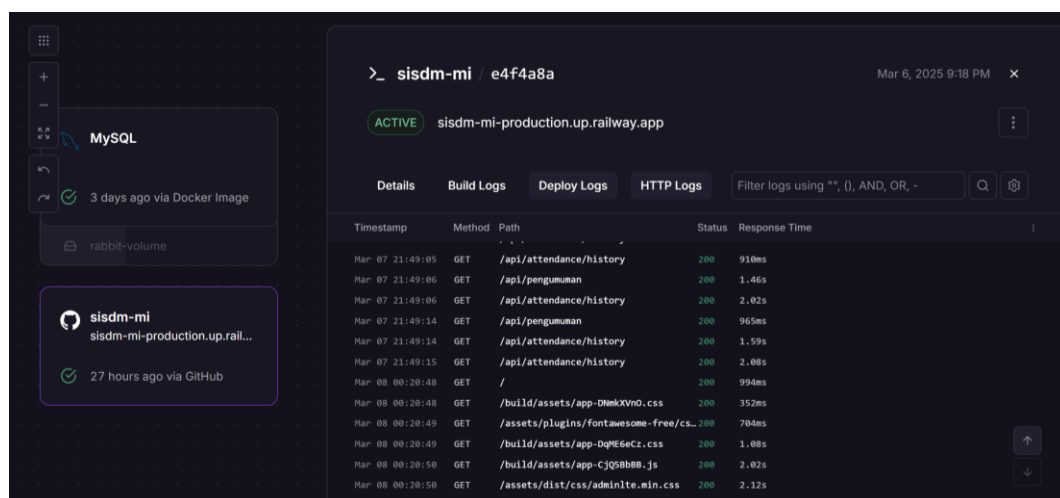
Langkah awal dalam deployment adalah menghubungkan *repository* GitHub proyek SISDM dengan Railway. Railway secara otomatis akan menjalankan proses *build* dan *deployment* setiap kali ada perubahan pada *repository*.

Setelah *repository* dihubungkan, beberapa konfigurasi *environment variables* (variabel lingkungan) harus disiapkan untuk menyesuaikan dengan kebutuhan aplikasi Laravel.

Sistem juga secara otomatis diberikan Sertifikat SSL (HTTPS), sehingga domain aplikasi seperti <https://sisdm-railway.app> akan berjalan dengan koneksi aman tanpa perlu konfigurasi tambahan seperti Certbot pada VPS.

Salah satu keunggulan Railway adalah automasi *deployment* yang memungkinkan setiap perubahan kode di *repository* GitHub langsung diterapkan ke server Railway.

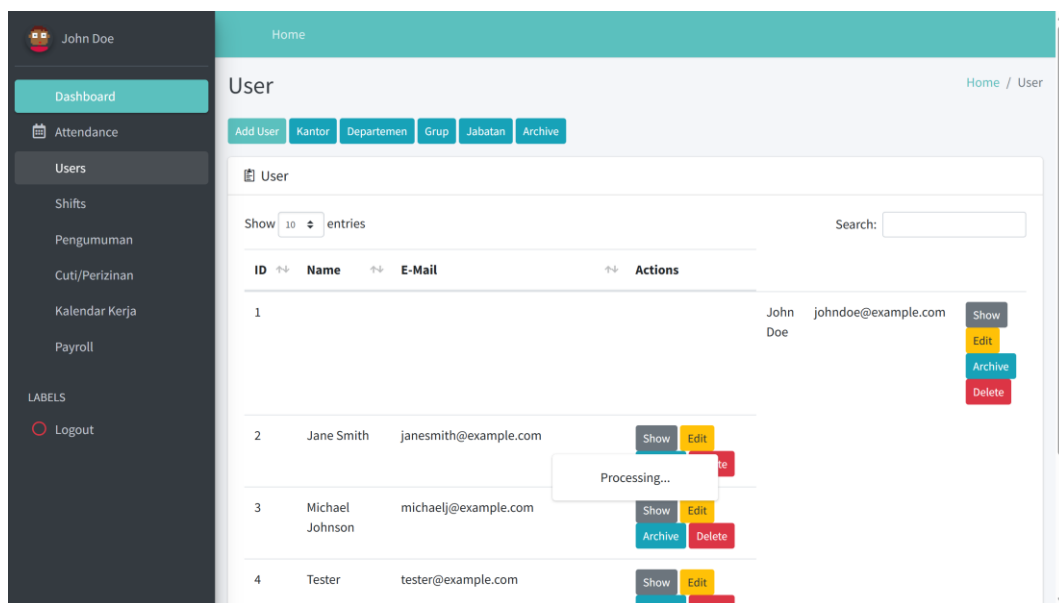
Railway menyediakan *dashboard monitoring*, yang memungkinkan tim pengembang untuk melihat penggunaan CPU, RAM, dan *database* secara *real-time*. Railway juga memungkinkan peningkatan skala aplikasi secara otomatis, sehingga jika jumlah pengguna meningkat, sistem dapat ditingkatkan tanpa perlu mengatur server secara manual.



Gambar 4. 23 Dashboard monitoring SISDM CV. MI pada platform Railway

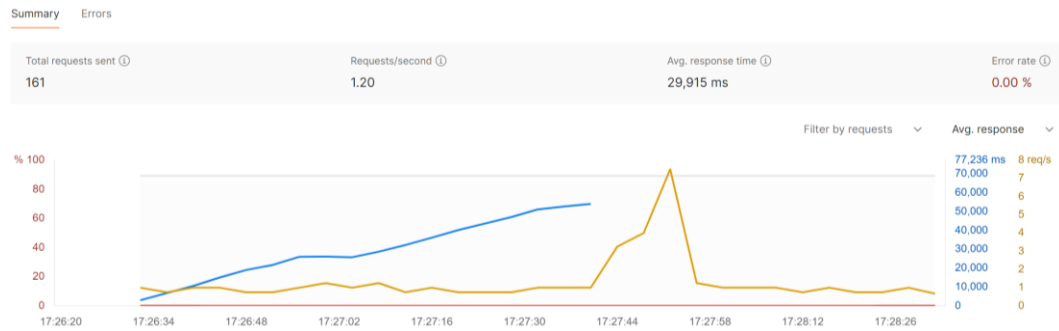
4.2 *Expert Feedback* terhadap Implementasi Awal

Berdasarkan hasil pengujian dan evaluasi yang dilakukan bersama pemangku kepentingan CV Mebel Internasional Semarang, ditemukan adanya permasalahan pada waktu respons *backend* yang lambat saat pengguna mengakses halaman daftar karyawan melalui aplikasi *web*. Hal ini berdampak pada kenyamanan pengguna karena memerlukan waktu yang cukup lama untuk memuat data, seperti yang terlihat pada Gambar 4. 24.



Gambar 4. 24 Tampilan *loading* data pengguna pada aplikasi *web* yang lambat

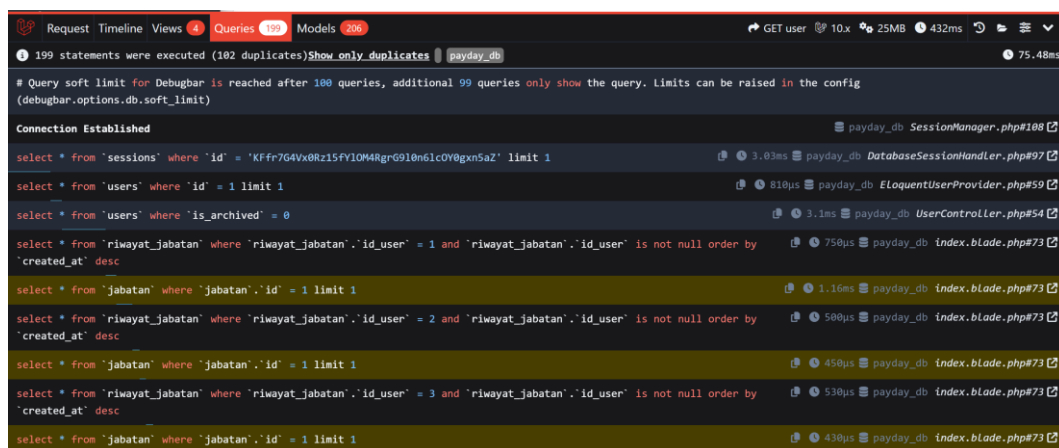
Gambar 4. 24 menunjukkan tampilan *loading data* pengguna pada aplikasi *web* yang berlangsung lambat, yang tentunya mempengaruhi pengalaman pengguna. Hasil pengujian *load testing* terhadap sistem *backend* menunjukkan performa sistem dalam menangani permintaan secara bersamaan. Pengujian ini dilakukan dengan menyimulasikan hingga 100 *virtual users* (VU) yang mengakses API secara bersamaan selama durasi 3 menit.



Gambar 4. 25 Hasil *load testing* API SISDM MI sebelum optimasi

Berdasarkan pengujian sebelum optimasi yang terlihat pada Gambar 4. 25, total permintaan yang berhasil diproses adalah sebanyak 161 *request*, dengan rata-rata waktu respons sebesar 29,915 ms (29,9 detik). Tingginya nilai waktu respons tersebut menunjukkan adanya *bottleneck* yang masih perlu ditangani oleh *backend* agar dapat memberikan pelayanan yang lebih optimal, khususnya saat menangani banyak permintaan secara bersamaan. Meskipun tidak ada *error* yang ditemukan dalam pengujian, rata-rata waktu respons yang dihasilkan masih jauh di atas nilai standar non-fungsional sistem (di bawah 5 detik).

Berdasarkan *profiling query* yang dilakukan menggunakan Laravel Debugbar, dapat terlihat bahwa banyaknya *query* yang dijalankan mengindikasikan perlunya optimasi *eager loading*, *pagination*, dan *indexing* pada kueri *database* untuk meningkatkan performa sistem.



Gambar 4. 26 *Profiling query* halaman *users* menggunakan Laravel Debugbar

Pada Gambar 4. 26, terlihat adanya banyak *query* untuk mengambil data yang saling terkait, seperti data pengguna dan detail absensi. Tanpa *eager loading*, Laravel melakukan N+1 *query problem*, yang berarti untuk setiap entri data, sistem

mengeluarkan *query* tambahan untuk mengambil data terkait. Ini sangat menghambat performa, terutama saat mengakses banyak data secara bersamaan. Menggunakan *eager loading* akan mengurangi jumlah *query* yang dieksekusi, sehingga waktu respons bisa lebih cepat.

Dengan melakukan optimasi menggunakan *eager loading*, *pagination*, dan *indexing*, diharapkan sistem dapat memproses permintaan dengan lebih efisien, sehingga dapat mengurangi *bottleneck* yang ada dan meningkatkan pengalaman pengguna secara keseluruhan.

4.3 Hasil Perbaikan *Expert Feedback*

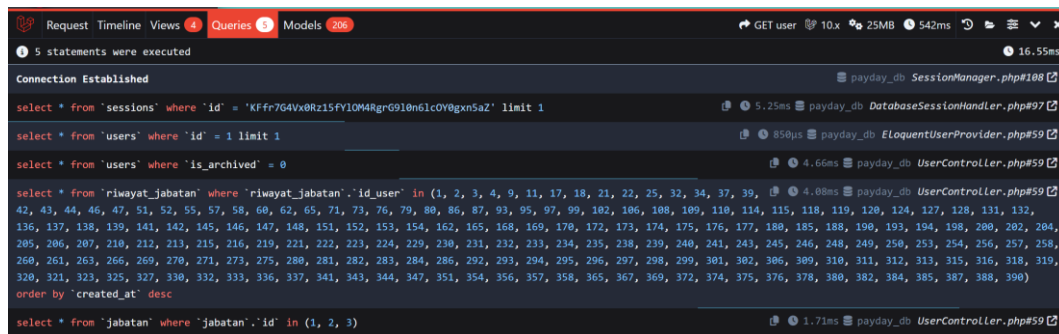
Sebelumnya, hasil *load testing* menunjukkan bahwa waktu respons rata-rata jauh melampaui standar non-fungsional yang diharapkan. Salah satu penyebab utama lambatnya performa adalah *N+1 Query Problem*, di mana setiap permintaan data *user* memicu banyak *query* tambahan untuk mengambil data dari tabel relasi *riwayatJabatan* dan *departemen*.

Untuk mengatasi masalah ini, diterapkan *eager loading* menggunakan metode `with()`, sehingga sistem dapat mengambil data *user* beserta relasinya dalam satu *query* besar, bukan dengan banyak *query* kecil yang dieksekusi secara terpisah. Implementasi *eager loading* diterapkan pada *UserController* dapat dilihat pada Gambar 4. 27.

```
public function index(Request $request)
{
    if ($request->ajax()) {
        try {
            $data = User::with(['riwayatJabatan', 'departemen'])
                ->where('is_archived', false);
```

Gambar 4. 27 Penerapan *eager loading* pada *UserController*

Dengan cara ini, jumlah *query* yang dieksekusi menjadi lebih sedikit dan lebih efisien.



Gambar 4. 28 *Profiling query* halaman *users* setelah penerapan *eager loading*

Selain menerapkan *eager loading*, optimasi berikutnya dilakukan dengan menggunakan *pagination*. Sebelumnya, sistem mengambil seluruh data *user* dalam satu permintaan, menyebabkan beban *query* yang besar dan memperlambat waktu eksekusi.

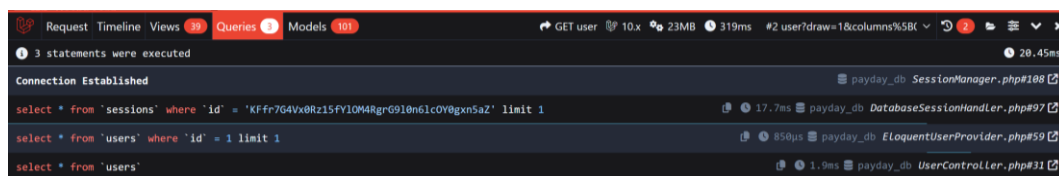
Dengan menerapkan metode `paginate()`, jumlah data yang diambil dalam satu permintaan dapat dibatasi, sehingga mengurangi beban *query* ke *database* dan mempercepat waktu respons. Implementasi *pagination* pada *UserController* dapat dilihat pada Gambar 4. 29.

```
$users = User::with(['riwayatJabatan.jabatan'])
    ->where('is_archived', false)
    ->paginate(10);

return view('pages.user.index', compact('users'));
```

Gambar 4. 29 Penerapan *pagination* pada *UserController*

Dengan cara ini, hanya 10 data per halaman yang diambil dalam satu *query*, sehingga waktu eksekusi *query* menjadi lebih cepat dibandingkan dengan mengambil seluruh data sekaligus seperti yang dapat dilihat pada hasil *profiling query* pada Gambar 4. 30.



Gambar 4. 30 *Profiling query* halaman *users* setelah *pagination*

Langkah optimasi terakhir adalah penerapan *indexing* pada beberapa kolom penting untuk meningkatkan efisiensi pencarian dan filtering data dalam *database*.

Penerapan *indexing* dilakukan dalam skema *database migration* sesuai dengan yang tertera pada Gambar 4. 31.

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->foreignId('id_atasan')->nullable()->constrained('users')
    ->onDelete('set null');
    $table->string('nama');
    $table->char('nik', 16)->unique()->default('');
    $table->char('npwp', 16)->unique()->default('');
    $table->string('email')->unique()->index();
    $table->string('password')->default('');
```

Gambar 4. 31 Penerapan *indexing* pada skema *database migration users*

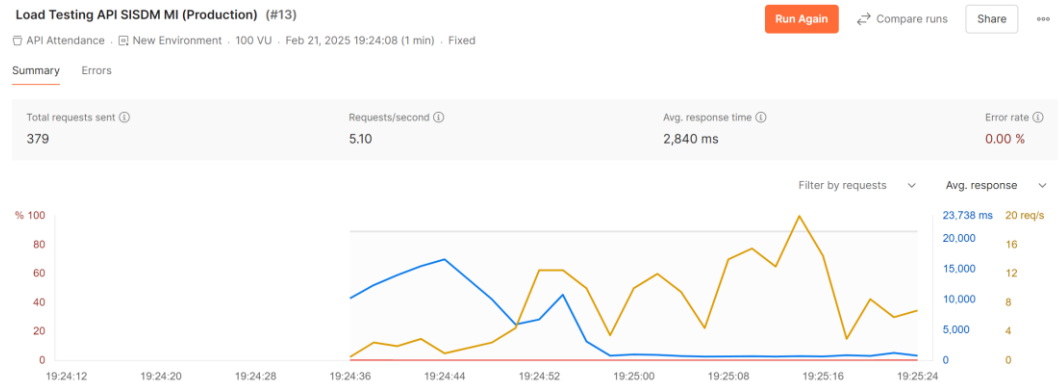
Dengan adanya *indexing* ini, proses pencarian dan *filtering* data menjadi lebih cepat, sehingga *query* yang dijalankan lebih efisien seperti yang dapat dilihat pada Gambar 4. 32.

Query	Execution Time
select * from 'sessions' where 'id' = 'KFfr7G4Vx8Rz15FY1Q4RgrG910n61c0Y0gxn5aZ' limit 1	7.79ms
select * from 'users' where 'id' = 1 limit 1	1.87ms
select count(*) as aggregate from 'users' where 'is_archived' = 0	2.87ms
select count(*) as aggregate from 'users' where 'is_archived' = 0 and 'email' like 'Xantwon%'	2.18ms
select * from 'users' where 'is_archived' = 0 and 'email' like 'Xantwon%' order by 'email' asc limit 10 offset 0	5.56ms
select * from 'riwayat_jabatan' where 'riwayat_jabatan`.`id_user` in (185) order by 'created_at' desc	1.5ms

Gambar 4. 32 *Profiling query* halaman *users* setelah *indexing*

Setelah ketiga metode optimasi ini diterapkan, dilakukan kembali *load testing* untuk mengukur peningkatan performa sistem. Hasilnya menunjukkan bahwa waktu respons rata-rata menurun drastis dari 29.915 ms (29,9 detik) menjadi 2.840 ms (2,8 detik).

Dengan hasil pada Gambar 4. 33, sistem kini telah memenuhi standar non-fungsional *response time* di bawah 5 detik, sehingga lebih optimal dalam menangani banyak permintaan secara simultan. *Backend* kini dapat memberikan respons yang lebih cepat dan stabil, meningkatkan efisiensi serta skalabilitas sistem dalam pengelolaan data karyawan dan operasional perusahaan.



Gambar 4. 33 Hasil Load Testing API Backend SISDM MI setelah optimasi

4.4 Pengujian Aplikasi

4.4.1 Whitebox Testing

Whitebox Testing adalah metode pengujian berbasis kode yang bertujuan untuk mengevaluasi struktur internal serta alur logika dalam perangkat lunak. Teknik ini memastikan bahwa setiap jalur eksekusi, kondisi, dan perulangan diuji secara menyeluruh guna mencegah terjadinya *bug* atau kesalahan. Beberapa pendekatan yang umum digunakan dalam *Whitebox Testing* meliputi *branch coverage*, *path coverage*, dan *loop testing*, yang bertujuan untuk menganalisis berbagai kemungkinan alur dalam kode program ^[24].

Proses pengujian *Whitebox* mencakup berbagai teknik, seperti *statement coverage* (mengukur sejauh mana pernyataan dalam kode telah dieksekusi), *branch coverage* (memastikan setiap percabangan logika diuji), serta *path coverage* (mengevaluasi semua jalur yang mungkin terjadi dalam eksekusi program). Dalam konteks pengujian aplikasi ini, *branch coverage* menjadi fokus utama untuk memastikan setiap cabang logika dalam fitur aplikasi berjalan sesuai desain serta mencegah potensi kesalahan saat *runtime*.

Untuk mendukung pengujian ini, digunakan Postman sebagai alat uji API, di mana pengembang dapat menguji berbagai skenario permintaan dan respons API guna mengevaluasi jalur eksekusi dalam sistem *backend*. Postman memastikan bahwa setiap jalur dalam API diuji secara sistematis dan mendukung analisis *response time* guna mengidentifikasi potensi *bottleneck* dalam sistem. Dengan fitur bawaan Postman seperti *Test Scripts* dan *Collection Runner*, pengujian API dalam

konteks *whitebox testing* dapat dilakukan dengan lebih terstruktur, memungkinkan analisis eksekusi kode berdasarkan skenario yang telah dirancang sebelumnya.

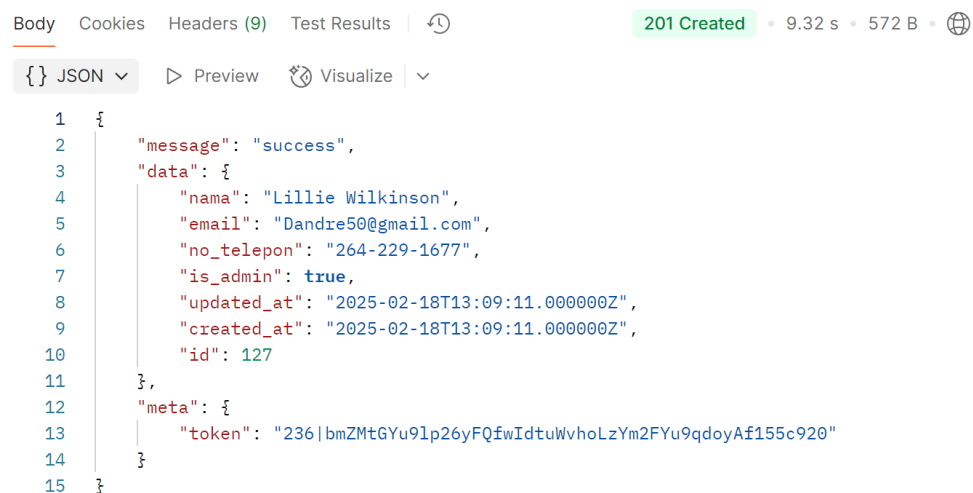
4.4.1.1. Pengujian Halaman *Register*

Tabel 4. 1 Pengujian *whitebox* halaman *register*

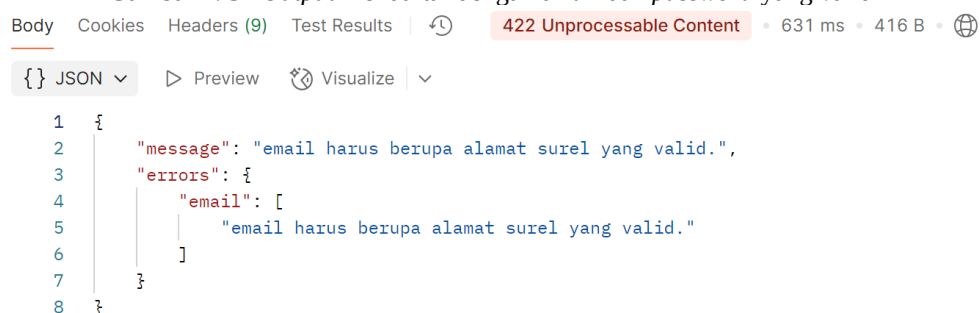
Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Mendaftar dengan <i>email</i> dan <i>password</i> yang valid	- Mengirimkan <i>request body</i> yang berisi <i>email</i> dan <i>password</i> valid menggunakan <i>method</i> POST ke <i>endpoint</i> <i>api/auth/register</i>. - Server memberikan respons HTTP 201 (<i>Created</i>) yang menunjukkan pendaftaran berhasil.	201 (<i>Created</i>)	201 (<i>Created</i>)	Berhasil
Mendaftar dengan <i>email</i> tidak valid	- Mengirimkan <i>request body</i> yang berisi <i>email</i> tidak valid menggunakan <i>method</i> POST ke <i>endpoint</i> <i>api/auth/register</i>. - Server memberikan respons HTTP 422 (<i>Unprocessable Content</i>) yang menunjukkan bahwa <i>email</i> tidak sesuai format yang benar.	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil
Mendaftar dengan <i>email</i> yang sudah terdaftar	- Mengirimkan <i>request body</i> yang berisi <i>email</i> yang sudah digunakan sebelumnya menggunakan <i>method</i> POST ke <i>endpoint</i> <i>api/auth/register</i>. - Server memberikan respons HTTP 422 (<i>Unprocessable Content</i>) yang menunjukkan bahwa <i>email</i> sudah terdaftar.	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil
Mendaftar dengan <i>password</i> kurang dari 8 karakter	- Mengirimkan <i>request body</i> yang berisi <i>password</i> dengan panjang kurang dari 8 karakter menggunakan <i>method</i> POST ke <i>endpoint</i> <i>api/auth/register</i>. - Server memberikan respons HTTP 422 (<i>Unprocessable Content</i>) yang menunjukkan bahwa <i>password</i> tidak memenuhi syarat minimum panjang karakter.	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil

Fungsi *register* bertanggung jawab dalam menangani proses pendaftaran pengguna dengan melakukan serangkaian validasi sebelum menyimpan data ke dalam sistem. Berdasarkan pengujian *whitebox* pada Tabel 4. 1, terdapat empat skenario utama yang diuji dalam proses pendaftaran.

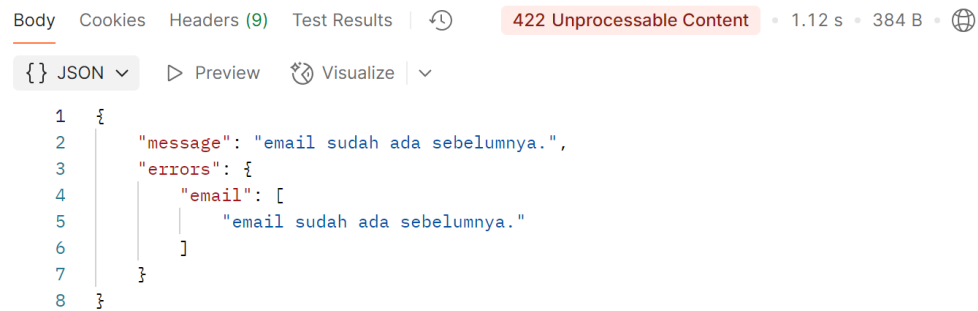
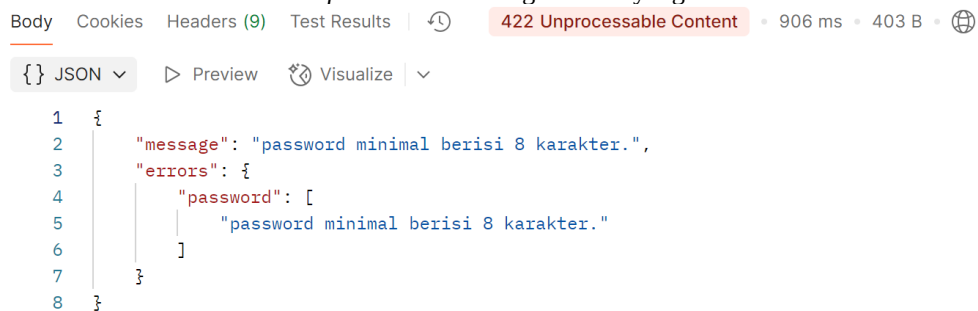
Pengujian terhadap fungsi *register* dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 34- Gambar 4. 37.



Gambar 4. 34 *Output* mendaftar dengan *email* dan *password* yang valid



Gambar 4. 35 *Output* mendaftar dengan *email* tidak valid

Gambar 4. 36 *Output* mendaftar dengan *email* yang sudah terdaftarGambar 4. 37 *Output* mendaftar dengan *password* kurang dari 8 karakter

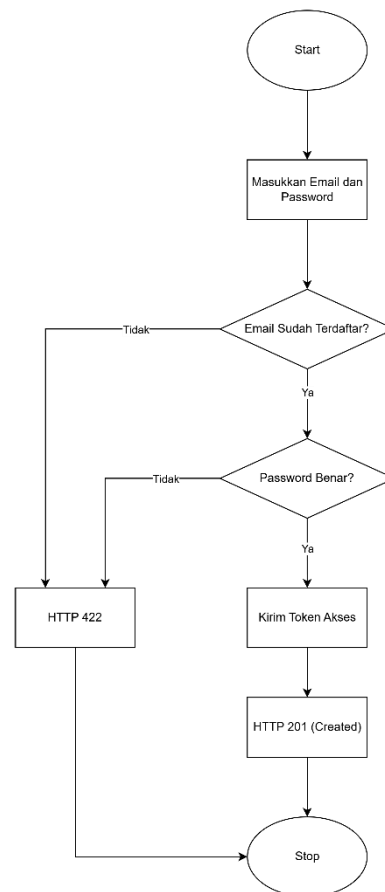
4.4.1.2. Pengujian Halaman Login

Tabel 4. 2 Pengujian *whitebox* halaman *login*

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Login dengan <i>email</i> dan <i>password</i> yang benar	- Mengirimkan <i>request body</i> dengan <i>email</i> dan <i>password</i> valid menggunakan <i>method</i> POST ke <i>endpoint</i> <code>api/auth/login</code>. - HTTP memberi respons 201 (<i>Created</i>) dengan <i>token</i> akses yang valid.	201 (<i>Created</i>)	201 (<i>Created</i>)	Berhasil
Login dengan <i>email</i> dan <i>password</i> yang tidak cocok	- Mengirimkan <i>request body</i> dengan <i>email</i> valid tetapi <i>password</i> salah menggunakan <i>method</i> POST ke <i>endpoint</i> <code>api/auth/login</code>. - HTTP memberi respons 422 (<i>Unprocessable Content</i>) karena kredensial tidak cocok.	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil
Login dengan <i>email</i> yang tidak terdaftar	- Mengirimkan <i>request body</i> dengan <i>email</i> yang belum terdaftar menggunakan <i>method</i> POST ke <i>endpoint</i> <code>api/auth/login</code>. - HTTP memberi respons 422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
	Content) karena email tidak ditemukan dalam database.			

Fungsi *login* bertanggung jawab dalam menangani proses autentikasi pengguna dengan memvalidasi email dan *password* yang dimasukkan. Proses ini memastikan bahwa hanya pengguna yang terdaftar dengan kredensial yang benar yang dapat mengakses sistem. Berdasarkan pengujian *whitebox* pada Tabel 4. 2, terdapat tiga skenario utama yang diuji dalam proses *login*.



Gambar 4. 38 Diagram *cyclomatic complexity* halaman *login*

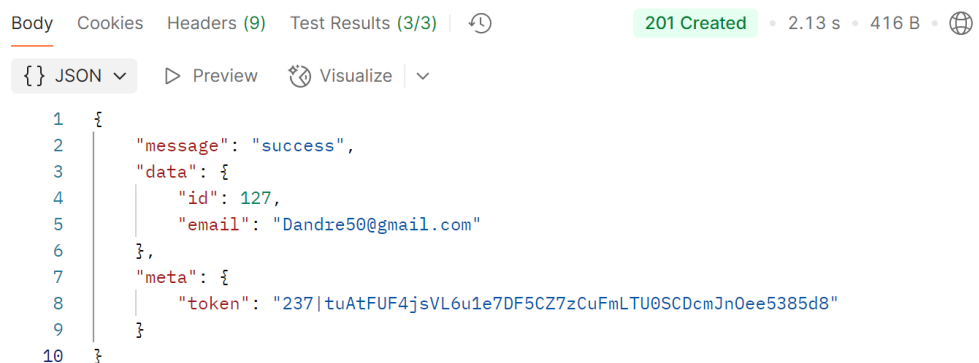
Berdasarkan diagram *cyclomatic complexity* pada Gambar 4. 38, nilai masing-masing elemen adalah:

$$\begin{aligned}
 M &= E - N + 2P \\
 &= 9 - 8 + 2(1) \\
 &= 3
 \end{aligned}$$

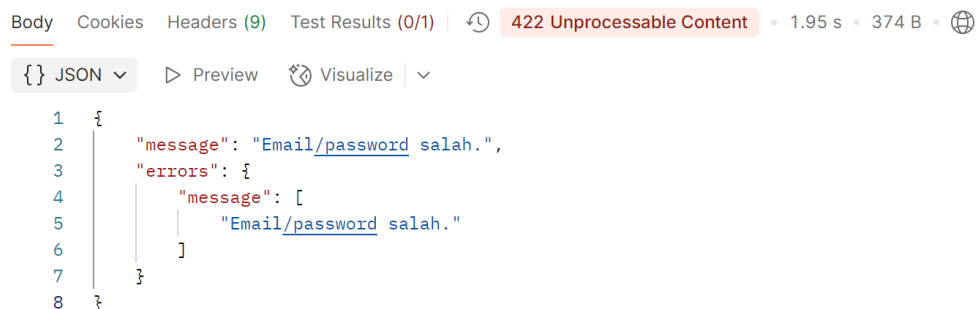
Dengan demikian, nilai *cyclomatic complexity* (CC) yang diperoleh adalah 3, yang menunjukkan bahwa terdapat tiga jalur independen dalam program. Nilai ini mengindikasikan bahwa setidaknya tiga pengujian perlu dilakukan untuk memastikan bahwa seluruh jalur kode telah diuji secara menyeluruh.

Mengacu pada klasifikasi Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*", nilai CC yang diperoleh masuk dalam kategori "Prosedur sederhana, risiko kecil" (1-10). Artinya, fungsi *login* memiliki tingkat kompleksitas rendah dan risiko minimal terhadap stabilitas kode.

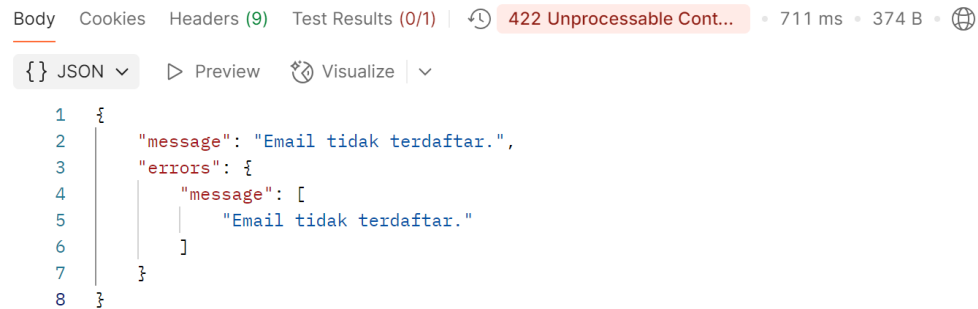
Pengujian terhadap fungsi *login* dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 39-Gambar 4. 41.



Gambar 4. 39 Output login dengan *email* dan *password* yang benar



Gambar 4. 40 Output login dengan *email* dan *password* yang tidak cocok



Gambar 4. 41 Output login dengan email yang tidak terdaftar

4.4.1.3. Pengujian Halaman Users

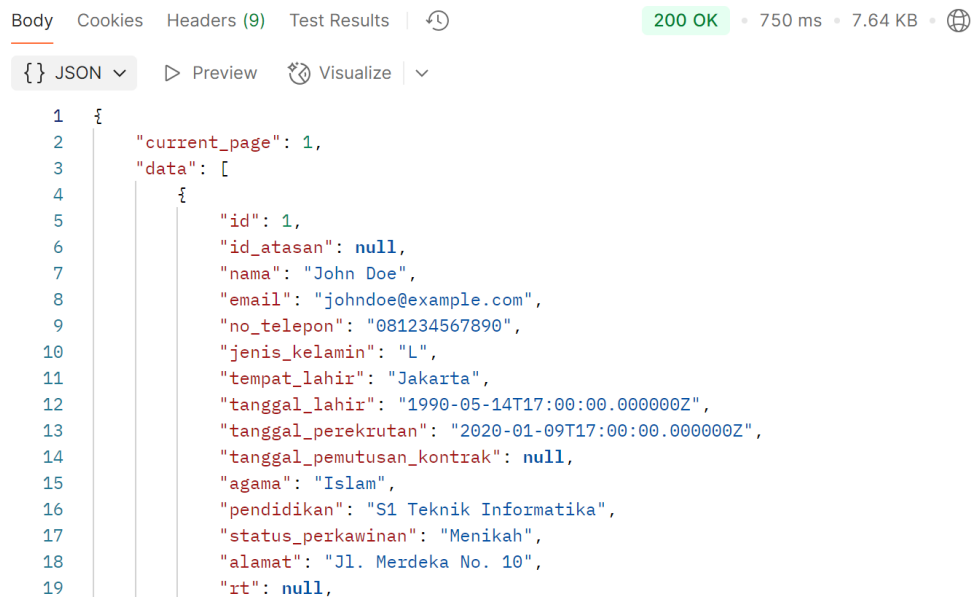
Tabel 4. 3 Pengujian whitebox halaman users

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Mendapatkan daftar semua pengguna	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>api/users</code>. - Server memberikan respons 200 (OK) dengan daftar semua pengguna dalam format JSON.	200 (OK)	200 (OK)	Berhasil
Mendapatkan data pengguna berdasarkan ID yang ada	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>api/users/{id}</code> dengan ID yang valid dan terdaftar. - Server memberikan respons 200 (OK) dengan data pengguna yang sesuai.	200 (OK)	200 (OK)	Berhasil
Mendapatkan data pengguna berdasarkan ID yang tidak ada	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>api/users/{id}</code> dengan ID yang tidak ditemukan dalam <i>database</i>. - Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i>.	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil
Meng-edit data pengguna berdasarkan ID yang ada	- Mengirimkan <i>request</i> PUT ke <i>endpoint</i> <code>api/users/{id}</code> dengan ID pengguna yang valid serta data yang diperbarui. - Server memberikan respons 200 (OK) dengan data pengguna yang telah diperbarui.	200 (OK)	200 (OK)	Berhasil
Meng-edit data pengguna berdasarkan ID yang tidak ada	Mengirimkan <i>request</i> PUT ke <i>endpoint</i> <code>api/users/{id}</code> dengan ID yang tidak	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil

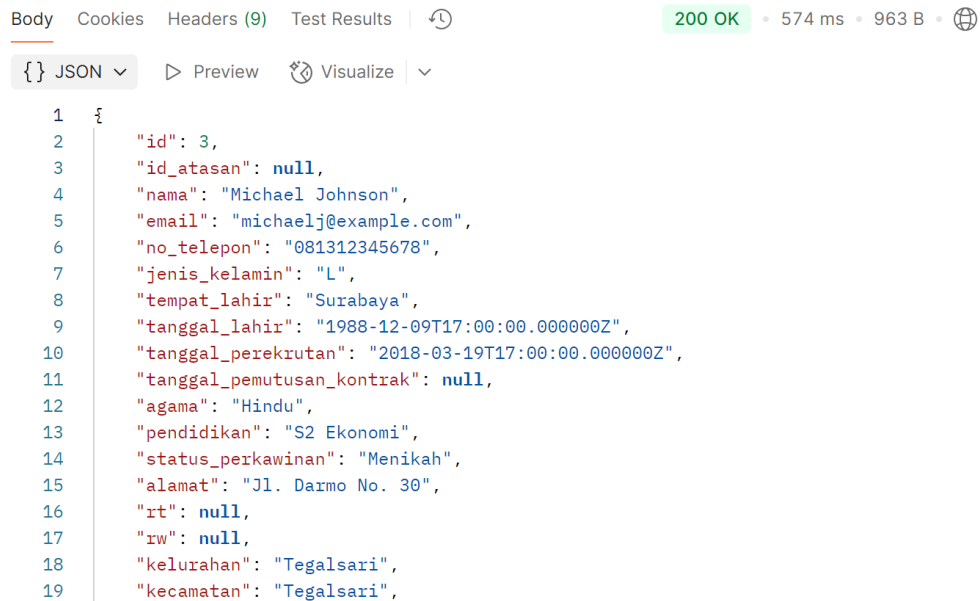
Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
ID yang tidak ada	ditemukan dalam <i>database</i> serta data yang diperbarui. 2. Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i> .			

Fungsi *User Management* bertanggung jawab untuk menangani proses pengelolaan data pengguna, termasuk melihat daftar pengguna, mendapatkan data pengguna berdasarkan ID, serta mengedit data pengguna. Berdasarkan pengujian *whitebox* pada Tabel 4. 3, terdapat lima skenario utama yang diuji dalam proses pengelolaan pengguna.

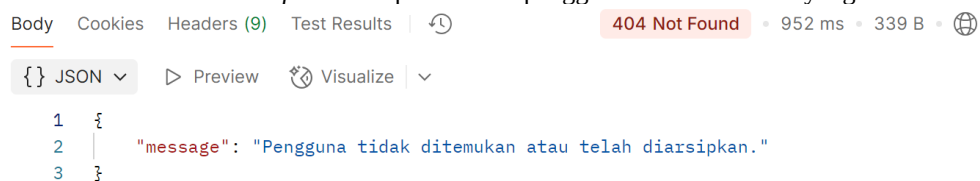
Pengujian terhadap fungsi *users* dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 42-Gambar 4. 44.



Gambar 4. 42 Output mendapatkan daftar semua pengguna



Gambar 4. 43 Output mendapatkan data pengguna berdasarkan ID yang ada



Gambar 4. 44 Output mendapatkan dan meng-edit data pengguna berdasarkan ID yang tidak ada

4.4.1.4. Pengujian Halaman Pengumuman

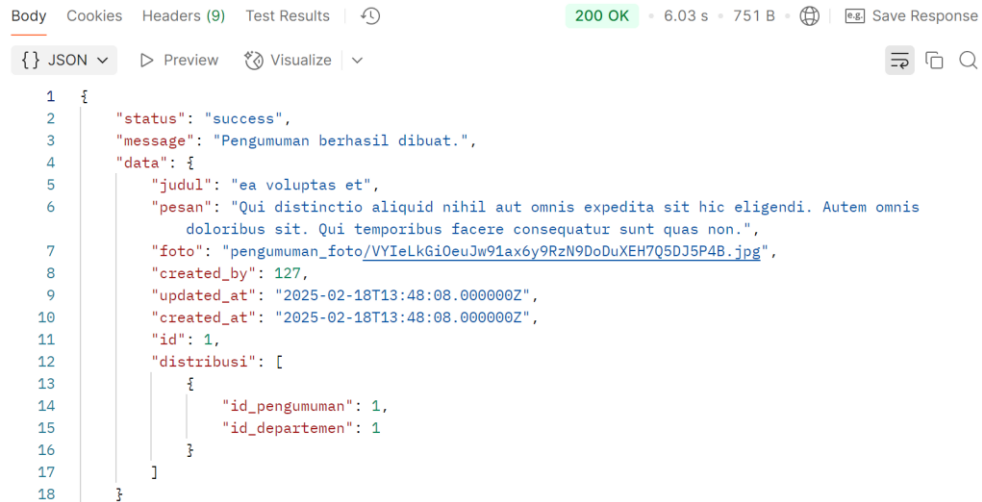
Tabel 4. 4 Pengujian *whitebox* halaman pengumuman

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Membuat pengumuman dengan semua <i>field</i> yang wajib diisi	- Mengirimkan <i>request</i> POST ke <i>endpoint</i> /api/pengumuman dengan semua <i>field</i> wajib diisi secara valid. - Server memberikan respons 200 (OK) dengan data pengumuman yang berhasil dibuat.	200 (OK)	200 (OK)	Berhasil
Membuat pengumuman dengan satu atau lebih <i>field</i> wajib kosong	- Mengirimkan <i>request</i> POST ke <i>endpoint</i> /api/pengumuman dengan satu atau lebih <i>field</i> wajib kosong. - Server memberikan respons 422 (<i>Unprocessable Content</i>) dengan pesan kesalahan validasi.	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil
Mendapatkan daftar semua	Mengirimkan <i>request</i> GET ke <i>endpoint</i> /api/pengumuman.	200 (OK)	200 (OK)	Berhasil

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
pengumuman	2. Server memberikan respons 200 (OK) dengan daftar semua pengumuman dalam format JSON.			
Mendapatkan pengumuman berdasarkan ID yang ada	1. Mengirimkan <i>request GET</i> ke <i>endpoint</i> <code>/api/pengumuman/{id}</code> dengan ID pengumuman yang valid dan terdaftar. 2. Server memberikan respons 200 (OK) dengan data pengumuman yang sesuai.	200 (OK)	200 (OK)	Berhasil
Mendapatkan pengumuman berdasarkan ID yang tidak ada	1. Mengirimkan <i>request GET</i> ke <i>endpoint</i> <code>/api/pengumuman/{id}</code> dengan ID yang tidak ditemukan dalam <i>database</i> . 2. Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i> .	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil

Fungsi pengumuman bertanggung jawab dalam menangani proses pembuatan, pengelolaan, dan pengambilan data pengumuman dalam sistem. Proses ini memastikan bahwa setiap pengumuman yang dibuat memiliki *field* yang lengkap dan memungkinkan pengguna untuk mengambil daftar pengumuman atau pengumuman spesifik berdasarkan ID. Berdasarkan pengujian *whitebox* pada Tabel 4. 4, terdapat lima skenario utama yang diuji dalam proses pengelolaan pengumuman.

Pengujian terhadap fungsi pengumuman dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 45.



Gambar 4. 45 Output membuat pengumuman dengan semua *field* yang wajib diisi

4.4.1.5. Pengujian Halaman Permohonan Cuti/Perizinan

Tabel 4. 5 Pengujian *whitebox* halaman pengajuan cuti/perizinan

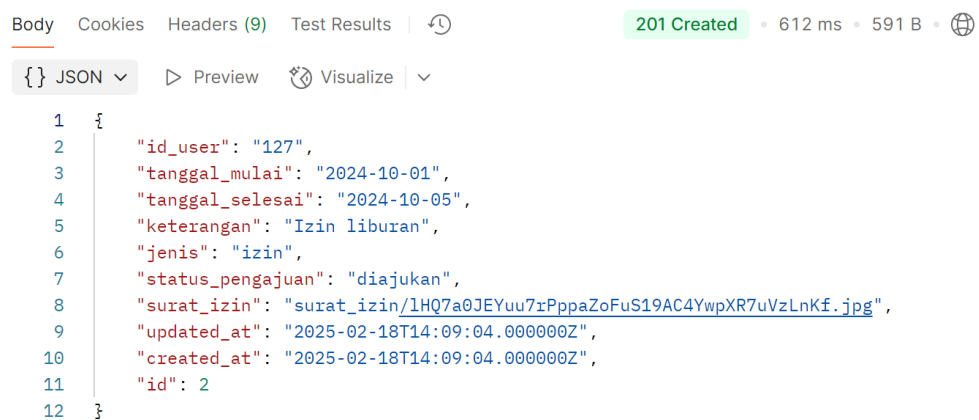
Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Membuat permohonan cuti/perizinan dengan semua <i>field</i> yang wajib diisi	- Mengirimkan <i>request</i> POST ke <i>endpoint</i> /api/cuti-perizinan dengan semua <i>field</i> wajib diisi secara valid. - Server memberikan respons 201 (<i>Created</i>) dengan data permohonan yang berhasil dibuat.	201 (<i>Created</i>)	201 (<i>Created</i>)	Berhasil
Membuat permohonan cuti/perizinan dengan satu atau lebih <i>field</i> wajib kosong	- Mengirimkan <i>request</i> POST ke <i>endpoint</i> /api/cuti-perizinan dengan satu atau lebih <i>field</i> wajib kosong. - Server memberikan respons 422 (<i>Unprocessable Content</i>) dengan pesan kesalahan validasi.	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil
Mendapatkan daftar semua permohonan cuti/perizinan	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> /api/cuti-perizinan. - Server memberikan respons 200 (OK) dengan daftar semua permohonan dalam format JSON.	200 (OK)	200 (OK)	Berhasil

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Mendapatkan permohonan cuti/perizinan berdasarkan ID yang ada	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> /api/cuti-perizinan/{id} dengan ID permohonan yang valid dan terdaftar. - Server memberikan respons 200 (OK) dengan data permohonan yang sesuai.	200 (OK)	200 (OK)	Berhasil
Mendapatkan permohonan cuti/perizinan berdasarkan ID yang tidak ada	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> /api/cuti-perizinan/{id} dengan ID yang tidak ditemukan dalam <i>database</i>. - Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i>.	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil
Meng-edit permohonan cuti/perizinan berdasarkan ID yang ada	- Mengirimkan <i>request</i> PUT ke <i>endpoint</i> /api/cuti-perizinan/{id} dengan ID permohonan yang valid serta data yang diperbarui. - Server memberikan respons 200 (OK) dengan data permohonan yang telah diperbarui.	200 (OK)	200 (OK)	Berhasil
Meng-edit permohonan cuti/perizinan berdasarkan ID yang tidak ada	- Mengirimkan <i>request</i> PUT ke <i>endpoint</i> /api/cuti-perizinan/{id} dengan ID yang tidak ditemukan dalam <i>database</i> serta data yang diperbarui. - Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i>.	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil
Meng-edit permohonan cuti/perizinan dengan satu atau lebih <i>field</i> wajib kosong	- Mengirimkan <i>request</i> PUT ke <i>endpoint</i> /api/cuti-perizinan/{id} dengan satu atau lebih <i>field</i> wajib kosong. - Server memberikan respons 422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	422 (<i>Unprocessable Content</i>)	Berhasil

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
	Content) dengan pesan kesalahan validasi.			

Fungsi Cuti/Perizinan bertanggung jawab dalam menangani proses pengajuan, pengelolaan, dan pengambilan data permohonan cuti/perizinan dalam sistem. Proses ini memastikan bahwa setiap permohonan yang dibuat memiliki *field* yang lengkap dan memungkinkan pengguna untuk melihat daftar permohonan atau permohonan spesifik berdasarkan ID, serta mengedit permohonan yang telah diajukan. Berdasarkan pengujian *whitebox* pada Tabel 4. 5, terdapat delapan skenario utama yang diuji dalam proses pengelolaan cuti/perizinan.

Pengujian terhadap fungsi cuti/perizinan dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 46.



Gambar 4. 46 Output membuat permohonan cuti/perizinan dengan semua *field* yang wajib diisi

4.4.1.6. Pengujian Halaman Kalender

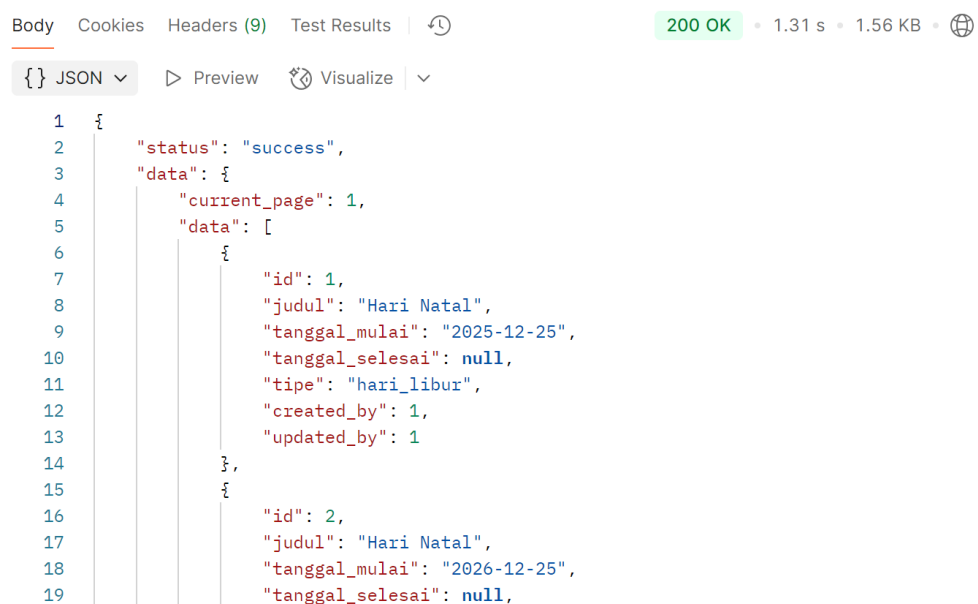
Tabel 4. 6 Pengujian *whitebox* halaman kalender

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Mendapatkan daftar semua <i>event</i> kalender	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> /api/kalender. - Server memberikan respons 200 (OK)	200 (OK)	200 (OK)	Berhasil

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
	dengan daftar semua <i>event</i> kalender dalam format JSON.			

Fungsi Kalender bertanggung jawab dalam menangani proses pengambilan daftar semua *event* kalender yang tersedia dalam sistem. Proses ini memastikan bahwa pengguna dapat melihat semua *event* yang terdaftar dengan format data yang sesuai. Berdasarkan pengujian *whitebox* pada Tabel 4. 6, terdapat satu skenario utama yang diuji dalam proses pengelolaan kalender.

Pengujian terhadap fungsi kalender dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 47.



Gambar 4. 47 Output mendapatkan daftar semua *event* kalender

4.4.1.7. Pengujian Halaman Shifts

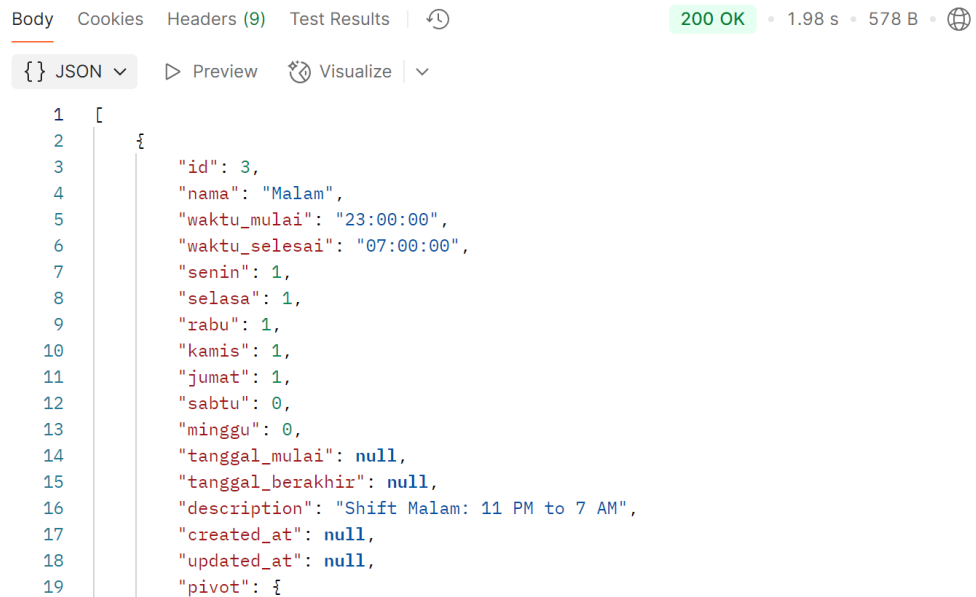
Tabel 4. 7 Pengujian *whitebox* halaman shifts

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Mendapatkan daftar <i>shift</i>	1. Mengirimkan <i>request</i> GET ke <i>endpoint</i>	200 (OK)	200 (OK)	Berhasil

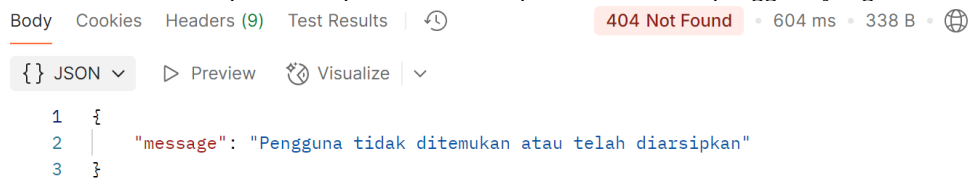
Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
berdasarkan ID pengguna yang valid	<code>/api/users/{id}/shifts</code> dengan ID pengguna yang valid dan terdaftar. 2. Server memberikan respons 200 (OK) dengan daftar shift pengguna dalam format JSON.			
Mendapatkan daftar <i>shift</i> berdasarkan ID pengguna yang tidak ada	1. Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>/api/users/{id}/shifts</code> dengan ID pengguna yang tidak ditemukan dalam <i>database</i> . 2. 2. Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i> .	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil

Fungsi *Shift* bertanggung jawab dalam menangani proses pengambilan daftar *shift* berdasarkan ID pengguna yang tersedia dalam sistem. Proses ini memastikan bahwa pengguna dapat melihat daftar *shift* yang sesuai dengan ID pengguna yang valid serta memberikan respons yang tepat jika ID tidak ditemukan. Berdasarkan pengujian *whitebox* pada Tabel 4. 7, terdapat dua skenario utama yang diuji dalam proses pengelolaan *shift*.

Pengujian terhadap fungsi *shifts* dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 48-Gambar 4. 49.



Gambar 4. 48 Output mendapatkan daftar *shift* berdasarkan ID pengguna yang valid



Gambar 4. 49 Output mendapatkan daftar *shift* berdasarkan ID pengguna yang tidak ada

4.4.1.8. Pengujian Halaman *Payroll*

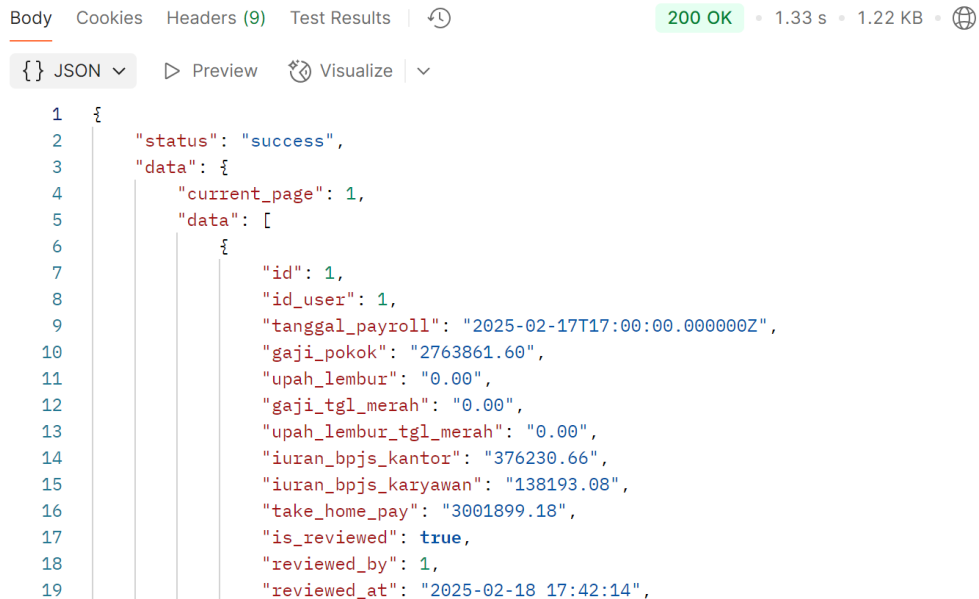
Tabel 4. 8 Pengujian *whitebox* halaman *payroll*

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
Mendapatkan daftar <i>payroll</i> berdasarkan ID pengguna yang valid	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>/api/users/{id}/payroll</code> dengan ID pengguna yang valid dan terdaftar. 2. Server memberikan respons 200 (OK) dengan daftar <i>payroll</i> pengguna dalam format JSON.	200 (OK)	200 (OK)	Berhasil
Mendapatkan daftar <i>payroll</i> berdasarkan ID pengguna yang tidak ada	- Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>/api/users/{id}/payroll</code> dengan ID pengguna yang tidak ditemukan dalam <i>database</i>. 2. Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i>.	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil
Mendapatkan <i>payroll</i> berdasarkan	Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>/api/payroll/{id}</code> dengan	200 (OK)	200 (OK)	Berhasil

Nama Fungsi	Skenario Pengujian	Ekspektasi (Kode HTTP)	Output (Kode HTTP)	Hasil
ID <i>payroll</i> yang valid	ID <i>payroll</i> yang valid dan terdaftar. 2. Server memberikan respons 200 (OK) dengan detail <i>payroll</i> dalam format JSON.			
Mendapatkan <i>payroll</i> berdasarkan ID <i>payroll</i> yang tidak ada	1. Mengirimkan <i>request</i> GET ke <i>endpoint</i> <code>/api/payroll/{id}</code> dengan ID <i>payroll</i> yang tidak ditemukan dalam <i>database</i> . 2. Server memberikan respons 404 (<i>Not Found</i>) dengan pesan <i>error</i> .	404 (<i>Not Found</i>)	404 (<i>Not Found</i>)	Berhasil

Fungsi *Payroll* bertanggung jawab dalam menangani proses pengambilan daftar *payroll* berdasarkan ID pengguna maupun ID *payroll* yang tersedia dalam sistem. Proses ini memastikan bahwa pengguna dapat melihat data *payroll* yang sesuai dengan ID pengguna atau ID *payroll* yang valid serta memberikan respons yang tepat jika ID tidak ditemukan. Berdasarkan pengujian *whitebox* pada Tabel 4. 8, terdapat empat skenario utama yang diuji dalam proses pengelolaan *payroll*.

Pengujian terhadap fungsi *payroll* dilakukan menggunakan Postman, dengan mengirimkan permintaan ke *endpoint* sesuai dengan skenario pengujian yang telah ditentukan. Hasil pengujian menunjukkan bahwa *output* yang dihasilkan telah sesuai dengan ekspektasi, sebagaimana ditampilkan pada Gambar 4. 50- Gambar 4. 53.

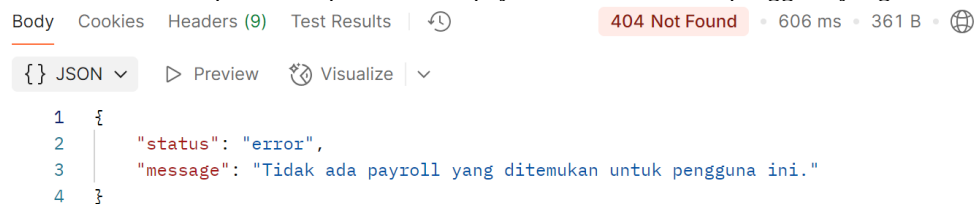


```

1  {
2    "status": "success",
3    "data": {
4      "current_page": 1,
5      "data": [
6        {
7          "id": 1,
8          "id_user": 1,
9          "tanggal_payroll": "2025-02-17T17:00:00.000000Z",
10         "gaji_pokok": "2763861.60",
11         "upah_lembur": "0.00",
12         "gaji_tgl_merah": "0.00",
13         "upah_lembur_tgl_merah": "0.00",
14         "iuran_bpjs_kantor": "376230.66",
15         "iuran_bpjs_karyawan": "138193.08",
16         "take_home_pay": "3001899.18",
17         "is_reviewed": true,
18         "reviewed_by": 1,
19         "reviewed_at": "2025-02-18 17:42:14",

```

Gambar 4. 50 Output mendapatkan daftar *payroll* berdasarkan ID pengguna yang valid

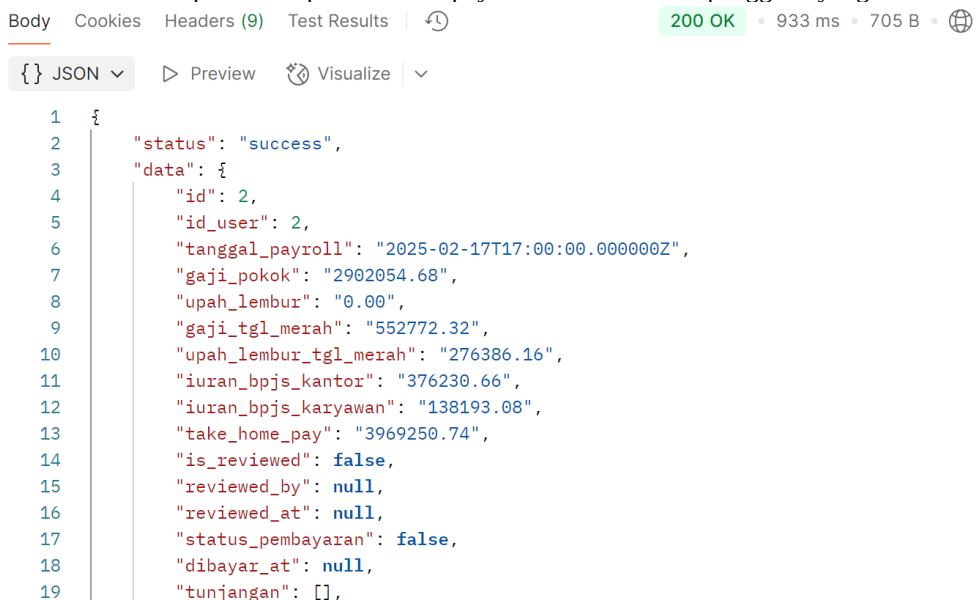


```

1  {
2    "status": "error",
3    "message": "Tidak ada payroll yang ditemukan untuk pengguna ini."
4  }

```

Gambar 4. 51 Output mendapatkan daftar *payroll* berdasarkan ID pengguna yang tidak ada

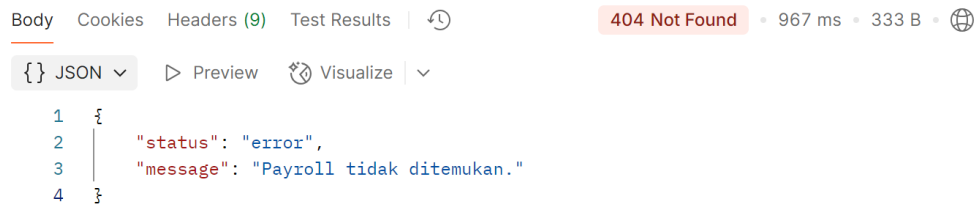


```

1  {
2    "status": "success",
3    "data": {
4      "id": 2,
5      "id_user": 2,
6      "tanggal_payroll": "2025-02-17T17:00:00.000000Z",
7      "gaji_pokok": "2902054.68",
8      "upah_lembur": "0.00",
9      "gaji_tgl_merah": "552772.32",
10     "upah_lembur_tgl_merah": "276386.16",
11     "iuran_bpjs_kantor": "376230.66",
12     "iuran_bpjs_karyawan": "138193.08",
13     "take_home_pay": "3969250.74",
14     "is_reviewed": false,
15     "reviewed_by": null,
16     "reviewed_at": null,
17     "status_pembayaran": false,
18     "dibayar_at": null,
19     "tunjangan": [],

```

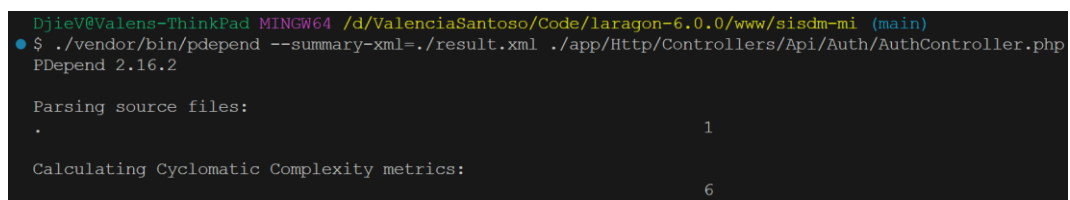
Gambar 4. 52 Output mendapatkan *payroll* berdasarkan ID *payroll* yang valid



Gambar 4. 53 Output mendapatkan *payroll* berdasarkan ID *payroll* yang tidak ada

4.5 Review Kode

4.5.1 AuthController

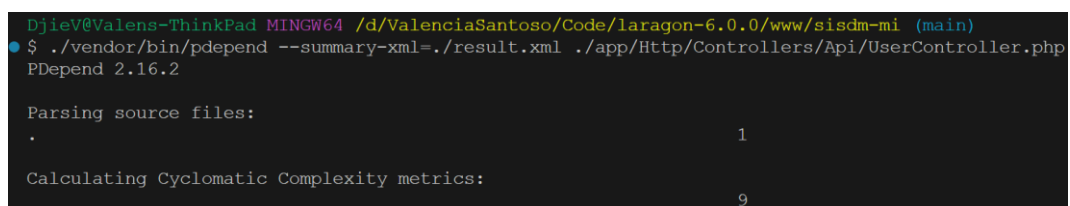


Gambar 4. 54 Hasil pengujian PDepend pada AuthController

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 54, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk AuthController adalah 6. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 6, ini menunjukkan bahwa kode pada AuthController memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini menunjukkan bahwa AuthController memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.5.2 UserController

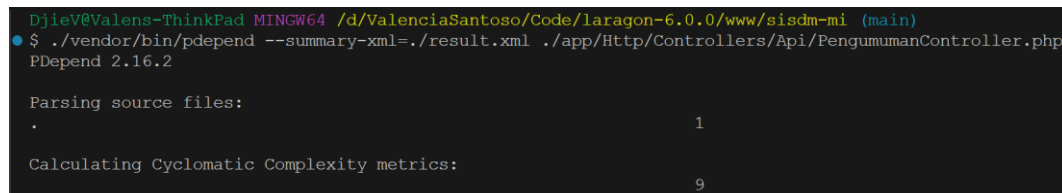


Gambar 4. 55 Hasil pengujian PDepend pada UserController

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 55, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk UserController adalah 9. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 9, ini menunjukkan bahwa kode pada UserController memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini menunjukkan bahwa UserController memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.5.3 PengumumanController



```
Djiev@Valens-ThinkPad MINGW64 /d/ValenciaSantoso/Code/laragon-6.0.0/www/sisdmi-mi (main)
$ ./vendor/bin/pdepend --summary-xml=./result.xml ./app/Http/Controllers/Api/PengumumanController.php
PDepend 2.16.2

Parsing source files:
. 1

Calculating Cyclomatic Complexity metrics:
9
```

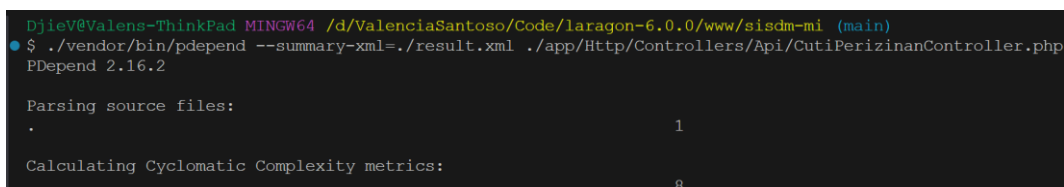
Gambar 4. 56 Hasil pengujian PDepend pada PengumumanController

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 56, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk PengumumanController adalah 9. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 9, ini menunjukkan bahwa kode pada PengumumanController memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini

menunjukkan bahwa `PengumumanController` memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.5.4 CutiPerizinanController



```
Djiev@Valens-ThinkPad MINGW64 /d/ValenciaSantoso/Code/laragon-6.0.0/www/sisdmi (main)
$ ./vendor/bin/pdepend --summary-xml=./result.xml ./app/Http/Controllers/Api/CutiPerizinanController.php
PDepend 2.16.2

Parsing source files:
. 1

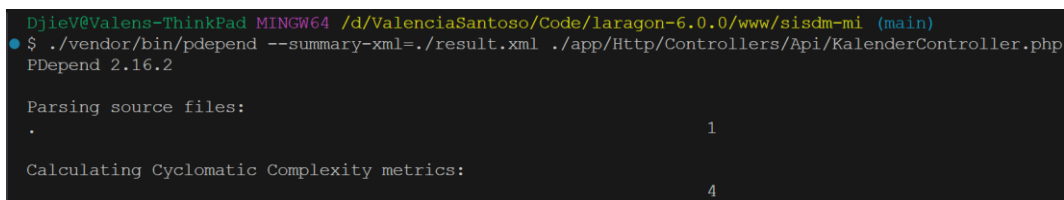
Calculating Cyclomatic Complexity metrics:
8
```

Gambar 4. 57 Hasil pengujian PDepend pada `CutiPerizinanController`

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 57, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk `CutiPerizinanController` adalah 8. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 8, ini menunjukkan bahwa kode pada `CutiPerizinanController` memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini menunjukkan bahwa `CutiPerizinanController` memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.5.5 KalenderController



```
Djiev@Valens-ThinkPad MINGW64 /d/ValenciaSantoso/Code/laragon-6.0.0/www/sisdmi (main)
$ ./vendor/bin/pdepend --summary-xml=./result.xml ./app/Http/Controllers/Api/KalenderController.php
PDepend 2.16.2

Parsing source files:
. 1

Calculating Cyclomatic Complexity metrics:
4
```

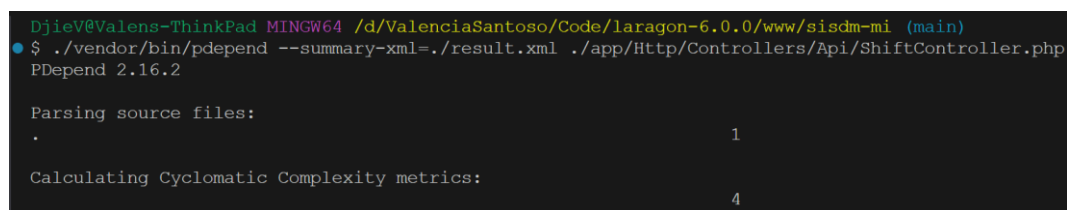
Gambar 4. 58 Hasil pengujian PDepend pada `KalenderController`

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 58, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk `KalenderController` adalah 4. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to*

Identify Risk" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 4, ini menunjukkan bahwa kode pada KalenderController memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini menunjukkan bahwa KalenderController memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.5.6 ShiftController



```
DjieV@Valens-ThinkPad MINGW64 /d/ValenciaSantoso/Code/laragon-6.0.0/www/sisdmi (main)
$ ./vendor/bin/pdepend --summary-xml=./result.xml ./app/Http/Controllers/Api/ShiftController.php
PDepend 2.16.2

Parsing source files:
. 1

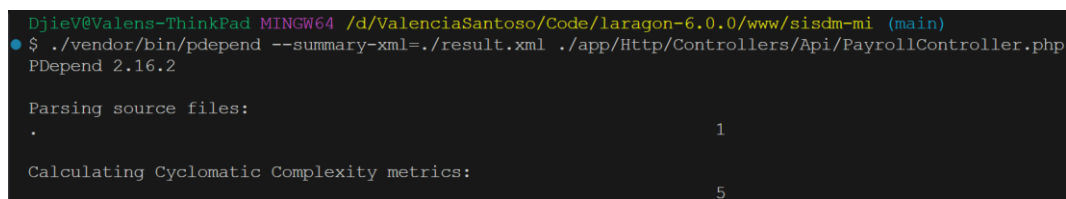
Calculating Cyclomatic Complexity metrics:
4
```

Gambar 4. 59 Hasil pengujian PDepend pada ShiftController

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 59, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk ShiftController adalah 4. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 4, ini menunjukkan bahwa kode pada ShiftController memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini menunjukkan bahwa ShiftController memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.5.7 PayrollController



```
Djiev@Valens-ThinkPad MINGW64 /d/ValenciaSantoso/Code/laragon-6.0.0/www/sisdmi (main)
$ ./vendor/bin/pdepend --summary-xml=./result.xml ./app/Http/Controllers/Api/PayrollController.php
PDepend 2.16.2

Parsing source files:
. 1

Calculating Cyclomatic Complexity metrics:
5
```

Gambar 4. 60 Hasil pengujian PDepend pada PayrollController

Berdasarkan hasil pengujian menggunakan PDepend pada Gambar 4. 60, nilai *Cyclomatic Complexity* (CC) yang diperoleh untuk PayrollController adalah 5. Menurut Tom McCabe dalam presentasinya "*Software Quality Metrics to Identify Risk*" untuk *Department of Homeland Security* ^[25] dalam rentang 1-10 mengindikasikan prosedur sederhana dengan risiko kecil.

Dengan nilai 5, ini menunjukkan bahwa kode pada PayrollController memiliki kompleksitas yang relatif rendah, sehingga risiko terkait kesalahan logika atau *bug* dalam kode juga tergolong kecil. Meskipun terdapat beberapa keputusan cabang dan alur logika, kompleksitasnya masih dalam batas yang wajar, sehingga pengelolaan kode ini cenderung mudah dilakukan dan tidak menambah kesulitan dalam perawatan atau pengujian lebih lanjut. Secara keseluruhan, hasil CC ini menunjukkan bahwa PayrollController memiliki tingkat stabilitas dan keandalan yang baik dalam hal desain kode.

4.6 Hasil Implementasi dan Pengujian

Secara keseluruhan, setelah tahap implementasi *backend* menggunakan *framework* Laravel, berbagai pengujian dilakukan untuk mengevaluasi efisiensi dan kinerja sistem. Pengujian *whitebox* berhasil memastikan bahwa alur logika dan struktur kode berjalan dengan baik sesuai dengan desain sistem yang diinginkan. Optimasi yang diterapkan, seperti penggunaan *eager loading*, *pagination*, dan *indexing* pada *database*, terbukti memberikan dampak signifikan terhadap performa sistem. Hasil pengujian menunjukkan bahwa waktu respons API *backend* berhasil menurun dari 29,915 ms menjadi 2,840 ms, yang telah memenuhi standar performa yang diharapkan.

Selain pengujian fungsional, dilakukan juga analisis *Cyclomatic Complexity* (CC) untuk menilai tingkat kompleksitas kode dan potensi risiko kesalahan logika dalam implementasi kode. Hasil pengujian menunjukkan bahwa kompleksitas kode pada berbagai *controller* (seperti *AuthController*, *UserController*, dan lainnya) berada pada level yang dapat dikelola dengan baik. Nilai CC antara 4 hingga 9 menunjukkan bahwa kode memiliki kompleksitas rendah hingga sedang, yang mengurangi risiko kesalahan logika dan meningkatkan efisiensi dalam pemeliharaan dan pengujian kode.

Dengan demikian, hasil implementasi, pengujian, dan *review* kode yang dilakukan pada sistem *backend* di CV Mebel Internasional Semarang menunjukkan bahwa optimasi yang diterapkan telah berhasil meningkatkan efisiensi, stabilitas, dan keandalan sistem. Sistem kini dapat mendukung operasional perusahaan dengan lebih baik, memenuhi kebutuhan pengelolaan sumber daya manusia secara lebih efisien dan tepat waktu.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian dan implementasi yang telah dilakukan, pengembangan *backend* SISDM berbasis Laravel dengan *database* MySQL telah berhasil memenuhi tujuan penelitian yang telah ditetapkan. Kesimpulan penelitian ini adalah sebagai berikut:

1. Sistem *backend* berbasis Laravel berhasil dikembangkan untuk mengotomatisasi pengelolaan data karyawan, termasuk pencatatan presensi, penjadwalan kerja, serta pengelolaan izin dan cuti. Implementasi ini telah terbukti meningkatkan efisiensi operasional perusahaan dengan mengurangi pekerjaan manual yang sebelumnya dilakukan oleh HRD.
2. Fitur perhitungan gaji dan lembur otomatis yang diterapkan dalam *backend* Laravel telah berhasil meningkatkan akurasi dalam pengolahan data keuangan karyawan. Dengan adanya fitur ini, risiko kesalahan perhitungan yang sebelumnya sering terjadi akibat proses manual dapat diminimalisir, sehingga meningkatkan transparansi penggajian dan kepuasan kerja karyawan di CV Mebel Internasional Semarang.
3. Integrasi *backend* Laravel dengan aplikasi Android melalui RESTful API telah berjalan dengan baik, memungkinkan karyawan mengakses informasi seperti jadwal kerja, riwayat presensi, slip gaji, serta pengajuan izin dan cuti secara digital. Hal ini memberikan kemudahan bagi karyawan dalam memperoleh informasi SDM secara *real-time* serta mengurangi ketergantungan terhadap komunikasi manual melalui HRD.
4. Pengujian *whitebox* telah dilakukan untuk memastikan kualitas dan keandalan sistem. Hasil pengujian menunjukkan bahwa alur logika sistem telah dikembangkan dengan baik, dengan minim kesalahan dalam struktur kode dan logika program. Analisis terhadap *cyclomatic complexity* memastikan bahwa kode memiliki tingkat kompleksitas yang dapat dikelola dengan baik, mengurangi potensi *bug* dan meningkatkan stabilitas sistem.

5. Pengujian *response time* melalui *load testing* menunjukkan adanya kebutuhan optimasi performa *backend*. Hasil pengujian awal menunjukkan waktu respons yang tinggi, dengan rata-rata 29.915 ms. Setelah dilakukan optimasi dengan *pagination*, *indexing*, dan *eager loading*, rata-rata waktu respons berhasil diturunkan menjadi 2.840 ms, sehingga memenuhi standar non-fungsional dengan waktu respons di bawah 5 detik. Dengan hasil ini, sistem dapat menangani beban kerja lebih besar tanpa mengalami degradasi performa yang signifikan.

Dengan demikian, proyek ini telah berhasil mencapai tujuan penelitian yang ditetapkan, yaitu mengembangkan sistem *backend* yang efisien, aman, dan terintegrasi untuk mendukung pengelolaan sumber daya manusia secara menyeluruh di CV Mebel Internasional Semarang.

5.2 Saran

Dari hasil pengembangan *backend* Laravel dan *database* MySQL yang telah dilaksanakan dalam proyek ini, terdapat beberapa rekomendasi yang dapat dijadikan acuan dalam pengembangan lebih lanjut:

1. Saat ini, *backend* telah terintegrasi dengan aplikasi Android, namun belum memiliki aksesibilitas yang luas di berbagai platform. Untuk meningkatkan fleksibilitas pengguna, disarankan untuk mengembangkan antarmuka berbasis web agar sistem dapat diakses dari perangkat desktop dan *mobile* secara lebih luas.
2. Untuk meningkatkan keamanan akses pengguna, sistem sebaiknya menerapkan 2FA berbasis OTP (*One-Time Password*) yang dikirimkan melalui email atau SMS. Dengan fitur ini, risiko akses tidak sah ke dalam sistem dapat dikurangi secara signifikan.
3. Meskipun optimasi telah dilakukan melalui *pagination*, *indexing*, dan *eager loading*, *backend* dapat lebih dioptimalkan dengan menerapkan *caching* menggunakan Redis atau Memcached. Teknik *caching* ini dapat mengurangi beban *database* dan meningkatkan kecepatan respons saat menangani permintaan yang sering diakses.

4. Agar proses penggajian lebih efisien, disarankan untuk mengintegrasikan *backend* dengan sistem perbankan atau *payment gateway* untuk memungkinkan transfer gaji secara otomatis ke rekening karyawan. Dengan demikian, proses penggajian menjadi lebih cepat, akurat, dan minim risiko kesalahan manual.
5. Untuk memastikan sistem dapat menangani peningkatan jumlah pengguna di masa depan, disarankan untuk melakukan *stress testing* berkala guna mengidentifikasi batasan kapasitas *backend* dan menentukan kapan perlu dilakukan *scale-up* atau *scale-out* terhadap infrastruktur sistem..

Dengan mempertimbangkan saran-saran ini, sistem *backend* SISDM CV Mebel Internasional Semarang diharapkan mampu memberikan layanan pengelolaan data karyawan yang semakin efisien, aman, dan fleksibel di masa mendatang.

DAFTAR PUSTAKA

- [1] E. Erwin *et al.*, *Sistem Informasi Manajemen: Teori, Prinsip dan Penerapan*. PT. Sonpedia Publishing Indonesia, 2024.
- [2] Carisna Terlia and Arizona Firdonsyah, "APPLICATION OF THE LARAVEL FRAMEWORK IN THE DEVELOPMENT OF A WEB-BASED INFORMATION SYSTEM FOR BIOPHYSIO PHYSIOTHERAPY CLINIC," *Antivirus: Jurnal Ilmiah Teknik Informatika*, vol. 18, no. 2, pp. 222–233, 2024, doi: <https://doi.org/10.35457/antivirus.v18i2.3953>.
- [3] R. Aryanti, E. Fitriani, D. Ardiansyah, and A. Saepudin, "Penerapan Metode Rapid Application Development Dalam Pengembangan Sistem Informasi Akademik Berbasis Web," *Paradigma - Jurnal Komputer dan Informatika*, vol. 23, no. 2, Oct. 2021, doi: <https://doi.org/10.31294/p.v23i2.11170>.
- [4] Y. Dewi, Liliana, N. Hikmah, and M. Harjono, "PENGEMBANGAN SISTEM PEMANTAUAN SENTIMEN BERITA BERBAHASA INDONESIA BERDASARKAN KONTEN DENGAN LONG SHORT-TERM MEMORY THE DEVELOPMENT OF CONTENT-BASED INDONESIAN NEWS SENTIMENT MONITORING SYSTEM USING LONG SHORT-TERM MEMORY," *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, vol. 8, no. 5, 2021, doi: <https://doi.org/10.25126/jtiik.202184624>.
- [5] A. E. Putra dan Ardiansyah, "PENGEMBANGAN SISTEM WEB BACKEND UNTUK ADMINISTRASI DAN ANALYTIC PADA APLIKASI POS MIKRO," *JSTIE (Jurnal Sarjana Teknik Informatika) (E-Journal)*, vol. 5, no. 1, pp. 60–69, Feb. 2017, doi: <https://doi.org/10.12928/jstie.v5i1.10814>.
- [6] A. E. Rosa, "Pengembangan Sistem Informasi Manajemen Magang Berbasis Website dengan Framework Laravel dan VueJS di Kementerian Agama Kota Surabaya," *upnjatim.ac.id*, Feb. 2025, doi: <https://repository.upnjatim.ac.id/34571/1/21082010118.-cover.pdf>.
- [7] M. A. Hakim, D. Triesia, and U. Teisnajaya, "Sistem Informasi Gaji Karyawan Menggunakan Framework Codeigniter Pada Yayasan Pendidikan Islam Al Waziriyah," *Jurnal Komputer, Informasi dan Teknologi*, vol. 4, no. 2, Dec. 2024, doi: <https://doi.org/10.53697/jkomitek.v4i2.2113>.
- [8] A. Akbar and Ari, "RANCANG BANGUN SISTEM INFORMASI PENGUKURAN INDEKS PROFESIONALITAS APARATUR SIPIL NEGARA DENGAN DJANGO," *Jurnal Informatika Teknologi dan Sains (Jinteks)*, vol. 7, no. 1, pp. 127–136, Feb. 2025, doi: <https://doi.org/10.51401/jinteks.v7i1.5285>.
- [9] G. B. Sulisty, "Perancangan Sistem Informasi Perekrutan Karyawan Pada PT Yogya Indah Sejahtera Yogyakarta," *Repository Universitas Bina Sarana Informatika (RUBSI)*, Jan. 2020.
- [10] H. A. Prayoga, "Pemanfaatan Face Recognition Facenet Dalam Pembangunan Sistem Informasi Human Resource Pada PT. Comtelindo -

- Repository ITK,” *Itk.ac.id*, Apr. 2025, doi: http://repository.itk.ac.id/22040/1/11211044_cover.pdf.
- [11] “TUGAS AKHIR RANCANG BANGUN SISTEM INFORMASI EVENT DAN PAYMENT GATEAWAY BERBASIS WEB DENGAN FRAMEWORK LARAVEL (STUDI KASUS AKIBA MATSURI) EKO SAPARNURIYAN PUTRA NIM : 235410080 PROGRAM STUDI INFORMATIKA PROGRAM SARJANA FAKULTAS TEKNOLOGI INFORMASI UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA YOGYAKARTA,” 2025. Accessed: Mar. 07, 2025. [Online]. Available: https://eprints.utdi.ac.id/10627/1/1_235410080_HALAMAN_DEPAN.pdf
- [12] H. Mahwahulhusna, “Implementasi Sistem Informasi SDM Untuk Manajemen Cuti Menggunakan OrangeHRM pada PT Atalla Tiyasa Abhipraya. - Repository STT Terpadu Nurul Fikri,” *Nurulfikri.ac.id*, Aug. 2023, doi: <https://repository.nurulfikri.ac.id/id/eprint/287/1/2022-Hizianatul%20Mahwahulhusna-Sistem%20Informasi-Fulltext%20-%20mahwahul%20husna.pdf>.
- [13] R. Sastra, Muhammad Rizki Akbar, and Dicky Hariyanto, “Rancang Bangun Perangkat Lunak System Pengelolaan Data Penggajian Berbasis Framework Codeigniter”, *INSANtek*, vol. 5, no. 2, pp. 49-55, Nov. 2024.
- [14] N. S. Wiguna and N. Nanang, “PERANCANGAN APLIKASI PENGGAJIAN KARYAWAN BERBASIS ANDROID DAN DART FLUTTER PADA PT ANDALAN K3 MENGGUNAKAN MODEL WATERFALL”, *bharasumba*, vol. 3, no. 04, pp. 131–154, Oct. 2024, doi: <https://doi.org/10.62668/bharasumba.v3i04.1268>.
- [15] S. M. Al Zikri, “PERANCANGAN SISTEM PENGELOLAAN DATA PENERIMA DANA ZAKAT, INFAQ DAN SEDEKAH MENGGUNAKAN FRAMEWORK LARAVEL,” *Jurnal Informatika dan Rekayasa Perangkat Lunak*, vol. 2, no. 3, pp. 344–352, Oct. 2021, doi: <https://doi.org/10.33365/jatika.v2i3.1234>.
- [16] Z. Li, C. Shang, J. Wu, and Y. Li, “Microservice extraction based on knowledge graph from monolithic applications,” *Information and Software Technology*, vol. 150, p. 106992, Oct. 2022, doi: <https://doi.org/10.1016/j.infsof.2022.106992>.
- [17] P. P. Panji, “Rancang Bangun Sistem Informasi Manajemen Data Pelayanan Jemaat Gereja Berbasis Website,” *Computatio Journal of Computer Science and Information Systems*, vol. 7, no. 2, pp. 167–178, Dec. 2023, doi: <https://doi.org/10.24912/computatio.v7i2.26383>.
- [18] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, “Online testing of RESTful APIs: promises and challenges,” *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Nov. 2022, doi: <https://doi.org/10.1145/3540250.3549144>.
- [19] Duarte Felício, José Simão, and Nuno Datia, “RapiTest: Continuous Black-Box Testing of RESTful Web APIs,” *Procedia Computer Science*, vol. 219, pp. 537–545, Jan. 2023, doi: <https://doi.org/10.1016/j.procs.2023.01.322>.
- [20] S. Nidhra and J. Dondeti, “Black Box and White Box Testing Techniques -

- A Literature Review,” *International Journal of Embedded Systems and Applications*, vol. 2, no. 2, pp. 29–50, Jun. 2012, doi: <https://doi.org/10.5121/ijesa.2012.2204>.
- [21] A. Chandrasekhar, S. Anjana, and Chandran, “COMPARATIVE ANALYSIS OF LOAD TESTING TOOLS,” vol. 9, no. 6, pp. 2320–2882, 2021, Accessed: Apr. 11, 2024. [Online]. Available: <https://ijcrt.org/papers/IJCRT2106814.pdf>
 - [22] L. Setiyani, dan E. Tjandra, "Analisis Kebutuhan Fungsional Aplikasi Penanganan Keluhan Mahasiswa Studi Kasus: STMIK Rosma Karawang," *Jurnal Inovasi Pendidikan dan Teknologi Informasi*, vol. 2, no. 1, p. 11, 2021, DOI: <https://doi.org/10.52060/pti.v2i01.465>
 - [23] K. 'Afiifah, Z. F. Azzahra, dan A. D. Anggoro, "Analisis Teknik Entity-Relationship Diagram dalam Perancangan Database: Sebuah Literature Review", *Jurnal Intech*, vol. 3, no. 1, pp. 9-11, 2022, DOI: <http://dx.doi.org/10.54895/intech.v3i2.1682>
 - [24] J. B. L. Sie, I. A. Musdar, dan S. Bahri, "Pengujian White Box Testing Terhadap Website Room Menggunakan Teknik Basis Path," *KHARISMA Tech*, vol. 2, no. 1, hlm. 15-25, 2022.
 - [25] T. McCabe, “Software Quality Metrics to Identify Risk,” McCabe Software, [Daring]. Tersedia: <http://www.mccabe.com/ppt/SoftwareQualityMetricsToIdentifyRisk.ppt>. [Diakses: 3 Mar. 2025].

LAMPIRAN 1
BIODATA MAHASISWA



Nama Mahasiswa : Djie Valencia Santoso
NIM : 21120121130055
Konsentrasi : Perangkat Lunak
Tempat/Tgl. Lahir : Semarang/24 Juni 2003
Alamat Sekarang : Jl. Kenanga No. 12, Tegalrejo,
Argomulyo, Salatiga, Jawa Tengah
No. Telepon/HP : 08975937175
Alamat e-mail : djievalenciasantoso@gmail.com
Nama orang tua : Tony Santoso
Alamat orang tua : Jl. Tirto Mukti Timur IV No. 22,
Tlogosari Kulon, Pedurungan,
Semarang, Jawa Tengah

Semarang, 10 Maret 2025

Djie Valencia Santoso

LAMPIRAN 2

SOURCE CODE

models/User.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'id_jabatan',
        'id_atasan',
        'nama',
        'nik',
        'email',
        'npwp',
        'password',
        'no_telepon',
        'jenis_kelamin',
        'tempat_lahir',
        'tanggal_lahir',
        'tanggal_perekrutan',
        'agama',
        'alamat',
        'rt',
        'rw',
        'kelurahan',
        'kecamatan',
        'kabupaten_kota',
        'foto_profil',
        'foto_ktp',
        'foto_bpjs_kesehatan',
        'foto_bpjs_ketenagakerjaan',
        'is_aktif',
        'is_admin',
        'is_archived',
    ];

    /**
     * The attributes that should be hidden for serialization.
     */
}
```

```

*
* @var array<int, string>
*/
protected $hidden = [
    'password',
    'remember_token',
    'nik',
    'npwp',
    'foto_ktp',
    'foto_bpjs_kesehatan',
    'foto_bpjs_ketenagakerjaan',
];

/**
 * The attributes that should be cast.
 */
* @var array<string, string>
*/
protected $casts = [
    'email_verified_at' => 'datetime',
    'password' => 'hashed',
    'tanggal_lahir' => 'date',
    'tanggal_perekrutan' => 'date',
    'tanggal_pemutusan_kontrak' => 'date',
];

public function riwayatJabatan()
{
    return $this->hasMany(RiwayatJabatan::class, 'id_user');
}

public function atasan()
{
    return $this->belongsTo(self::class, 'id_atasan');
}

public function bawahan()
{
    return $this->hasMany(self::class, 'id_atasan');
}

public function attendances()
{
    return $this->hasMany(Attendance::class, 'id_user');
}

public function shifts()
{
    return $this->belongsToMany(Shift::class,
'penjadwalan_shift', 'id_user', 'id_shift');
}

public function payroll()
{
    return $this->hasMany(Payroll::class);
}

```

```
}
```

models/Attendance.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Carbon\Carbon;

class Attendance extends Model
{
    use HasFactory;

    protected $table = 'attendances';

    protected $fillable = [
        'id_user',
        'tanggal',
        'status',
        'hari_kerja',
        'jumlah_jam lembur',
        'is_tanggal_merah',
    ];

    protected $casts = [
        'tanggal' => 'date',
        'status' => 'boolean',
        'hari_kerja' => 'decimal:2',
        'jumlah_jam lembur' => 'decimal:2',
        'is_tanggal_merah' => 'boolean',
    ];

    public function detail()
    {
        return $this->hasMany(AttendanceDetail::class, 'id_attendance');
    }

    public function user()
    {
        return $this->belongsTo(User::class, 'id_user');
    }

    public static function countAttendance(bool $status): int
    {
        return self::where('status', $status)->count();
    }
}
```

models/AttendanceDetail.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Carbon\Carbon;

class AttendanceDetail extends Model
{
    use HasFactory;

    protected $table = 'attendance_details';

    protected $fillable = [
        'id_attendance',
        'long',
        'lat',
        'address',
        'photo',
        'type',
    ];

    public function attendance()
    {
        return $this->belongsTo(Attendance::class,
'id_attendance');
    }
}

```

models/CutiPerizinan.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class CutiPerizinan extends Model
{
    use HasFactory;

    protected $table = 'cuti_perizinan';

    protected $fillable = [
        'id_user',
        'tanggal_mulai',
        'tanggal_selesai',
        'keterangan',
        'jenis',
        'status_pengajuan',
        'disetujui_oleh',
        'surat_izin',
    ];
}

```



```

];

public function user()
{
    return $this->belongsTo(User::class, 'id_user');
}

public function disetujuiOleh()
{
    return $this->belongsTo(User::class, 'disetujui_oleh');
}
}

```

models/Departemen.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Departemen extends Model
{
    use HasFactory;

    protected $table = 'departemen';

    protected $fillable = [
        'id_kantor',
        'nama',
    ];

    public function kantor()
    {
        return $this->belongsTo(Kantor::class, 'id_kantor');
    }
}

```

models/DistribusiPengumuman.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class DistribusiPengumuman extends Model
{
    use HasFactory;
}

```

```

        protected $table = 'distribusi_pengumuman';

        protected $fillable = [
            'id_pengumuman',
            'id_departemen',
        ];

        public function pengumuman()
        {
            return $this->belongsTo(Pengumuman::class,
'id_pengumuman');
        }

        public function departemen()
        {
            return $this->belongsTo(Departemen::class,
'id_departemen');
        }
    }

```

models/Grup.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Grup extends Model
{
    use HasFactory;

    protected $table = 'grup';

    protected $fillable = [
        'id_departemen',
        'nama',
    ];

    public function departemen()
    {
        return $this->belongsTo(Departemen::class,
'id_departemen');
    }
}

```

models/Jabatan.php

```

<?php

namespace App\Models;

```

```

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Jabatan extends Model
{
    use HasFactory;

    protected $table = 'jabatan';

    protected $fillable = [
        'id_grup',
        'nama',
        'description',
    ];

    public function grup()
    {
        return $this->belongsTo(Grup::class, 'id_grup');
    }
}

```

models/Kalender.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Kalender extends Model
{
    use HasFactory;

    protected $table = 'kalender';

    protected $fillable = [
        'judul',
        'tanggal_mulai',
        'tanggal_selesai',
        'tipe',
        'repeat_type',
        'repeat_until',
        'created_by',
        'updated_by'
    ];

    public function createdBy()
    {
        return $this->belongsTo(User::class, 'created_by');
    }
}

```

```
        public function updatedBy()
        {
            return $this->belongsTo(User::class, 'updated_by');
        }
    }
}
```

models/Kantor.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Kantor extends Model
{
    use HasFactory;

    protected $table = 'kantor';

    protected $fillable = ['nama', 'alamat', 'koordinat_x',
'koordinat_y', 'radius', 'id_manager'];

    public function manager()
    {
        return $this->belongsTo(User::class, 'id_manager');
    }
}
```

models/Payroll.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Payroll extends Model
{
    use HasFactory;

    protected $table = 'payroll';

    protected $fillable = [
```

```

        'id_user',
        'tanggal_payroll',
        'gaji_pokok',
        'upah lembur',
        'gaji_tgl_merah',
        'upah lembur_tgl_merah',
        'iuran_bpjs_kantor',
        'iuran_bpjs_karyawan',
        'take_home_pay',
        'is_reviewed',
        'reviewed_by',
        'reviewed_at',
        'status_pembayaran',
        'dibayar_at',
    ];

    protected $casts = [
        'tanggal_payroll' => 'date',
        'is_reviewed' => 'boolean',
        'status_pembayaran' => 'boolean',
    ];

    public function user()
    {
        return $this->belongsTo(User::class, 'id_user');
    }

    public function reviewer()
    {
        return $this->belongsTo(User::class, 'reviewed_by');
    }

    protected function statusText(): Attribute
    {
        return Attribute::make(
            get: fn (bool $value) => $value ? 'Paid' : 'Pending'
        );
    }

    public function tunjangan()
    {
        return $this->hasMany(Tunjangan::class, 'id_payroll');
    }

    public function potongan()
    {
        return $this->hasMany(Potongan::class, 'id_payroll');
    }
}

```

models/Pengumuman.php

```
<?php
```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Pengumuman extends Model
{
    use HasFactory;

    protected $table = 'pengumuman';

    protected $fillable = [
        'judul',
        'pesan',
        'foto',
        'created_by',
        'updated_by',
    ];

    public function distribusi()
    {
        return $this->hasMany(DistribusiPengumuman::class, 'id_pengumuman');
    }

    public function creator()
    {
        return $this->belongsTo(User::class, 'created_by');
    }

    public function updater()
    {
        return $this->belongsTo(User::class, 'updated_by');
    }
}

```

models/PenjadwalanShift.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class PenjadwalanShift extends Model
{
    use HasFactory;

    protected $table = 'penjadwalan_shift';

    protected $fillable = [
        'id user',

```

```

        'id_shift',
        'is_ditampilkan',
    ];

    protected $casts = [
        'is_ditampilkan' => 'boolean',
    ];

    public function user()
    {
        return $this->belongsTo(User::class, 'id_user');
    }

    public function shift()
    {
        return $this->belongsTo(Shift::class, 'id_shift');
    }
}

```

models/Potongan.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Potongan extends Model
{
    use HasFactory;

    protected $table = 'potongan';

    protected $fillable = [
        'id_payroll',
        'nama',
        'nominal',
        'status'
    ];

    protected $casts = [
        'status' => 'boolean',
        'nominal' => 'decimal:2',
    ];

    public function payroll()
    {
        return $this->belongsTo(Payroll::class, 'id_payroll');
    }
}

```

models/RiwayatJabatan.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class RiwayatJabatan extends Model
{
    use HasFactory;

    protected $table = 'riwayat_jabatan';

    protected $fillable = [
        'id_user',
        'id_jabatan',
        'tanggal_mulai',
        'tanggal_selesai',
    ];

    public function user()
    {
        return $this->belongsTo(User::class, 'id_user');
    }

    public function jabatan()
    {
        return $this->belongsTo(Jabatan::class, 'id_jabatan');
    }
}
```

models/Shift.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Carbon\Carbon;

class Shift extends Model
{
    use HasFactory;

    protected $table = 'shift';

    protected $fillable = [
        'nama',
        'waktu_mulai',
        'waktu_selesai',
        'senin',
    ];
}
```



```

        'selasa',
        'rabu',
        'kamis',
        'jumat',
        'sabtu',
        'minggu',
        'tanggal_mulai',
        'tanggal_berakhir',
        'description',
    ];

    public function user():
    \Illuminate\Database\Eloquent\Relations\HasManyThrough
    {
        return $this->hasManyThrough(User::class, Shift::class,
            'id_shift', 'id', 'id', 'id_user')
            ->where('shift.tanggal_berakhir', null);
    }

    public function getNamaFormattedAttribute(): string
    {
        return $this->nama . ' (' .
            Carbon::parse($this->waktu_mulai)->format('H:i') . '
- ' .
            Carbon::parse($this->waktu_selesai)->format('H:i') .
        ')';
    }

    public function isActiveOn(string $day): bool
    {
        $dayMap = [
            'senin' => $this->senin,
            'selasa' => $this->selasa,
            'rabu' => $this->rabu,
            'kamis' => $this->kamis,
            'jumat' => $this->jumat,
            'sabtu' => $this->sabtu,
            'minggu' => $this->minggu,
        ];

        return $dayMap[strtolower($day)] ?? false;
    }

    public function getDurationAttribute(): float
    {
        $start = Carbon::parse($this->waktu_mulai);
        $end = Carbon::parse($this->waktu_selesai);

        if ($end->lessThan($start)) {
            $end->addDay(); // Handles shifts that cross midnight.
        }

        return $end->diffInMinutes($start) / 60;
    }

    public function users()

```

```
{
    return $this->belongsToMany(User::class,
'penjadwalan_shift', 'id_shift', 'id_user');
}
}
```

models/Tunjangan.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Tunjangan extends Model
{
    use HasFactory;

    protected $table = 'tunjangan';

    protected $fillable = [
        'id_payroll',
        'nama',
        'nominal',
        'status'
    ];

    protected $casts = [
        'status' => 'boolean',
        'nominal' => 'decimal:2',
    ];

    public function payroll()
    {
        return $this->belongsTo(Payroll::class, 'id_payroll');
    }
}
```

Controllers/AttendanceController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Attendance;
use App\Models\AttendanceDetail;
use App\Models\User;
use Illuminate\Http\Request;
use Carbon\Carbon;
use App\Traits\ImageStorage;
use Symfony\Component\HttpFoundation\Response;
use Illuminate\Support\Facades\Auth;
```

```

use App\Models\Kalender;
use App\Models\PenjadwalanShift;

class AttendanceController extends Controller
{
    /**
     * Tampilkan daftar Attendance.
     */

    /**
     * Form untuk membuat Attendance baru.
     */
    use ImageStorage;
    public function index()
    {
        // Contoh minimal:
        $attendances = Attendance::all();
        return view('pages.attendance.index',
compact('attendances'));
    }

    /**
     * Tampilkan form create attendance.
     */
    public function create()
    {
        // Form untuk absen. Bisa menampilkan form 'type' (in/out),
lat/long, dsb
        return view('pages.attendance.create');
    }

    public function store(Request $request)
    {
        // Validasi
        $request->validate([
            'long' => 'required',
            'lat' => 'required',
            'address' => 'required',
            'type' => 'required|in:in,out',
            'photo' => 'required|file|image',
        ]);

        $user = Auth::user(); // User yg login
        $now = Carbon::now('Asia/Jakarta');
        $tanggal = $now->format('Y-m-d');
        $hari = strtolower($now->format('l')); // monday,
tuesday, dll
        $type = $request->type;

        // Cek Hari Libur di tabel 'kalenders' (tipe=hari_libur)
        $isLibur = Kalender::where('tipe', 'hari_libur')
->whereDate('tanggal_mulai', '<=', $tanggal)
->where(function($q) use($tanggal) {
            $q->whereNull('tanggal_selesai')
->orWhere('tanggal_selesai', '>=', $tanggal);
        })->exists();
    }
}

```

```

        // Jika hari libur, user tidak bisa absen
        if($isLibur){
            return redirect()->back()
                ->with('error','Hari ini libur. Tidak bisa
absen.');
```

```

        }

// 1. Ambil nama hari (bahasa Inggris) dari Carbon
$dayEnglish = strtolower($now->format('l')); // "monday",
"tuesday", dsb

// 2. Peta ke bahasa Indonesia (sesuaikan kolom di DB)
$map = [
    'monday' => 'senin',
    'tuesday' => 'selasa',
    'wednesday' => 'rabu',
    'thursday' => 'kamis',
    'friday' => 'jumat',
    'saturday' => 'sabtu',
    'sunday' => 'minggu'
];

// 3. Ambil nama hari versi DB
$hariDb = $map[$dayEnglish];

// 4. Baru jalankan query ke penjadwalan shift
$shiftAssignment = PenjadwalanShift::where('id_user', $user->id)
    ->whereHas('shift', function($query) use($hariDb) {
        // kolom "senin", "selasa", "rabu", "kamis", dll
        $query->where($hariDb, true);
    })
    ->with('shift')
    ->first();

    // Cek SHIFT user. Asumsikan penjadwalan shift punya field
'id_user','id_shift'
    // shift nya punya field jam mulai & jam selesai + boolean
senin, selasa, dsb

    if(!$shiftAssignment){
        return redirect()->back()
            ->with('error','Anda tidak memiliki jadwal shift
hari ini.');
```

```

    }

    $shift = $shiftAssignment->shift;
    $shiftStart = Carbon::createFromTimeString($shift->waktu_mulai, 'Asia/Jakarta')
        ->setDate($now->year, $now->month, $now->day);
    $shiftEnd = Carbon::createFromTimeString($shift->waktu_selesai, 'Asia/Jakarta')
        ->setDate($now->year, $now->month, $now->day);

    // Cek attendance user hari ini

```

```

        $attendanceToday = Attendance::where('id_user',$user->id)
        ->whereDate('tanggal',$tanggal)
        ->first();

        if($type=='in')
        {
            // Check-in
            if(!$attendanceToday){
                // Hitung potensi keterlambatan
                $hariKerja = 1.0; // default
                if($now->gt($shiftStart)){
                    // telat = selisih jam
                    $diffInHours = $shiftStart-
>diffInMinutes($now)/60;
                    if($diffInHours>0 && $diffInHours<=2){
                        $hariKerja -= 0.25; // misal potong 0.25
                    } elseif($diffInHours>2){
                        $hariKerja -= 0.5; // misal potong 0.5
                        // Atau logic lain kalau telat banyak →
0
                    }
                }

                // Simpan ke attendance
                $attendance = Attendance::create([
                    'id_user'          => $user->id,
                    'tanggal'          => $now, // simpan datetime
                    'status'           => false,
                    'hari_kerja'       => $hariKerja,
                    'jumlah_jam lembur'=> 0,
                    'is_tanggal_merah' => false, // not libur
                ]);

                // Simpan detail in
                $attendance->detail()->create([
                    'type'             => 'in',
                    'long'             => $request->long,
                    'lat'              => $request->lat,
                    'photo'           => $this->uploadImage($request-
>file('photo'),$user->nama??$user->id,'attendance'),
                    'address'         => $request->address,
                ]);

                return redirect()->back()->with('success','Check-
in berhasil');
            } else {
                return redirect()->back()->with('error','Anda
sudah check-in hari ini');
            }
        }
        else {
            // type == out (check-out)
            if($attendanceToday && !$attendanceToday->status){
                // Hitung lembur
                $overtimeHours = 0;
                if($now->gt($shiftEnd)){

```

```

                                $overtimeHours = $shiftEnd-
>diffInMinutes($now)/60;
                                }

                                // Update attendance => status checkout + lembur
                                $attendanceToday->update([
                                    'status' => true,
                                    'jumlah_jam_lembur'=> $overtimeHours,
                                ]);

                                // Simpan detail out
                                $attendanceToday->detail()->create([
                                    'type' => 'out',
                                    'long' => $request->long,
                                    'lat' => $request->lat,
                                    'photo' => $this->uploadImage($request-
>file('photo'),$user->nama??$user->id,'attendance'),
                                    'address' => $request->address,
                                ]);

                                return redirect()->back()->with('success','Check-
out berhasil');
                                } else {
                                    return redirect()->back()->with('error',
$attendanceToday ? 'Anda sudah check-out hari
ini'
                                : 'Anda belum check-in');
                                }
                            }
                        }

                        private function uploadImage($file, $userName, $dir){
                            // Silakan sesuaikan logic simpan foto
                            $ext = $file->getClientOriginalExtension();
                            $filename = uniqid().'.'.$userName.'.'.$ext;
                            $file->storeAs($dir,$filename,'public');
                            return "storage/$dir/$filename";
                        }

                        /**
                         * Tampilkan satu attendance (beserta detail).
                         */
                        public function show($id)
                        {
                            $attendance = Attendance::with('detail','user')-
>findOrFail($id);
                            return view('pages.attendance.show',
compact('attendance'));
                        }

                        /**
                         * Form edit Attendance.
                         */
                        public function edit($id)
                        {
                            $attendance = Attendance::findOrFail($id);

```

```

        $users      = User::all();
        return      view('pages.attendance.edit',
compact('attendance','users'));
    }

    /**
     * Update data Attendance ke DB.
     */
    public function update(Request $request, $id)
    {
        $request->validate([
            'id_user' => 'required|exists:users,id',
            'tanggal' => 'required|date',
            'status'  => 'required|boolean',
            'hari_kerja'      => 'nullable|numeric',
            'jumlah_jam lembur' => 'nullable|numeric',
            'is_tanggal_merah' => 'required|boolean',
        ]);

        $attendance = Attendance::findOrFail($id);
        $attendance->update($request->all());

        return redirect()->route('attendance.index')-
>with('success','Attendance berhasil diperbarui.');
```

```

    }

    /**
     * Hapus Attendance.
     */
    public function destroy($id)
    {
        $attendance = Attendance::findOrFail($id);
        $attendance->delete();
        return redirect()->route('attendance.index')-
>with('success','Attendance berhasil dihapus.');
```

```

    }
}

```

Controllers/CutiPerizinanController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\CutiPerizinan;
use App\Models\User;
use Illuminate\Http\Request;

class CutiPerizinanController extends Controller
{
    public function index()
    {

```

```

        $cutiPerizinans = CutiPerizinan::with('user')->get(['id',
'id_user', 'tanggal_mulai', 'tanggal_selesai', 'keterangan',
'status_pengajuan']);
        return view('pages.cuti_perizinan.index',
compact('cutiPerizinans'));
    }

    public function edit(CutiPerizinan $cutiPerizinan)
    {
        $users = User::all();
        return view('pages.cuti_perizinan.edit',
compact('cutiPerizinan', 'users'));
    }

    public function update(Request $request, CutiPerizinan
$cutiPerizinan)
    {
        $request->validate([
            'id_user' => 'required|exists:users,id',
            'tanggal_mulai' => 'required|date',
            'tanggal_selesai' =>
'required|date|after_or_equal:tanggal_mulai',
            'keterangan' => 'required|string',
            'jenis' => 'required|in:izin,alpa,sakit',
        ]);

        $cutiPerizinan->update($request->all());
        return redirect()->route('cuti-perizinan.index')-
>with('success', 'Data permohonan izin berhasil diperbarui.');
```

```

    }

    public function approve(CutiPerizinan $cutiPerizinan)
    {
        $cutiPerizinan->update([
            'status_pengajuan' => 'disetujui',
            'disetujui_oleh' => auth()->id() // Pastikan user sedang
login
        ]);

        return redirect()->route('cuti-perizinan.index')-
>with('success', 'Permohonan telah disetujui.');
```

```

    }

    public function reject(CutiPerizinan $cutiPerizinan)
    {
        $cutiPerizinan->update([
            'status_pengajuan' => 'ditolak',
            'disetujui_oleh' => auth()->id()
        ]);

        return redirect()->route('cuti-perizinan.index')-
>with('success', 'Permohonan telah ditolak.');
```

```

    }

    public function hasilPermohonan(Request $request)
    {

```



```

        // Filter hanya permohonan yang disetujui atau ditolak
        $status = $request->input('status');

        $query = CutiPerizinan::with('user')
            ->whereIn('status_pengajuan', ['disetujui',
            'ditolak']);

        if ($status) {
            $query->where('status_pengajuan', $status);
        }

        $cutiPerizinans = $query->orderBy('updated_at', 'desc')-
        >get();

        return view('pages.cuti_perizinan.hasil',
        compact('cutiPerizinans', 'status'));
    }

    public function undoApproval(CutiPerizinan $cutiPerizinan)
    {
        // Update status menjadi "diajukan" kembali
        $cutiPerizinan->update([
            'status_pengajuan' => 'diajukan',
            'disetujui_oleh' => null
        ]);

        return redirect()->route('cuti-perizinan.hasil')-
        >with('success', 'Status permohonan berhasil dikembalikan.');
```

```

    }

    public function destroy(CutiPerizinan $cutiPerizinan)
    {
        $cutiPerizinan->delete();
        return redirect()->route('cuti-perizinan.index')-
        >with('success', 'Permohonan izin berhasil dihapus.');
```

```

    }

    public function show(CutiPerizinan $cutiPerizinan)
    {
        return view('pages.cuti_perizinan.show',
        compact('cutiPerizinan'));
    }
}

```

Controllers/DepartemenController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Departemen;
use App\Models\Kantor;

```

```

use Illuminate\Http\Request;

class DepartemenController extends Controller
{
    public function index()
    {
        $departemen = Departemen::with('kantor')->get();
        return view('pages.departemen.index',
compact('departemen'));
    }

    public function create()
    {
        $kantor = Kantor::all();
        return view('pages.departemen.create',
compact('kantor'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'id_kantor' => 'nullable|exists:kantor,id',
            'nama' => 'required|string|max:255',
        ]);

        Departemen::create($request->all());
        return redirect()->route('departemen.index')-
>with('status', 'Departemen created successfully!');
    }

    public function edit(Departemen $departemen)
    {
        $kantor = Kantor::all();
        return view('pages.departemen.edit',
compact('departemen', 'kantor'));
    }

    public function update(Request $request, Departemen
$departemen)
    {
        $request->validate([
            'id_kantor' => 'nullable|exists:kantor,id',
            'nama' => 'required|string|max:255',
        ]);

        $departemen->update($request->all());
        return redirect()->route('departemen.index')-
>with('status', 'Departemen updated successfully!');
    }

    public function destroy(Departemen $departemen)
    {
        $departemen->delete();
        return redirect()->route('departemen.index')-
>with('status', 'Departemen deleted successfully!');
    }
}

```

```

        public function getByKantor($id)
        {
            $departemen = Departemen::where('kantor_id', $id)->get();
            return response()->json($departemen);
        }
    }
}

```

Controllers/GrupController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Grup;
use App\Models\Departemen;
use Illuminate\Http\Request;

class GrupController extends Controller
{
    public function index()
    {
        $grup = Grup::with('departemen')->get();
        return view('pages.grup.index', compact('grup'));
    }

    public function create()
    {
        $departemen = Departemen::all();
        return view('pages.grup.create', compact('departemen'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'id_departemen' => 'nullable|exists:departemen,id',
            'nama' => 'required|string|max:255',
        ]);

        Grup::create($request->all());
        return redirect()->route('grup.index')->with('status',
'Grup created successfully!');
    }

    public function edit(Grup $grup)
    {
        $departemen = Departemen::all();
        return view('pages.grup.edit', compact('grup',
'departemen'));
    }

    public function update(Request $request, Grup $grup)
    {

```

```

        $request->validate([
            'id_departemen' => 'nullable|exists:departemen,id',
            'nama' => 'required|string|max:255',
        ]);

        $grup->update($request->all());
        return redirect()->route('grup.index')->with('status',
'Grup updated successfully!');
    }

    public function destroy(Grup $grup)
    {
        $grup->delete();
        return redirect()->route('grup.index')->with('status',
'Grup deleted successfully!');
    }

    public function getByDepartemen($id)
    {
        $grup = Grup::where('departemen_id', $id)->get();
        return response()->json($grup);
    }
}

```

Controllers/HomeController.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class HomeController extends Controller
{
    /**
     * Create a new controller instance.
     *
     * @return void
     */
    public function __construct()
    {
        $this->middleware('auth');
    }

    /**
     * Show the application dashboard.
     *
     * @return \Illuminate\Contracts\Support\Renderable
     */
    public function index()
    {
        return view('home');
    }
}

```

Controllers/JabatanController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Jabatan;
use App\Models\Grup;
use Illuminate\Http\Request;

class JabatanController extends Controller
{
    public function index()
    {
        $jabatan = Jabatan::with('grup')->get();
        return view('pages.jabatan.index', compact('jabatan'));
    }

    public function create()
    {
        $grup = Grup::all();
        return view('pages.jabatan.create', compact('grup'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'id_grup' => 'nullable|exists:grup,id',
            'nama' =>
'required|string|max:255|unique:jabatan,nama',
            'description' => 'nullable|string',
        ]);

        Jabatan::create($request->all());
        return redirect()->route('jabatan.index')->with('status',
'Jabatan created successfully.');
```

```

    });

    $jabatan->update($request->all());
    return redirect()->route('jabatan.index')->with('status',
    'Jabatan updated successfully.');
```

```

    }

    public function destroy(Jabatan $jabatan)
    {
        $jabatan->delete();
        return redirect()->route('jabatan.index')->with('status',
    'Jabatan deleted successfully.');
```

```

    }

    public function getByDepartemen($id)
    {
        $jabatan = Jabatan::where('departemen_id', $id)->get();
        return response()->json($jabatan);
    }
}

```

Controllers/KalenderController.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Kalender;
use Illuminate\Support\Facades\Auth;

class KalenderController extends Controller
{
    // Display all events
    public function index()
    {
        $sevents = Kalender::orderBy('tanggal_mulai', 'asc')->get();
        return view('pages.kalender.index', compact('events'));
    }

    // Show form to create a new event
    public function create()
    {
        return view('pages.kalender.create');
    }

    // Store new event
    public function store(Request $request)
    {
        $validatedData = $this->validateRequest($request);

        $sevent = Kalender::create(array_merge($validatedData, [
            'created by' => Auth::id(),

```

```

        'updated_by' => Auth::id(),
    ]));

    $this->generateRecurringEvents($event);

        return redirect()->route('kalender.index')-
>with('success', 'Event berhasil dibuat!');
    }

    // Show details of a single event
    public function show(Kalender $kalender)
    {
        return view('pages.kalender.show', compact('kalender'));
    }

    // Show form to edit an event
    public function edit(Kalender $kalender)
    {
        return view('pages.kalender.edit', compact('kalender'));
    }

    // Update event
    public function update(Request $request, Kalender $kalender)
    {
        $validatedData = $this->validateRequest($request);

        $kalender->update(array_merge($validatedData, [
            'updated_by' => Auth::id(),
        ]));

        // Delete old repeating events if recurrence is turned
off
        if ($validatedData['repeat_type'] === 'never') {
            Kalender::where('judul', $kalender->judul)
                ->where('id', '!=', $kalender->id)
                ->delete();
        } else {
            $this->generateRecurringEvents($kalender, true);
        }

        return redirect()->route('kalender.index')-
>with('success', 'Event berhasil diperbarui!');
    }

    // Delete event
    public function destroy(Kalender $kalender)
    {
        $kalender->delete();

        return redirect()->route('kalender.index')-
>with('success', 'Event berhasil dihapus!');
    }

    /**
     * Validate the request for creating or updating an event.
     */
    private function validateRequest(Request $request)

```

```

    {
        return $request->validate([
            'tanggal_mulai' => 'required|date',
            'tanggal_selesai' =>
'nullable|date|after_or_equal:tanggal_mulai',
            'judul' => 'required|string|max:255',
            'tipe' =>
'required|in:hari_libur,meeting,acara,lainnya',
            'repeat_type' =>
'required|in:never,weekly,monthly,yearly',
            'repeat_until' =>
'nullable|date|after:tanggal_mulai|required_if:repeat_type,weekly,monthly,yearly'
        ]);
    }

    private function generateRecurringEvents(Kalender $event,
$update = false)
    {
        if ($event->repeat_type === 'never' || !$event->repeat_until) {
            return;
        }

        $startDate = $event->tanggal_mulai;
        $endDate = $event->tanggal_selesai;
        $repeatUntil = $event->repeat_until;

        $interval = match ($event->repeat_type) {
            'weekly' => '+1 week',
            'monthly' => '+1 month',
            'yearly' => '+1 year',
            default => null,
        };

        if (!$interval) return;

        // Remove old repeating events if updating
        if ($update) {
            Kalender::where('judul', $event->judul)
                ->where('id', '!=', $event->id)
                ->delete();
        }

        $nextStartDate = strtotime($interval,
strtotime($startDate));
        $nextEndDate = $endDate ? strtotime($interval,
strtotime($endDate)) : null;

        while ($nextStartDate <= strtotime($repeatUntil)) {
            Kalender::create([
                'tanggal_mulai' => date('Y-m-d', $nextStartDate),
                'tanggal_selesai' => $nextEndDate ? date('Y-m-d',
$nextEndDate) : null,
                'judul' => $event->judul,
                'tipe' => $event->tipe,
            ]);
        }
    }

```



```

        'repeat_type' => 'never',
        'created_by' => $event->created_by,
        'updated_by' => $event->updated_by,
    ]);

    $nextStartDate = strtotime($interval, $nextStartDate);
    $nextEndDate = $nextEndDate ? strtotime($interval,
$nextEndDate) : null;
    }
}
}

```

Controllers/KantorController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Kantor;
use Illuminate\Http\Request;
use App\Models\User;

class KantorController extends Controller
{
    public function index()
    {
        $kantor = Kantor::all();
        return view('pages.kantor.index', compact('kantor'));
    }

    public function create()
    {
        // Fetch only users with a jabatan containing "Manager"
        $managers = User::whereHas('riwayatJabatan', function
($query) {
            $query->whereNull('tanggal_selesai') // Only current
positions
            ->whereHas('jabatan', function ($subQuery) {
                $subQuery->where('nama', 'like',
'%Manager%'); // Jabatan contains "Manager"
            });
        })->get();

        return view('pages.kantor.create',
compact('managers'));
    }

    public function store(Request $request)
    {
        $validatedData = $this->validateKantor($request);

        Kantor::create($validatedData);
        return redirect()->route('kantor.index')->with('status',
'Kantor created successfully!');
    }
}

```

```

    }

    public function edit(Kantor $kantor)
    {
        $managers = User::whereHas('riwayatJabatan', function
($query) {
            $query->whereNull('tanggal_selesai') // Only current
positions
                ->whereHas('jabatan', function ($subQuery) {
                    $subQuery->where('nama', 'like',
'%Manager%'); // Jabatan contains "Manager"
                });
            })->get();

        return view('pages.kantor.edit', compact('kantor',
'managers'));
    }

    public function update(Request $request, Kantor $kantor)
    {
        $validatedData = $this->validateKantor($request, $kantor-
>id);

        $kantor->update($validatedData);
        return redirect()->route('kantor.index')->with('status',
'Kantor updated successfully!');
    }

    public function destroy(Kantor $kantor)
    {
        $kantor->delete();
        return redirect()->route('kantor.index')->with('status',
'Kantor deleted successfully!');
    }

    private function validateKantor(Request $request, $id = null)
    {
        return $request->validate([
            'nama' => 'required|string|max:255|unique:kantor,nama'
. ($id ? ",$id" : ''),
            'alamat' => 'required|string|unique:kantor,alamat' .
($id ? ",$id" : ''),
            'koordinat_x' => 'required|numeric|between:-180,180',
            'koordinat_y' => 'required|numeric|between:-90,90',
            'radius' => 'required|numeric|min:0',
            'id_manager' => 'nullable|exists:users,id',
        ]);
    }
}

```

Controllers/PayrollController.php

```
<?php
```

```

namespace App\Http\Controllers;

use App\Models\Attendance;
use App\Models\Payroll;
use App\Models\Tunjangan;
use App\Models\Potongan;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;

class PayrollController extends Controller
{
    public function index()
    {
        $payrolls = Payroll::with('user')->get();
        return view('pages.payroll.index', compact('payrolls'));
    }

    public function review($id)
    {
        $payroll = Payroll::with(['user', 'tunjangan', 'potongan'])->findOrFail($id);

        return view('pages.payroll.review', [
            'payroll' => $payroll,
            'tunjangan' => $payroll->tunjangan,
            'potongan' => $payroll->potongan,
        ]);
    }

    public function create()
    {
        $users = User::all();
        return view('pages.payroll.create', compact('users'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'id_user' => [
                'required',
                'exists:users,id',
                Rule::unique('payroll')->where(function ($query)
use ($request) {
                    return $query->where('tanggal_payroll',
$request->tanggal_payroll);
                })),
            ],
            'tanggal_payroll' => 'required|date',
            'gaji_pokok' => 'required|numeric',
            'upah_lembur' => 'required|numeric',
            'gaji_tgl_merah' => 'required|numeric',
            'upah_lembur_tgl_merah' => 'required|numeric',
            'iuran_bpjs_kantor' => 'required|numeric',
            'iuran_bpjs_karyawan' => 'required|numeric',
            'take_home_pay' => 'nullable|numeric',

```

```

    });

    Payroll::create($request->all());

    return redirect()->route('payroll.index')-
>with('success', 'Payroll berhasil dibuat.');
```

```

    }

    public function edit($id)
    {
        $payroll = Payroll::findOrFail($id);

        if ($payroll->is_reviewed) {
            return redirect()->route('payroll.index')-
>with('error', 'Payroll yang sudah direview tidak dapat
diedit.');
```

```

        }

        $users = User::all();
        $tunjangan = Tunjangan::where('id_payroll', $id)->get();
        $potongan = Potongan::where('id_payroll', $id)->get();
        return view('pages.payroll.edit', compact('payroll',
'users', 'tunjangan', 'potongan'));
    }

    public function update(Request $request, $id)
    {
        $payroll = Payroll::findOrFail($id);

        $request->validate([
            'id_user' => 'required|exists:users,id',
            'tanggal_payroll' => [
                'required',
                'date',
                Rule::unique('payroll')->where(function ($query)
use ($request) {
                    return $query->where('id_user', $request-
>id_user);
                })->ignore($payroll->id),
            ],
            'gaji_pokok' => 'required|numeric',
            'upah lembur' => 'required|numeric',
            'gaji_tgl_merah' => 'required|numeric',
            'upah lembur_tgl_merah' => 'required|numeric',
            'iuran_bpjs_kantor' => 'required|numeric',
            'iuran_bpjs_karyawan' => 'required|numeric',
            'take_home_pay' => 'nullable|numeric',
        ]);

        // Calculate total tunjangan and potongan
        $totalTunjangan = $payroll->tunjangan()->sum('nominal');
        $totalPotongan = $payroll->potongan()->sum('nominal');

        // Calculate final take-home pay
        $finalTakeHomePay = $request->gaji_pokok
            + $request->upah lembur

```

```

        + $request->gaji_tgl_merah
        + $request->upah lembur_tgl_merah
        + $request->iuran_bpjs_kantor
        + $totalTunjangan
        - $request->iuran_bpjs_karyawan
        - $totalPotongan;

    // Update payroll data
    $payroll->update([
        'gaji_pokok' => $request->gaji_pokok,
        'upah lembur' => $request->upah lembur,
        'gaji_tgl_merah' => $request->gaji_tgl_merah,
        'upah lembur_tgl_merah' => $request-
>upah lembur_tgl_merah,
        'iuran_bpjs_kantor' => $request->iuran_bpjs_kantor,
        'iuran_bpjs_karyawan' => $request-
>iuran_bpjs_karyawan,
        'tanggal_payroll' => $request->tanggal_payroll,
        'take_home_pay' => $finalTakeHomePay,
    ]);

    return redirect()->route('payroll.index')-
>with('success', 'Payroll berhasil diperbarui.');
```

```

    }

    public function destroy($id)
    {
        $payroll = Payroll::findOrFail($id);
        $payroll->delete();
        return redirect()->route('payroll.index')-
>with('success', 'Payroll berhasil dihapus.');
```

```

    }

    public function calculatePayroll(Request $request)
    {
        $id_user = $request->input('id_user');
        $tanggal_payroll = $request->input('tanggal_payroll');
        $UMK = $request->input('umk');

        if (!$id_user || !$tanggal_payroll || !$UMK) {
            return response()->json(['error' => 'Missing required
inputs'], 400);
        }

        $user = User::findOrFail($id_user);
        $attendances = Attendance::where('id_user', $id_user)
->whereMonth('tanggal', date('m',
strtotime($tanggal_payroll)))
->get();

        $jabatan = strtolower($user->jabatan);
        $gaji_per_hari = $UMK / 25;
        $total_hari_kerja = 0;
        $total_jam_lembur = 0;
        $total_gaji_tgl_merah = 0;
        $total_upah lembur_tgl_merah = 0;

```

```

        foreach ($attendances as $attendance) {
            $total_hari_kerja += $attendance->hari_kerja;

            if ($attendance->jumlah_jam lembur) {
                if ($attendance->is_tanggal_merah || !$attendance->status) {
                    $total_upah_lembur_tgl_merah += ($gaji_per_hari / 7) * 2 * $attendance->jumlah_jam_lembur;
                } else {
                    $total_jam_lembur += $attendance->jumlah_jam_lembur;
                }
            }

            if ($attendance->is_tanggal_merah) {
                $total_gaji_tgl_merah += $gaji_per_hari * 2 * $attendance->hari_kerja;
            }
        }

        $gaji_pokok = ($jabatan === 'staff') ? min($total_hari_kerja, 25) * $gaji_per_hari : $total_hari_kerja * $gaji_per_hari;
        $upah_lembur = $total_jam_lembur * 1.5 * ($gaji_per_hari / 7);

        $iuran_bpjs_kantor = $UMK * (0.04 + 0.0089 + 0.037 + 0.003 + 0.02);
        $iuran_bpjs_karyawan = $UMK * (0.01 + 0.02 + 0.01);

        $total_pay = $gaji_pokok + $upah_lembur + $total_gaji_tgl_merah + $total_upah_lembur_tgl_merah + $iuran_bpjs_kantor - $iuran_bpjs_karyawan;

        return response()->json([
            'gaji_pokok' => $gaji_pokok,
            'upah_lembur' => $upah_lembur,
            'gaji_tgl_merah' => $total_gaji_tgl_merah,
            'upah_lembur_tgl_merah' => $total_upah_lembur_tgl_merah,
            'iuran_bpjs_kantor' => $iuran_bpjs_kantor,
            'iuran_bpjs_karyawan' => $iuran_bpjs_karyawan,
            'total_hari_kerja' => $total_hari_kerja,
            'take_home_pay' => $total_pay,
        ]);
    }

    public function markAsReviewed($id)
    {
        $payroll = Payroll::findOrFail($id);

        // Update payroll review status
        $payroll->update([
            'is_reviewed' => true,
            'reviewed_by' => auth()->user()->id,
        ]);
    }

```

```

        'reviewed_at' => now(),
    ]);

        return redirect()->route('payroll.index')-
>with('success', 'Payroll berhasil ditandai sudah direview.');
```

```

    public function markAsPaid($id)
    {
        $payroll = Payroll::findOrFail($id);

        // Mark payroll as paid
        $payroll->update([
            'status_pembayaran' => true,
            'dibayar_at' => now(),
        ]);
    }

```

```

        return redirect()->route('payroll.index')-
>with('success', 'Payroll berhasil ditandai sudah dibayar.');
```

Controllers/PengumumanController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Pengumuman;
use App\Models\DistribusiPengumuman;
use Illuminate\Http\Request;

class PengumumanController extends Controller
{
    public function index()
    {
        // Ambil data pengumuman dengan distribusi dan creator
        $pengumuman = Pengumuman::with(['distribusi', 'creator',
'updater'])->get();

        return view('pages.pengumuman.index',
compact('pengumuman'));
    }

    public function create()
    {
        // Load semua departemen untuk pilihan distribusi
        $departemen = \App\Models\Departemen::all();
        return view('pages.pengumuman.create',
compact('departemen'));
    }

    public function store(Request $request)
    {
        $validated = $request->validate([

```

```

        'judul' => 'required|string|max:255',
        'pesan' => 'required',
        'foto' => 'nullable|image|max:2048',
        'departemen' => 'required|array', // Departemen harus
berupa array
        'departemen.*' => 'exists:departemen,id', // Setiap
ID harus ada di tabel departemen
    });

    $validated['created_by'] = auth()->id();

    if ($request->hasFile('foto')) {
        $validated['foto'] = $request->file('foto')->store('pengumuman_foto', 'public');
    }

    // Buat pengumuman baru
    $pengumuman = Pengumuman::create($validated);

    // Tambahkan distribusi ke departemen yang dipilih
    foreach ($validated['departemen'] as $id_departemen) {
        DistribusiPengumuman::create([
            'id_pengumuman' => $pengumuman->id,
            'id_departemen' => $id_departemen,
        ]);
    }

    return redirect()->route('pengumuman.index')->with('success', 'Pengumuman berhasil dibuat.');
```

```

    }

    public function edit(Pengumuman $pengumuman)
    {
        // Load semua departemen dan distribusi terkait pengumuman
        $departemen = \App\Models\Departemen::all();
        $distribusi = $pengumuman->distribusi->pluck('id_departemen')->toArray();

        return view('pages.pengumuman.edit', compact('pengumuman', 'departemen', 'distribusi'));
    }

    public function update(Request $request, Pengumuman $pengumuman)
    {
        $validated = $request->validate([
            'judul' => 'required|string|max:255',
            'pesan' => 'required',
            'foto' => 'nullable|image|max:2048',
            'departemen' => 'required|array',
            'departemen.*' => 'exists:departemen,id',
        ]);

        $validated['updated_by'] = auth()->id();

        if ($request->hasFile('foto')) {

```



```

        $validated['foto'] = $request->file('foto')->store('pengumuman_foto', 'public');
    }

    // Update pengumuman
    $pengumuman->update($validated);

    // Update distribusi pengumuman
    DistribusiPengumuman::where('id_pengumuman', $pengumuman->id)->delete();
    foreach ($validated['departemen'] as $id_departemen) {
        DistribusiPengumuman::create([
            'id_pengumuman' => $pengumuman->id,
            'id_departemen' => $id_departemen,
        ]);
    }

    return redirect()->route('pengumuman.index')->with('success', 'Pengumuman berhasil diperbarui.');
```

```

    public function destroy(Pengumuman $pengumuman)
    {
        // Hapus distribusi terkait
        DistribusiPengumuman::where('id_pengumuman', $pengumuman->id)->delete();

        // Hapus pengumuman
        $pengumuman->delete();

        return redirect()->route('pengumuman.index')->with('success', 'Pengumuman berhasil dihapus.');
```

Controllers/PotonganController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Potongan;
use Illuminate\Http\Request;

class PotonganController extends Controller
{
    public function store(Request $request, $id_payroll)
    {
        // Validate the incoming request data
        $request->validate([
            'nama' => 'required|string|max:255',
            'nominal' => 'required|numeric|min:0',
        ], [

```

```

        'nama.required' => 'Nama potongan harus diisi.',
        'nominal.required' => 'Nominal potongan harus diisi.',
        'nominal.numeric' => 'Nominal harus berupa angka.',
    ]);

    // Check if a potongan with the same name already exists
    for this payroll
        if (Potongan::where('id_payroll', $id_payroll)-
>where('nama', $request->nama)->exists()) {
            return redirect()->back()->withErrors(['error' =>
'Potongan ini sudah ada untuk payroll ini.']);
        }

        // Create the new potongan
        Potongan::create([
            'id_payroll' => $id_payroll,
            'nama' => $request->nama,
            'nominal' => $request->nominal,
        ]);

        return back()->with('success', 'Potongan berhasil
ditambahkan.');
```

```

    }

    public function edit($id)
    {
        $spotongan = Potongan::findOrFail($id);
        return view('potongan.edit', compact('potongan'));
    }

    public function update(Request $request, $id)
    {
        // Validate the incoming request data
        $request->validate([
            'nama' => 'required|string|max:255',
            'nominal' => 'required|numeric|min:0',
        ]);

        // Find and update the existing potongan
        $spotongan = Potongan::findOrFail($id);
        $spotongan->update([
            'nama' => $request->nama,
            'nominal' => $request->nominal,
        ]);

        return redirect()->route('payroll.edit', $spotongan-
>id_payroll)->with('success', 'Potongan berhasil diperbarui.');
```

```

    }

    public function destroy($id)
    {
        // Find and delete the potongan
        $spotongan = Potongan::findOrFail($id);
        $spotongan->delete();
    }

```

```

        return back()->with('success', 'Potongan berhasil
dihapus.');
```

Controllers/RiwayatJabatanController.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Jabatan;
use App\Models\RiwayatJabatan;
use App\Models\User;

class RiwayatJabatanController extends Controller
{
    public function create($user_id)
    {
        $user = User::findOrFail($user_id);
        $jabatanList = Jabatan::with(['grup.departemen.kantor'])->get();

        return view('pages.riwayat_jabatan.create', compact('user', 'jabatanList'));
    }

    public function store(Request $request, $user_id)
    {
        $validatedData = $this->validateData($request);
        $validatedData['id_user'] = $user_id;

        $existingActiveJabatan = RiwayatJabatan::where('id_user', $user_id)
            ->whereNull('tanggal_selesai')
            ->exists();

        if ($existingActiveJabatan && $request->tanggal_selesai == null) {
            return redirect()->back()->withErrors(['error' => 'Karyawan sudah memiliki jabatan yang aktif. Harap berikan tanggal berakhirnya jabatan saat ini sebelum menambahkan jabatan baru.']);
        }

        RiwayatJabatan::create($validatedData);

        return redirect()->route('user.edit', $user_id)->with('status', 'Riwayat Jabatan added successfully!');
    }

    public function edit($user_id, $id)
    {

```

```

        $user = User::findOrFail($user_id);
        $riwayatJabatan = RiwayatJabatan::findOrFail($id);
        $jabatanList = Jabatan::with(['grup.departemen.kantor'])->get();

        return view('pages.riwayat_jabatan.edit', compact('user', 'riwayatJabatan', 'jabatanList'));
    }

    public function update(Request $request, $user_id, $id)
    {
        $validatedData = $this->validateData($request);
        $riwayatJabatan = RiwayatJabatan::findOrFail($id);

        $existingActiveJabatan = RiwayatJabatan::where('id_user', $user_id)
            ->whereNull('tanggal_selesai')
            ->where('id', '!=', $id)
            ->exists();

        if ($existingActiveJabatan && $request->tanggal_selesai == null) {
            return redirect()->back()->withErrors(['error' => 'Karyawan sudah memiliki jabatan yang aktif. Harap berikan tanggal berakhirnya jabatan saat ini sebelum menambahkan jabatan baru.']);
        }

        $riwayatJabatan->update($validatedData);

        return redirect()->route('user.edit', $user_id)->with('status', 'Riwayat Jabatan updated successfully!');
    }

    public function destroy($user_id, $id)
    {
        $riwayatJabatan = RiwayatJabatan::findOrFail($id);
        $riwayatJabatan->delete();

        return redirect()->route('user.edit', $user_id)->with('status', 'Riwayat Jabatan deleted successfully!');
    }

    private function validateData(Request $request)
    {
        return $request->validate([
            'id_jabatan' => 'required|exists:jabatan,id',
            'tanggal_mulai' => 'required|date|before_or_equal:today',
            'tanggal_selesai' => 'nullable|date|after_or_equal:tanggal_mulai|before_or_equal:today',
        ]);
    }
}

```

Controllers/ShiftController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Shift;
use App\Models\User;
use App\Models\Departemen;
use App\Models\PenjadwalanShift;

class ShiftController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $shifts = Shift::all();
        return view('pages.shift.index', compact('shifts'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        $departments = Departemen::all();
        return view('pages.shift.create', compact('departments'));
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama' => 'required|unique:shift|max:255',
            'waktu_mulai' => 'required',
            'waktu_selesai' => 'required',
            'senin' => 'boolean',
            'selasa' => 'boolean',
            'rabu' => 'boolean',
            'kamis' => 'boolean',
            'jumat' => 'boolean',
            'sabtu' => 'boolean',
            'minggu' => 'boolean',
            'tanggal_mulai' => 'nullable|date',
            'tanggal_berakhir' => 'nullable|date',
            'description' => 'nullable|string',
        ]);
    }
}
```

```

        Shift::create($validated);

        return redirect()->route('shift.index')->with('success',
'Shift created successfully.');
```

}

```

/**
 * Display the specified resource.
 */
public function show(Shift $shift)
{
    return view('pages.shift.show', compact('shift'));
}

/**
 * Show the form for editing the specified resource.
 */
public function edit(Shift $shift)
{
    return view('pages.shift.edit', compact('shift'));
}

/**
 * Update the specified resource in storage.
 */
public function update(Request $request, Shift $shift)
{
    $validated = $request->validate([
        'nama' => 'required|max:255|unique:shift,nama,' .
$shift->id,
        'waktu_mulai' => 'required',
        'waktu_selesai' => 'required',
        'senin' => 'boolean',
        'selasa' => 'boolean',
        'rabu' => 'boolean',
        'kamis' => 'boolean',
        'jumat' => 'boolean',
        'sabtu' => 'boolean',
        'minggu' => 'boolean',
        'tanggal_mulai' => 'nullable|date',
        'tanggal_berakhir' => 'nullable|date',
        'description' => 'nullable|string',
    ]);

    $shift->update($validated);

    return redirect()->route('shift.index')->with('success',
'Shift updated successfully.');
```

}

```

/**
 * Remove the specified resource from storage.
 */
public function destroy(Shift $shift)
{

```

```

        $shift->delete();

        return redirect()->route('shift.index')->with('success',
'Shift deleted successfully.');
```

}

```

/**
 * Show the form for assigning users to a shift.
 */
public function assignForm($shiftId)
{
    $shift = Shift::findOrFail($shiftId);
    $users = User::all(); // Fetch all users to select from
    $assignedUsers = PenjadwalanShift::where('id_shift',
$shiftId)->pluck('id_user')->toArray();

    return view('pages.shift.assign', compact('shift',
'users', 'assignedUsers'));
}

/**
 * Assign users to a shift.
 */
public function assign(Request $request, $id)
{
    $shift = Shift::findOrFail($id);

    // Get user IDs from the request (default to an empty
array if not provided)
    $userIds = $request->input('users', []);

    // Remove all current assignments for this shift
    PenjadwalanShift::where('id_shift', $shift->id)-
>delete();

    // Assign users to the shift if any are provided
    if (!empty($userIds)) {
        foreach ($userIds as $userId) {
            PenjadwalanShift::create([
                'id_user' => $userId,
                'id_shift' => $shift->id,
            ]);
        }
    }

    return redirect()->route('shift.index')->with(
        'success',
        !empty($userIds) ? 'Users assigned to the shift
successfully.' : 'All users unassigned from the shift.'
    );
}

/**
 * Unassign a user from a shift.
 */
public function unassign($shiftId, $userId)
```

```

    {
        $assignment = PenjadwalanShift::where('id_shift',
$shiftId)
        ->where('id_user', $userId)
        ->first();

        if ($assignment) {
            $assignment->delete();
            return redirect()->back()->with('success', 'User
unassigned from shift successfully.');
```

```

        }

        return redirect()->back()->with('error', 'User is not
assigned to this shift.');
```

```

    }
}
```

Controllers/TunjanganController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Tunjangan;
use Illuminate\Http\Request;

class TunjanganController extends Controller
{
    public function store(Request $request, $id_payroll)
    {
        // Validate the incoming request data
        $request->validate([
            'nama' => 'required|string|max:255',
            'nominal' => 'required|numeric|min:0',
        ], [
            'nama.required' => 'Nama tunjangan harus diisi.',
            'nominal.required' => 'Nominal tunjangan harus
diisi.',
            'nominal.numeric' => 'Nominal harus berupa angka.',
        ]);

        // Check if the tunjangan with the same name already
exists
        if (Tunjangan::where('id_payroll', $id_payroll)-
>where('nama', $request->nama)->exists()) {
            return redirect()->back()->withErrors(['error' =>
'Tunjangan ini sudah ada untuk payroll ini.']);
        }

        // Create the tunjangan
        Tunjangan::create([
            'id_payroll' => $id_payroll,
            'nama' => $request->nama,
```



```

        'nominal' => $request->nominal,
    ]);

    return back()->with('success', 'Tunjangan berhasil
    ditambahkan.');
```

```

    }

    public function edit($id)
    {
        $tunjangan = Tunjangan::findOrFail($id);
        return view('tunjangan.edit', compact('tunjangan'));
    }

    public function update(Request $request, $id)
    {
        // Validate the incoming request data
        $request->validate([
            'nama' => 'required|string|max:255',
            'nominal' => 'required|numeric|min:0',
        ]);

        // Find and update the tunjangan
        $tunjangan = Tunjangan::findOrFail($id);
        $tunjangan->update([
            'nama' => $request->nama,
            'nominal' => $request->nominal,
        ]);

        return redirect()->route('payroll.edit', $tunjangan-
        >id_payroll)->with('success', 'Tunjangan berhasil diperbarui.');
```

```

    }

    public function destroy($id)
    {
        // Find and delete the tunjangan
        $tunjangan = Tunjangan::findOrFail($id);
        $tunjangan->delete();

        return back()->with('success', 'Tunjangan berhasil
        dihapus.');
```

```

    }
}

```

Controllers/UserController.php

```

<?php

namespace App\Http\Controllers;

use App\Traits\ImageStorage;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

```

```

use Yajra\DataTables\Facades\DataTables;
use App\Models\RiwayatJabatan;
use App\Models\Grup;
use App\Models\Departemen;
use App\Models\Kantor;

class UserController extends Controller
{
    use ImageStorage;

    public function __construct()
    {
        $this->middleware(['auth', 'is_admin']);
    }

    /**
     * Display a listing of the resource.
     */
    public function index(Request $request)
    {
        if ($request->ajax()) {
            try {
                $data = User::with(['riwayatJabatan',
                'departemen'])
                ->where('is_archived', false);

                return DataTables::of($data)
                ->addColumn('action', function ($data) {
                    return view('layouts._action', [
                        'model' => $data,
                        'edit_url' => route('user.edit',
                $data->id),
                        'show_url' => route('user.show',
                $data->id),
                        'archive_url' => route('user.archive',
                $data->id),
                        'delete_url' => route('user.destroy',
                $data->id),
                    ]);
                })
                ->addIndexColumn()
                ->rawColumns(['action'])
                ->toJson();
            } catch (\Exception $e) {
                return response()->json([
                    'error' => 'Server error: ' . $e->getMessage()
                ], 500);
            }
        }

        // Menggunakan eager loading di query utama
        $users = User::with(['riwayatJabatan', 'departemen'])
        ->where('is_archived', false)
        ->paginate(10); // Menggunakan pagination untuk
        optimasi lebih lanjut
    }
}

```

```

        return view('pages.user.index', compact('users'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        return view('pages.user.create');
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        $data = $this->validateUserRequest($request);

        $data = $this->handleFileUploads($request, $data);
        $data['password'] = Hash::make($request->password);

        User::create($data);

        return redirect()->route('user.index');
    }

    /**
     * Display the specified resource.
     */
    public function show($id)
    {
        $user = User::with('riwayatJabatan')->findOrFail($id);
        return view('pages.user.show', compact('user'));
    }

    /**
     * Show the form for editing the specified resource.
     */
    public function edit($id)
    {
        $user = User::with('riwayatJabatan')->findOrFail($id);
        return view('pages.user.edit', compact('user'));
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, $id)
    {
        $user = User::findOrFail($id);

        $data = $this->validateUserRequest($request, $user);
        $data = $this->handleFileUploads($request, $data, $user);

        if ($request->password) {
            $data['password'] = Hash::make($request->password);
        }
    }

```

```

    }

    $user->update($data);

    return redirect()->route('user.index');
}

/**
 * Remove the specified resource from storage.
 */
public function destroy($id)
{
    $user = User::find($id);

    if ($user->foto_profil) {
        $this->deleteImage($user->foto_profil, 'profile');
    }

    $user->delete();

    return redirect()->route('user.index');
}

public function archive($id)
{
    $user = User::findOrFail($id);
    $user->update(['is_archived' => true]);
    return redirect()->route('user.index')->with('status',
'User archived successfully!');
}

public function archivedUsers(Request $request)
{
    if ($request->ajax()) {
        $data = User::where('is_archived', true)->get();

        return DataTables::of($data)
            ->addColumn('action', function ($data) {
                return view('layouts._action', [
                    'model' => $data,
                    'restore_url' => route('user.restore',
$data->id),
                    'delete_url' => route('user.destroy',
$data->id),
                ]);
            })
            ->addIndexColumn()
            ->rawColumns(['action'])
            ->toJson();
    }

    $users = User::where('is_archived', true)->get();
    return view('pages.user.archived', compact('users'));
}

public function restore($id)

```

```

    {
        $user = User::findOrFail($id);
        $user->update(['is_archived' => false]);
        return redirect()->route('user.archived')->with('status',
        'User restored successfully!');
    }

    /**
     * Validate user request data.
     */
    private function validateUserRequest(Request $request, $user
= null)
    {
        $rules = [
            'id_atasan' => 'nullable|exists:users,id',
            'nama' => 'required|string|max:255',
            'nik' => 'required|string|size:16|unique:users,nik'
. ($user ? ",{$user->id}" : ''),
            'email' => 'required|email|max:255|unique:users,email'
. ($user ? ",{$user->id}" : ''),
            'npwp' => 'nullable|string|size:16|unique:users,npwp'
. ($user ? ",{$user->id}" : ''),
            'password' => $user ? 'nullable|min:8' :
'required|min:8',
            'no_telepon' =>
'nullable|string|max:15|unique:users,no_telepon' . ($user ?
",{$user->id}" : ''),
            'jenis_kelamin' => 'required|in:P,L',
            'tempat_lahir' => 'nullable|string|max:255',
            'tanggal_lahir' => 'nullable|date',
            'tanggal_perekrutan' => 'nullable|date',
            'tanggal_pemutusan_kontrak' =>
'nullable|date|after_or_equal:tanggal_perekrutan',
            'agama' => 'nullable|string|max:255',
            'alamat' => 'nullable|string|max:255',
            'rt' => 'nullable|string|max:10',
            'rw' => 'nullable|string|max:10',
            'kelurahan' => 'nullable|string|max:255',
            'kecamatan' => 'nullable|string|max:255',
            'kabupaten_kota' => 'nullable|string|max:255',
            'foto_profil' =>
'nullable|image|mimes:jpeg,png,jpg|max:2048',
            'foto_ktp' =>
'nullable|image|mimes:jpeg,png,jpg|max:2048',
            'foto_bpjs_kesehatan' =>
'nullable|image|mimes:jpeg,png,jpg|max:2048',
            'foto_bpjs_ketenagakerjaan' =>
'nullable|image|mimes:jpeg,png,jpg|max:2048',
            'is_aktif' => 'nullable|boolean',
            'is_admin' => 'nullable|boolean',
            'is_archived' => 'nullable|boolean',
            'is_remote' => 'nullable|boolean',
            'email_verified_at' => 'nullable|date',
        ];

        return $request->validate($rules);
    }

```

```

    }

    /**
     * Handle file uploads for the user.
     */
    private function handleFileUploads(Request $request, array
    $data, $user = null)
    {
        $files = [
            'foto_profil' => 'profile',
            'foto_ktp' => 'ktp',
            'foto_bpjs_kesehatan' => 'bpjs_kesehatan',
            'foto_bpjs_ketenagakerjaan' => 'bpjs_ketenagakerjaan',
        ];

        foreach ($files as $field => $path) {
            if ($request->hasFile($field)) {
                if ($user && $user->$field) {
                    \Storage::disk('public')->delete($user->
                    $field);
                }
                $data[$field] = $request->file($field)->
                store($path, 'public');
            }
        }

        return $data;
    }
}

```

routes/web.php

```

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\{
    JabatanController,
    RiwayatJabatanController,
    GrupController,
    DepartemenController,
    KantorController,
    ShiftController,
    UserController,
    PengumumanController,
    CutiPerizinanController,
    KalenderController,
    AttendanceController,
    PayrollController,
    TunjanganController,
    PotonganController,
    HomeController
};

```

```

// Authentication Routes
Auth::routes();

Route::get('/', function () {
    return view('welcome');
});

// Home Route
Route::get('/home', [HomeController::class, 'index'])->
>name('home');

// Protect all web routes for admin access only
Route::middleware(['auth', 'is_admin'])->group(function () {
    // User Management
    Route::patch('user/{id}/archive', [UserController::class,
'archive'])->name('user.archive');
    Route::get('user/archived', [UserController::class,
'archivedUsers'])->name('user.archived');
    Route::patch('user/{id}/restore', [UserController::class,
'restore'])->name('user.restore');
    Route::resource('user', UserController::class);

    // Attendance
    Route::resource('attendance', AttendanceController::class);

    // Shift Management
    Route::get('shift/{shift}/assign', [ShiftController::class,
'assignForm'])->name('shift.assignForm');
    Route::post('shift/{shift}/assign', [ShiftController::class,
'assign'])->name('shift.assign');
    Route::resource('shift', ShiftController::class);

    // Jabatan, Grup, Departemen, Kantor
    Route::resource('jabatan', JabatanController::class);
    Route::resource('grup', GrupController::class);
    Route::resource('departemen', DepartemenController::class)-
>parameters([
        'departemen' => 'departemen'
    ]);
    Route::resource('kantor', KantorController::class);

    Route::get('/kantor/{kantor}/edit',
[KantorController::class, 'edit'])->name('kantor.edit');

    // Riwayat Jabatan
    Route::prefix('riwayat_jabatan')->name('riwayat_jabatan.')-
>group(function () {
        Route::get('/create/{user_id}',
[RiwayatJabatanController::class, 'create'])->name('create');
        Route::post('/store/{user_id}',
[RiwayatJabatanController::class, 'store'])->name('store');
        Route::get('/edit/{user_id}/{id}',
[RiwayatJabatanController::class, 'edit'])->name('edit');
        Route::put('/update/{user_id}/{id}',
[RiwayatJabatanController::class, 'update'])->name('update');
    });

```

```

Route::delete('/destroy/{user_id}/{id}',
[RiwayatJabatanController::class, 'destroy'])->name('destroy');
});

// Pengumuman
Route::resource('pengumuman', PengumumanController::class);

// Cuti & Perizinan
Route::get('cuti-perizinan/hasil-permohonan',
[CutiperizinanController::class, 'hasilPermohonan'])->
>name('cuti-perizinan.hasil');
Route::post('cuti-perizinan/{cutiperizinan}/approve',
[CutiperizinanController::class, 'approve'])->name('cuti-
perizinan.approve');
Route::post('cuti-perizinan/{cutiperizinan}/reject',
[CutiperizinanController::class, 'reject'])->name('cuti-
perizinan.reject');
Route::post('cuti-perizinan/{cutiperizinan}/undo',
[CutiperizinanController::class, 'undoApproval'])->name('cuti-
perizinan.undo');
Route::resource('cuti-perizinan',
CutiperizinanController::class)->except(['create']);

// Kalender
Route::resource('kalender', KalenderController::class);

// Payroll
Route::prefix('payroll')->name('payroll.')->group(function
() {
Route::put('{id}/mark-as-paid',
[PayrollController::class, 'markAsPaid'])->name('markAsPaid');
Route::get('{id}/review', [PayrollController::class,
'Review'])->name('review');
Route::put('{id}/review', [PayrollController::class,
'markAsReviewed'])->name('review.submit');
Route::get('calculate', [PayrollController::class,
'calculatePayroll'])->name('calculate');
});

Route::resource('payroll', PayrollController::class);

// Tunjangan & Potongan
Route::prefix('tunjangan')->name('tunjangan.')->
>group(function () {
Route::post('/store/{id_payroll}',
[TunjanganController::class, 'store'])->name('store');
Route::put('/{id}', [TunjanganController::class,
'update'])->name('update');
Route::delete('/{id}', [TunjanganController::class,
'destroy'])->name('destroy');
});

Route::prefix('potongan')->name('potongan.')->group(function
() {
Route::post('/store/{id_payroll}',
[PotonganController::class, 'store'])->name('store');

```



```

        Route::put('/{id}', [PotonganController::class,
'update'])->name('update');
        Route::delete('/{id}', [PotonganController::class,
'destroy'])->name('destroy');
    });
});

```

routes/api.php

```

<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\Auth\AuthController;
use App\Http\Controllers\Api\Auth>PasswordController;
use App\Http\Controllers\Api\AttendanceController;
use App\Http\Controllers\Api\UserController;
use App\Http\Controllers\Api\PengumumanController;
use App\Http\Controllers\Api\CutiPerizinanController;
use App\Http\Controllers\Api\KalenderController;
use App\Http\Controllers\Api\ShiftController;
use App\Http\Controllers\Api\PayrollController;

/*
|-----
|
| API Routes
|-----
|
| Here is where you can register API routes for your application.
These
| routes are loaded by the RouteServiceProvider and all of them
will
| be assigned to the "api" middleware group. Make something great!
|
*/

// Auth Routes (Public)
Route::prefix('auth')->group(function () {
    Route::post('/register', [AuthController::class,
'register']);
    Route::post('/login', [AuthController::class, 'login']);
    Route::post('/password/forgot', [PasswordController::class,
'sendResetLinkEmail']);
});

// Protected Routes (Require Authentication)
Route::middleware('auth:sanctum')->group(function () {

    // Authenticated User Info
    Route::get('/user', function (Request $request) {
        return $request->user();
    });
});

```

```

        // Authenticated Actions
        Route::post('/auth/logout', [AuthController::class,
'logout']);

        Route::post('/auth/password/reset',
[PasswordController::class, 'reset']);

        // User Routes
        Route::prefix('users')->group(function () {
            Route::get('/', [UserController::class, 'index']);
            Route::get('/{id}', [UserController::class, 'show']);
            Route::put('/{id}', [UserController::class, 'update']);
            Route::delete('/{id}', [UserController::class,
'destroy']);
            Route::get('/{userId}/shifts', [UserController::class,
'getShiftsByUser']);
            Route::get('/{userId}/payroll',
[PayrollController::class, 'getPayrollByUserId']);
        });

        // Attendance Routes
        Route::prefix('attendance')->group(function () {
            Route::post('/', [AttendanceController::class, 'store']);
            Route::get('/history', [AttendanceController::class,
'history']);
        });

        // Payroll Routes
        Route::prefix('payroll')->group(function () {
            Route::get('/{payrollId}', [PayrollController::class,
'getPayrollById']);
        });

        // Announcement Routes
        Route::apiResource('pengumuman',
PengumumanController::class, array("as" => "api"));

        // Shift Routes
        Route::get('/shift/{shiftId}/users',
[ShiftController::class, 'getUsersByShift']);

        // Leave & Permission Routes (Cuti Perizinan)
        Route::prefix('cuti-perizinan')->group(function () {
            Route::get('/', [CutiPerizinanController::class,
'getAllPermohonan']);
            Route::get('/{id}', [CutiPerizinanController::class,
'getPermohonanById']);
            Route::post('/', [CutiPerizinanController::class,
'store']);
            Route::put('/{id}', [CutiPerizinanController::class,
'update']);
        });

        // Calendar Routes
        Route::get('/kalender', [KalenderController::class,
'index']);

```

```
} ) ;
```